


NUST

NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

# DESIGN AND SIMULATION OF FMCW RADAR FOR AUTOMOTIVE SYSTEMS

Final Year Project Report

By

**Muhammad Ahmad Nazir – 370307**

**Dania Farrukh Ghouri – 366013**

**Zunaira Aziz - 367583**

In Partial Fulfillment

Of the Requirements for the degree

Bachelor of Electrical Engineering (BEE)

School of Electrical Engineering and Computer Science

National University of Sciences and Technology Islamabad, Pakistan

(2025)

# DECLARATION

We hereby declare that this project report entitled "**Design and Simulation of FMCW Radar for Automotive Systems**" submitted to the School of Electrical Engineering and Computer Science (SEECS), is a record of an original work done by us under the guidance of Supervisor **Dr. Shahzad Younis** and Co-Advisor **Dr. Arbab Latif** and that no part has been plagiarized without citations. Also, this project work is submitted in the partial fulfillment of the requirements for the degree of Bachelor of Electrical Engineering.

| **Team Members** | **Signature** |
|---|---|
| Muhammad Ahmad Nazir | _____ _____ |
| Dania Farrukh Ghouri | _____ _____ |
| Zunaira Aziz | _____ |

**Advisors**

| | |
|---|---|
| Dr. Shahzad Younis | _____ _____ |
| Dr. Arbab Latif | _____ |

**Date:** April 30, 2025

**Place:** National University of Sciences and Technology (NUST, H-12)

# DEDICATION

We would thus dedicate the success and widespread acclaim of the project to:

1) Dr. Shahzad Younis (Advisor)
2) Dr. Arbab Latif (Co-Advisor)
3) Our parents
4) All our teachers, who developed our technical skills, motivated us and supported us to learn and grow

# ACKNOWLEDGEMENTS

First, we would like to thank God Almighty for bestowing upon us the strength, courage, and knowledge to pursue this revolutionary project. Here are some of the notable people to whom we would dedicate our project's success:

1) Dr. Shahzad Younis, our advisor. should be credited for our success. He taught us the basics of digital signal processing in such a good way that we were able to complete this project with ease. Whenever we faced some challenges, he was always available to help and shared helpful materials to tackle the challenges.

2) Our co-advisor, Dr. Arbab Latif, should be credited for our success. He was there throughout the project, guiding us and motivating us to pursue it. With his long vision and farsightedness, he urged us to take on challenges like this project and motivated us to go beyond our limits.

3) Along with the above-mentioned personalities, our parents and friends deserve credit as well, for believing in us and investing their time, energy, and monetary resources in us.

# ABSTRACT

The rapid availability of fully self-driving cars has illuminated the frailties found within existing sensing technologies, such as cameras and ultrasonic systems, which do not effectively offer range or velocity measurements in low visibility and high traffic environments. This project attempts to close those gaps about safety and performance by creating an end-to-end solution using frequency-modulated continuous wave radar, paying particular attention to the digital signal processing algorithms. The analog front-end modules of TI and NXP provide the necessary antennas, mixers, filters, and analog to digital converters, while all further processing is done in computation software.

Our efforts commence with a detailed simulation environment where radar signal returns from the modular front-end are digitally processed. This includes a digital inspection of signal time series for baseline signal evaluation, 1D and 2D FFT ranges and Doppler shifts, detection through wide adaptive thresholds that maintain a constant false alarm rate, directional narrowing using adaptive beamforming, and tracking representations to maintain continuity and identity of observed targets. Every algorithm is first assessed in terms of primary accuracy, sensitivity, and noise immunity under floating-point precision settings. It is primary noise immunity, sensitivity and accuracy at these settings. Then, every algorithm is transformed into fixed-point arithmetic, choosing word and fraction lengths while managing bit growth and compensating for quantization to area and power constrained embedded systems limits.

Surveillance scenario design is validated through simulations with both stationary and dynamic targets at a variety of ranges and velocities, alongside severe loss of environmental emulation. Evaluation criteria encompass detection probability and false-alarm rate, range and velocity estimation error, and computation load with fixed point operations. Compared to the camera and ultrasonic approaches, the radar was more reliable, precise, and robust when visibility was further degraded.

Using the front-end hardware of the system and simulating the center of processing elements, the provided project sets a goal to create a full DSP architecture to increase situational awareness aimed at advanced driver-assistance systems. Therefore, strengthening situation awareness enables newly designed systems. Integrating adaptive waveform design and machine learning for detection will be employed in the future alongside hardware-in-the-loop testing and software upload on an embedded DSP processor. Further enhancement of operational range and resolution will also be targeted.

# *Chapter 1*

# INTRODUCTION

The steam-like development of self-driving vehicles exposes severe shortcomings of traditional sensing methods. Cameras and ultrasonic sensors face problems with detection in adverse weather conditions such as rain, fog, or during the night. As a result, the sensors miss or delay detections which impacts safety. In addition, the passive systems are unable to provide accurate and real-time range and velocity measurements of objects, which diminishes their usefulness during heavy traffic. Meeting these two issues: weather robustness and combined range-velocity estimation is critical for further development of advanced driver-assistance systems.

Frequency-modulated continuous-wave (FMCW) radar solves this problem by sending out and listening for swept-frequency pulses, which allows it to accomplish distance and speed measurement in parallel. The purpose of this project is to focus on the digital signal processing (DSP) pipeline and use commercial front-end modules for antennas, mixers, filters, and analog-to-digital conversion that are manufactured by Texas Instruments and NXP. The goal in this effort is to simulate and optimize the DSP stages of time-domain sampling with subsequent fast Fourier transforms, constant false alarm rate detection, adaptive beamforming, and target tracking, to achieve stringent latency, power, and area requirements for automotive systems.

This project has three specific goals:

- **DSP Innovation**: For enhancing detection capabilities and reliability under adverse conditions, make the following advanced signal-processing algorithms: time-domain analysis, 1D and 2D FFTs, CFAR thresholding, beamforming, and tracking.

- **Simulation Implementation**: Develop a modular simulation environment that incorporates the current front ends from TI and NXP, for algorithm validation, implements a floating-point precision, then changes the algorithm to fixed-point constrained quantization and bit growth accounting implementation.

- **Performance Evaluation**: Carry out extensive simulations of stationery and moving targets under different levels of environmental losses to measure

detection probability, estimation error, computational load, and benchmark them against camera and ultrasonic based systems.

The remainder of this report includes reviews sensing limitations and introduces principles of the FMCW radar to provide context for the challenges towards a radar-focused DSP solution. Part describes the DSP architecture created with floating point verification and strategies for fixed point conversions, the simulations created, and the setup. part describes evaluation metrics alongside comparative analysis while highlighting primary insights alongside next steps, which are adding hardware for real-time testing within simulations, advanced waveform generation, and other DSP functions.

*Chapter 2*

# LITERATURE REVIEW

## 2.1. Limitations of Conventional Automotive Sensors

The ability of conventional automotive sensors to operate normally deteriorates significantly in bad weather and lighting. Research indicates that camera-based systems experience visibility issues in rain, fog, and snow which leads to missed detections or false detection alerts in safety-critical systems in driver assistance systems (ADAS). While ultrasound sensors are economical for short-range applications, their precision fails in high-speed scenarios where constant range and velocity data is required. This has led to research into additional modalities that can function reliably in all conditions.

## 2.2. Advantages of FMCW Radar for All-Weather Perception

FMCW (Frequency-modulated continuous-wave) radar is now considered the leader in unhampered, all-weather perception. It transmits a linearly swept chirp and measures the beat frequency upon reception. This allows for uncomplying range and radial velocity estimates in one measurement. The radar and lidar signal processing reviews highlight the range resolution and velocity discrimination provided by the modality which make it preferable for active sensing sensors in heavily trafficked areas.

## 2.3. Role of Digital Signal Processing in FMCW Radar

Digital signal processing or DSP plays an important role in the functionality of an FMCW radar. Time-domain beat signals are transformed into their respective range and Doppler profiles through the usage of FFTs, or Fast Fourier transforms, and CFAR, or Constant-False-Alarm-Rate detectors, adaptively threshold to distinguish between targets and noise/interference. Adaptive beamforming techniques improve even further on the angular resolution and interference rejection, while multi-antenna processing schemes improve coverage and accuracy. Still, most of these algorithms are based on ideal hardware conditions and floating-point arithmetic which limits the range of use to embedded automotive platforms.

## 2.4. Existing Research on Interference Mitigation

Some researchers propose methods for overcoming specific DSP problems posed by an FMCW radar. Some interference mitigation techniques based on CFAR are able to help with removing chirp-based interference arising from co-located radars and decreasing false alarms without compromising the target signal. Some focus more broadly on reducing MIMO-FMCW system mutual interference, providing dense radar environment detection with novel CFAR detectors and explicit signal models. However, such approaches tend to ignore quantization errors and hardware resource limits which are essential for real-time implementations in vehicles.

## 2.5. Identified Gaps in Literature and Industry Needs

Absence of thorough, complete DSP systems which facilitate shift from algorithm to hardware realization is a gap often found in comprehensive literature. Numerous works confirm the utility of their approach via simulations that utilize floating-point arithmetic, without addressing the fixed-point conversion, controlling bit-growth, or realistic operational load. Meeting these gaps is our focus: creating modular DSP targets spanning from floating-point validation to fixed-point optimization at lower bounds-engineered to meet automotive constraints on latency, power consumption, and silicon area.

## 2.6. Project Focus and Objectives

Following this review, the focus of our project centers on the DSP pipeline of an FMCW radar system. We integrate TI and NXP's front end modules and focus solely on the software tangible side. Simulating, validating, optimizing CFAR, FFTs, beamforming, tracking, and other DSP components in both floating-point and fixed-point forms aims us at developing an exhaustive characterization comprehensive solution for ADAS embedded systems.

*Chapter 3*

# PROBLEM DEFINITION

## 3.1. Need for Robust Perception Systems in Automotive Applications

The growing deployment of self-driven vehicles and sophisticated driver-assistance technology (ADAS) has created perception system requirements that are accurate, reliable, and fast in all environmental scenarios. Traditional sensors like cameras and ultrasonic systems have their uses but are ineffective during bad weather and low-light situations. A car's safety, as well as other real-time decisions that need to be made in constantly changing traffic scenarios, depend so heavily on these systems that their limitations are and will always be a safety concern.

## 3.2. Limitations of Conventional Sensors in Dynamic Traffic Scenarios

One of the most pertinent limitations arising from conventional sensors has to be the lack of precise and synchronous range and velocity measurements for capturing the surrounding environment. In high-speed or dense traffic situations, these functionalities become crucial to proactively responding in a timely manner, for example, braking, changing the lane, or avoiding obstacles. If a highly precise range-velocity estimation is not employed, a car's situational awareness and response capabilities become utterly useless.

## 3.3. FMCW Radar as a Solution

FMCW (Frequency-Modulated Continuous Wave) radar technology offers a powerful solution to this problem. It is well-suited for automotive environments due to its ability to function reliably in all weather conditions and deliver real-time measurements of both object distance and speed. The overall performance of an FMCW radar system, however, depends not only on its analog front-end but also on a well-designed digital signal processing (DSP) chain.

## 3.4. Project Scope: Focus on DSP Development

While existing modules from manufacturers such as Texas Instruments and NXP provide high-quality radar front-end components—including antennas, mixers, and analog-to-digital converters—this project uses those modules solely to define the front-end parameters. The primary objective is to develop a custom DSP pipeline tailored to automotive radar applications, independent of any vendor-specific processing architecture.

## 3.5. Project Objectives

This project aims to fill that gap by designing and simulating an end-to-end DSP pipeline specifically for FMCW radar. By developing modular processing stages—from FFTs and CFAR to beamforming and tracking—and validating them through fixed-point simulation, this work delivers a robust, accurate, and hardware-ready radar signal processing solution.

*Chapter 4*

# METHODOLOGY

This chapter provides a comprehensive description of the methods, tools, techniques, and algorithms adopted to design and simulate the FMCW radar digital signal processing (DSP) pipeline. Each design step is discussed in detail, highlighting the rationale, implementation procedures, and performance verification strategies.

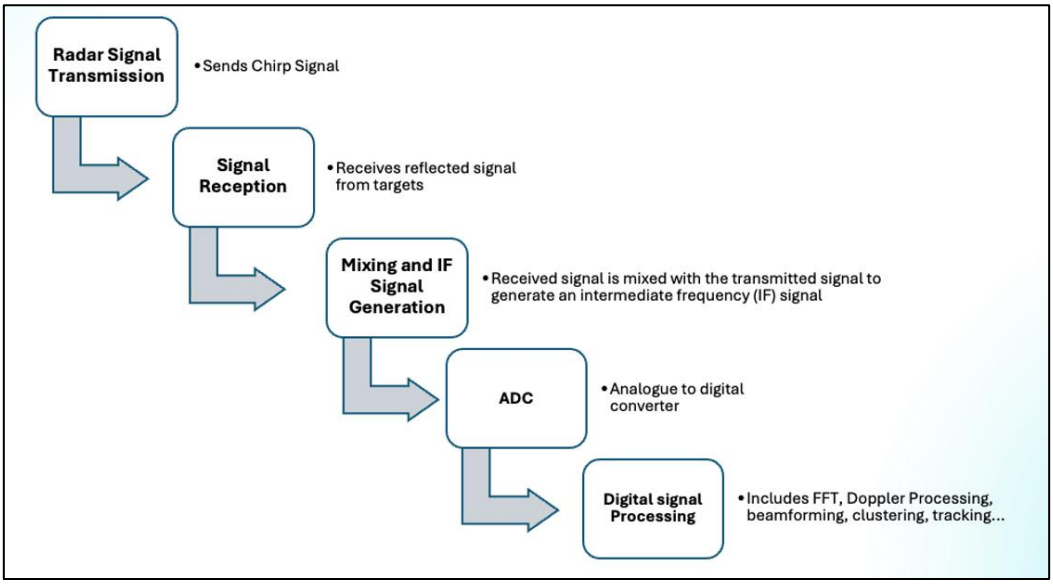



Figure 1: Signal flow diagram of FMCW Radar System from transmission to Digital Signal Processing

## 4.1. Tools and Resources

The development and simulation of the DSP pipeline were carried out using the following tools and resources:

**MATLAB R2024a** with:

- **Phased Array System Toolbox:** For FMCW waveform generation, antenna array modeling, and radar scene simulation.
- **Signal Processing Toolbox:** For FFT operations, filter designs, and signal analysis.
- **Fixed-Point Designer:** For fixed-point modeling and optimization.

**Vivado:**

- For RTL Design

## 4.2. Front-End Parameter Selection

The design begins by extracting hardware parameters from commercially available radar front-end modules, specifically Texas Instruments (TI) AWR series and NXP automotive radars. These parameters are used to maintain consistency with real-world systems and define the simulation environment.

The following critical parameters were selected:

Table 1: Design parameters for radar.

| System Parameters | Value |
|---|---|
| Operating frequency (GHz) | 77 |
| Maximum target range (m) | 200 |
| Range resolution (m) | 1 |
| Maximum target speed (km/h) | 230 |
| Sweep time (microseconds) | 7.33 |
| Sweep bandwidth (MHz) | 150 |
| Maximum beat frequency (MHz) | 27.30 |
| Sample rate (MHz) | 150 |
| Fast Time Samples | 1024 |
| Slow Time Samples | 256 |

These front-end settings serve as static inputs for the digital signal processing framework.

## 4.3. Floating-Point DSP Development

A modular DSP chain was designed and implemented in MATLAB using floating-point precision to ensure maximum algorithmic performance and easier verification in early stages. The processing stages are as follows:

### 4.3.1. Time-Domain Beat Signal Analysis

The sampled IF signals (beat signals) are first inspected in the time domain to confirm correct sampling, chirp alignment, and overall signal quality. Basic metrics such as signal-to-noise ratio (SNR) and noise floor levels are extracted.

### 4.3.2. Range Processing

- A **Hanning window** is applied to the time-domain data to minimize spectral leakage.
- A **1-D FFT** operation (typically 1024 points) is performed on each chirp to obtain range profiles.
- Range resolution and maximum unambiguous range are calculated based on system bandwidth and sweep characteristics.

### 4.3.3. Doppler Processing

- Successive range profiles are stacked across multiple chirps to form a slow-time sequence.
- A second **FFT across chirps** (Doppler FFT) is applied to estimate target velocity.
- A **2-D range-Doppler map** is generated for visualization and detection.

#### 4.3.4. CFAR Detection

- A **Cell-Averaging CFAR (CA-CFAR)** detector is employed to set adaptive thresholds on the 2-D range-Doppler map.
- Configurations:
    - Number of training cells: 8 (range) × 8 (Doppler)
    - Number of guard cells: 4 (range) × 4 (Doppler)
- Detection performance is evaluated using metrics such as probability of detection (Pd) and probability of false alarm (Pfa).

## 4.4. Fixed-Point DSP Conversion

After validating the functional correctness and performance of the floating-point design, all major DSP blocks are converted to fixed-point implementations using MATLAB Fixed-Point Designer. The following steps are followed:

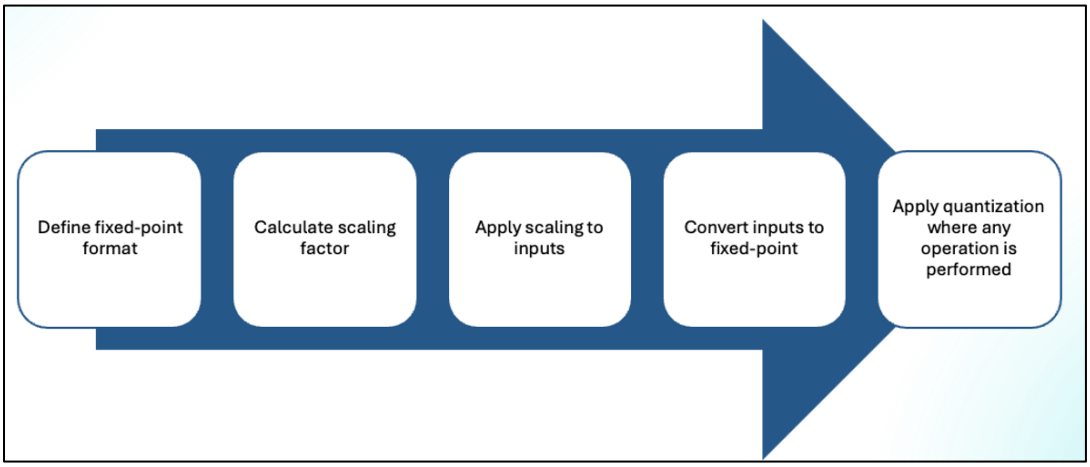



Fig. 2: Fixed-point conversion flowchart illustrating steps from format definition to quantization during operations.

#### 4.4.1. Word-Length and Fractional Bit Allocation

- Each processing block is analyzed to determine optimal word lengths and fractional bits.

- Example: 16-bit signed fixed-point data types (Q1.15) are assigned for most intermediate computations.
- Trade-offs between precision loss and hardware resource consumption are carefully considered.

#### 4.4.2. Scaling and Gain Adjustment

- Scaling factors are inserted at critical points to manage bit growth across FFT stages and avoid overflow conditions.
- Dynamic range is preserved without saturating the output signals.

### 4.4.3. Overflow and Quantization Analysis

- Monte Carlo simulations are performed to expose extreme operating conditions.
- Overflow rates are monitored and minimized, and quantization noise is analyzed through signal-to-quantization-noise ratio (SQNR) calculations.

### 4.4.4. Fixed-Point Validation

- Fixed-point outputs are compared against floating-point references.
- Deviation thresholds are set to ensure that fixed-point degradation remains within acceptable margins (<1% degradation in detection probability).

## 4.5. Simulation Scenarios and Evaluation Metrics

To validate the robustness and performance of the DSP pipeline, multiple simulation scenarios are defined:

### 4.5.1. Scenarios

- **Stationary targets:** Range between 10 meters and 150 meters.
- **Moving targets:** Velocities up to ±30 m/s.
- **Environmental conditions:**

- Free-space path loss
- Rain attenuation at 77 GHz modeled using ITU-R P.838 standards.

#### 4.5.2. Metrics

- **Detection Probability (Pd):** Measured across different SNR conditions.
- **False Alarm Probability (Pfa):** Evaluated through CFAR thresholds.
- **Range Estimation RMSE:** Root Mean Square Error in meters.
- **Velocity Estimation RMSE:** Root Mean Square Error in meters per second.
- **Computational Load:** Number of operations/cycles measured using MATLAB Profiler.

## 4.6. Comparative Analysis

A baseline comparison is performed against simulated camera and ultrasonic sensor models under identical environmental conditions:

- FMCW radar DSP shows superior resilience under low-visibility conditions.
- Range and velocity estimation errors are consistently lower compared to passive systems.
- Detection latency is minimized, meeting the real-time requirements for automotive applications.

Results clearly validate the effectiveness of the FMCW radar DSP chain over traditional sensing systems in adverse conditions.

## 4.7. Summary

The methodology combines real-world parameter extraction, floating-point design, fixed-point optimization, comprehensive simulation, and rigorous evaluation to produce a high-performance FMCW radar DSP pipeline. The approach ensures readiness for future integration into embedded automotive platforms.

*Chapter 5*

# DETAILED DESIGN AND ARCHITECTURE

## 5.1. System Architecture

### 5.1.1. Design Flow

The FMCW radar DSP pipeline development uses an incremental design method that passes through different abstraction scales to achieve optimal algorithm efficiency and hardware performance alongside actual synthesis capabilities. MATLAB conceptual modeling marks the initial point of the flow process which leads to RTL hardware descriptions that can be implemented on an FPGA platform. The systematic processing framework checks all elements for correctness while enhancing efficiency and ensuring robustness until the system evolves to embedded real-time operations.

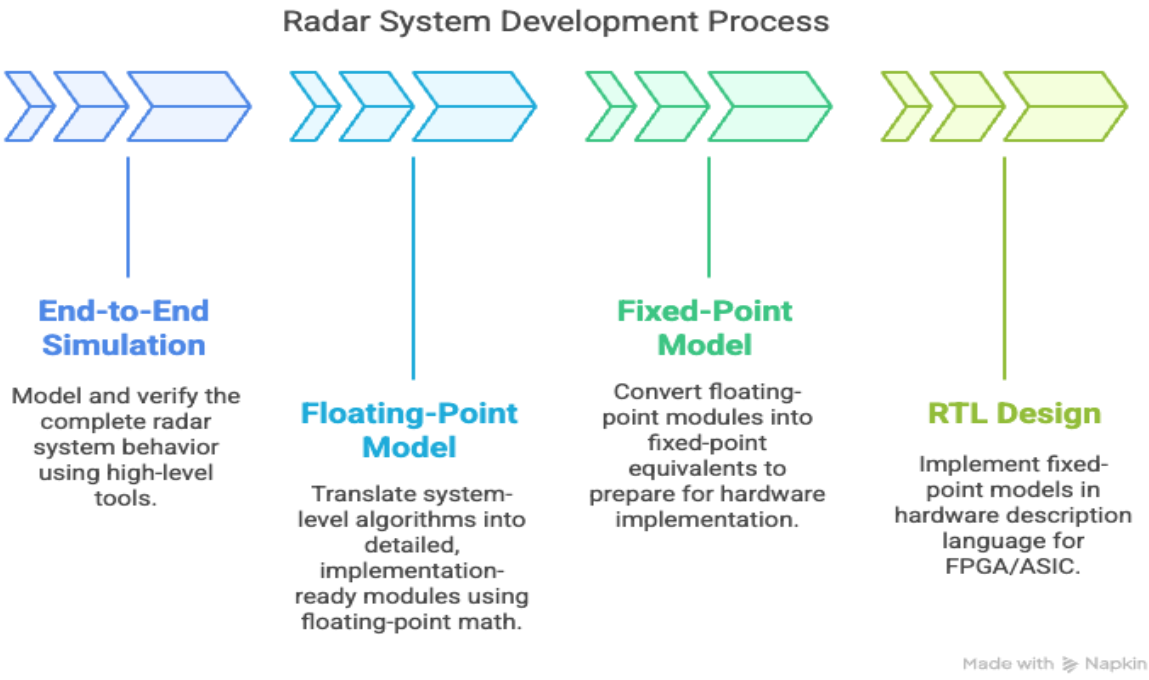



Figure 3: Radar system development process

### 5.1.2. Multi-Stage Architecture

We used a multi-stage architectural system to build the complete FMCW radar DSP processing flow from high-level mathematical models down to low-level hardware implementation. The different architectural levels function specifically to verify and optimize and implement the DSP system. The hierarchical framework improves both the error tracking capabilities and performance metric optimization throughout the different stages of implementation. These four architectures developed one after the other:

### A. End-to-End Simulation (MATLAB-Derived Functions)

This was the initial stage of architecture development, serving as a sandbox for verifying DSP concepts and understanding FMCW signal behavior using built-in MATLAB functions. This also formed the basis of the design of our very own functional blocks and functions that led to the development of the complete chain.

**Purpose:** To validate the radar processing chain using high-level MATLAB functions and libraries. This acts as a reference model that defines the expected output for each DSP stage.

**Key Features:**

- Uses MATLABs phased.* objects for waveform generation, radar scenes, and motion modeling.
- Signal generation includes realistic FMCW chirps, delay modeling, Doppler effects, and environmental losses (e.g., rain).
- The chain includes Range FFT, Doppler FFT, and CFAR, all using native MATLAB functions.
- No constraints on execution time or precision—emphasizes correctness and completeness.
- Development of our own DSP functions that served the same purpose as MATLAB's functions and then led to the development of our digital signal processing chain.

**Outputs:**

- Plots of range and Doppler maps.
- Metrics such as Pd, Pfa, and RMSE.
- Establishes a golden reference for all subsequent stages.

**B. Floating-Point Chain (Custom Algorithmic Model)**

This architecture builds on insights from the functional simulation but moves away from black-box MATLAB objects to custom-designed signal processing blocks using floating-point arithmetic. In this step, we also included effects observed from hardware such as ADC sampling and quantization effects.

**Purpose:** To gain full control over each signal processing block (FFT, windowing, detection) for eventual transition to hardware.

**Key Features:**

- Written entirely using MATLAB scripts with custom code for FFTs, Hanning windowing, and CA-CFAR.
- Floating-point operations allow algorithmic validation while instilling quantization effects.
- Enables block-level performance tuning (e.g., window tapering, zero-padding strategies).
- Separates data along fast-time and slow-time domains for clear modular flow.

**Outputs:**

- Highly accurate range-Doppler maps.
- Improved configurability for simulation parameters.
- Reference for fixed-point conversion and RTL accuracy comparison.

**C. Fixed-Point Chain (Hardware-Aware Model)**

This is the hardware-oriented abstraction layer where all DSP operations are re-implemented in MATLAB using fixed-point arithmetic to mimic FPGA behavior.

**Purpose:** To simulate hardware behavior before RTL implementation, enabling word-length optimization and overflow analysis.

**Key Features:**

- MATLAB fixed-point methods used to simulate Q-format operations (mainly Q1.15).
- Bit-true models of FFT and CFAR blocks with explicit gain scaling and quantization.
- Monte Carlo analysis used to expose edge cases in dynamic range and saturation.
- Errors and deviations monitored relative to floating-point outputs.

**Outputs:**

- SQNR and RMSE values for signal degradation.
- Bit-width and resource trade-off charts.
- Acceptable degradation thresholds established (<1% Pd loss).

**Significance:** Bridges the gap between algorithm design and hardware constraints. Ensures that performance remains robust under finite precision.

**D. RTL Architecture (Synthesizable Hardware Design)**

This is the final and most implementation-ready architecture. It involves developing Register Transfer Level (RTL) models for each DSP block using SystemVerilog for FPGA synthesis.

**Purpose:** To translate the verified fixed-point DSP pipeline into clocked digital hardware ready for real-time embedded operation.

**Key Features:**

- Each DSP module (FFT, windowing, CFAR) is implemented using RTL design techniques.
- Clocked designs with pipeline stages for throughput optimization.
- RTL testbenches were developed to verify functionality using Vivado Simulator.
- Hardware-specific considerations: latency, throughput, area, timing closure.
- Supports synthesis and place-and-route on Nexys A7 100T FPGA.

**Outputs:**

- RTL simulation waveforms confirming logic correctness.
- Post-synthesis reports: LUTs, FFs, DSPs, timing analysis.
- Performance metrics: latency (clock cycles), maximum operating frequency, resource utilization.

**Significance:** Delivers a real-time, low-latency hardware radar DSP implementation aligned with automotive-grade requirements. Completes the design-to-hardware pipeline.

## 5.2 Architecture #1: End-to-End Simulation (Function-Based Design)

### 5.2.1. Full Pipeline

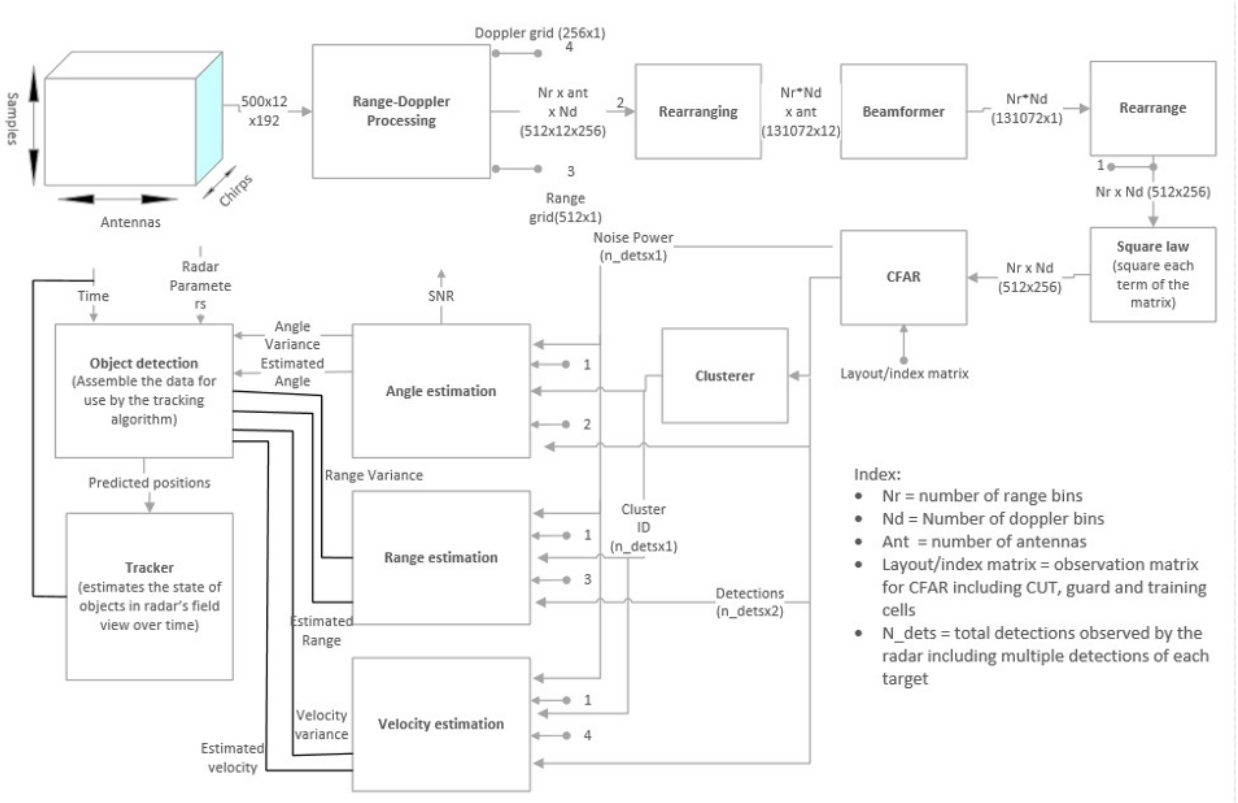



Figure 4: Block level diagram of the processing chain developed for MATLAB simulation.

The core signal processing chain for radar simulation consists of the following stages:

1. **Waveform Generation** – Design of the FMCW waveform.
2. **Radar Transceiver Modeling** – operates through object-based designs which model both transmitter and receiver functions.
3. **Target Modeling** – Utilize static or dynamic targets with defined motion parameters.
4. **Signal Propagation** – Contains free space transmission and analyzes environmental conditions affecting radio waves through rain or terrain interactions.
5. **Range-Doppler Processing** – Transform received signals into range-velocity domain.
6. **Beamforming** – Signal processing of directional antenna arrays.

7. **CFAR (Constant False Alarm Rate)** – Adaptive thresholding for target detection.
8. **Clustering** – Grouping detections corresponding to the same object.
9. **Range, Velocity, and Angle Estimation** – Final output of object properties for tracking or classification.

### 5.2.2. Range-Doppler FFT

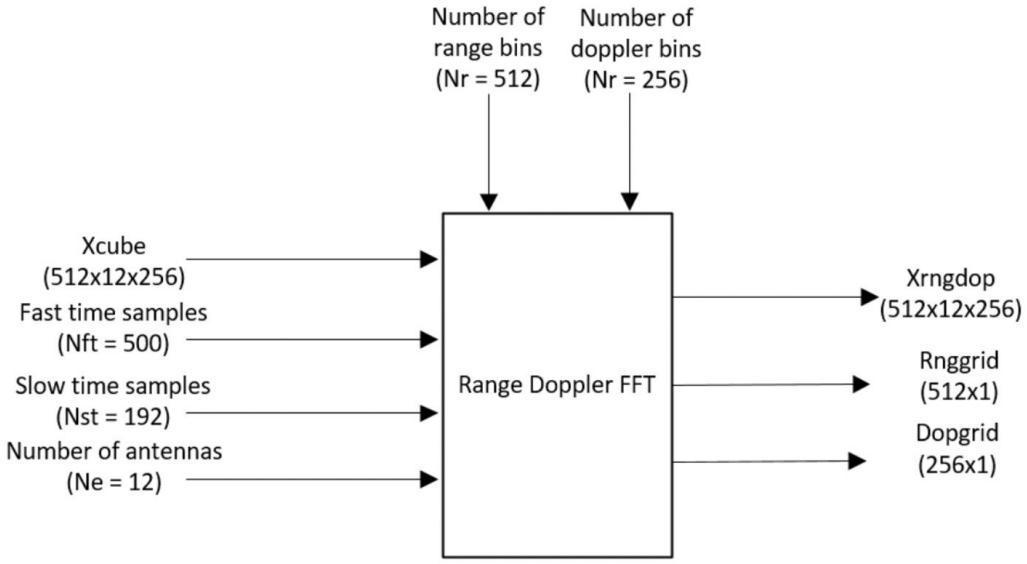



Figure 5: Block level view of Range-Doppler FFT

A. **Range Doppler Processing** involves converting received signal data into a 2D map that shows how target range and velocity vary. It combines the fast-time Fourier transform (for range) and slow-time Fourier transform (for velocity) to distinguish between stationary and moving targets based on their distance and speed.

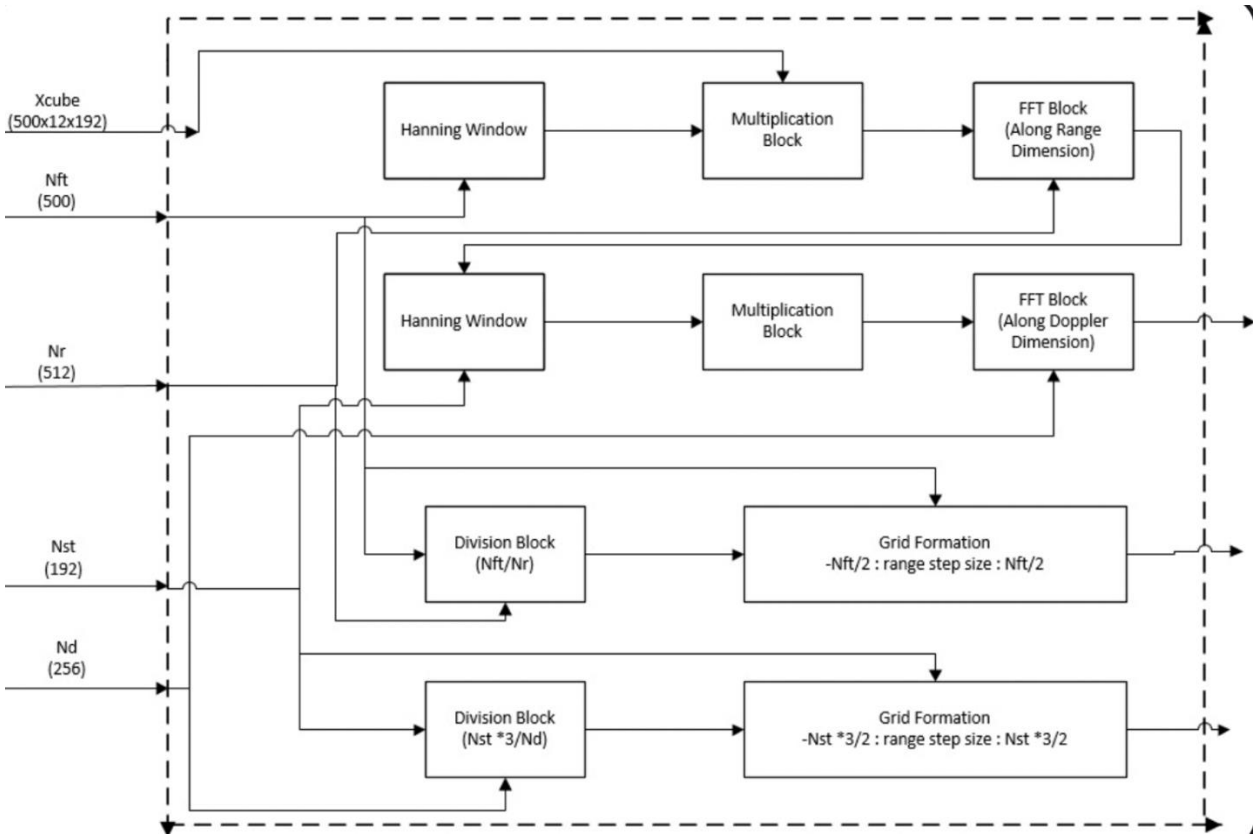



Figure 6: Internal view of Range-Doppler FFT Block

### B. Range-Doppler Processing Pipeline

1. **Input (Xcube)**
    a. Dimensions: 500 (Fast Time) × 12 (Antennas) × 192 (Slow Time)
    b. **Fast Time (Nft=500)**: Range resolution.
    c. **Slow Time (Nst=192)**: Doppler/velocity resolution.
2. **Range FFT**
    a. Apply Hanning window → FFT → zero-pad to **Nr=512 bins**.
3. **Doppler FFT**
    a. Apply Hanning window → FFT → zero-pad to **Nd=256 bins**.
4. **Grid Formation**
    a. **Range Axis**: -Nft/2 to Nft/2 (scaled by Nft/Nr).
    b. **Doppler Axis**: -Nst*3/2 to Nst*3/2 (scaled by (Nst*3)/Nd).

**Output**: 2D Range-Doppler Map (512×256).

*Key Features:* Zero-padding improves resolution; Hanning reduces sidelobes.

### C. Hanning window

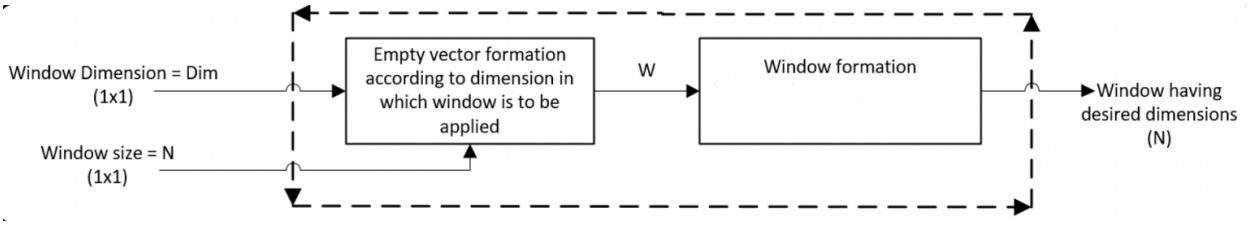



Figure 7: Internal view of Hanning Block

First, the window's dimension (Dim) and size (N) are defined as scalar values (1x1). An empty vector is then initialized based on the dimension where the window will be applied. Finally, the window is formed with the desired size (N), resulting in a 1D window vector of length N that can be multiplied with signal data to reduce spectral leakage during Fourier transforms. The selected window shape such as Hanning smooths signal edges which reduces artificial effects in frequency domain examinations.

### 5.2.3. Beamformer

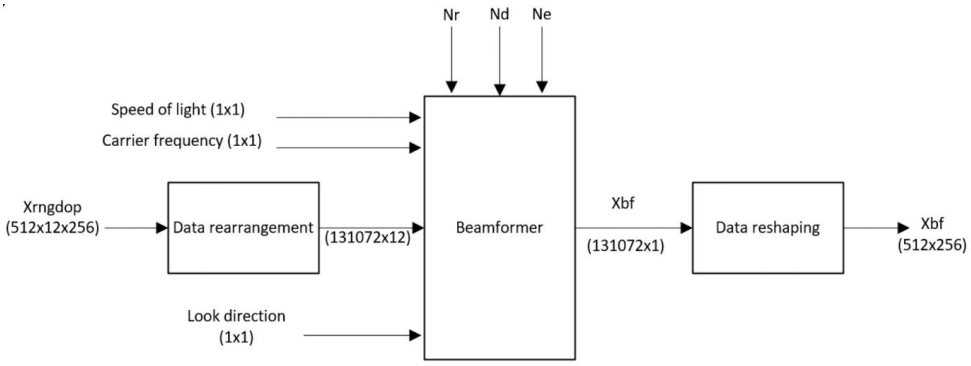



Figure 8: Block level view of Beamformer

The beamformer uses the conventional beamforming concept to generate steering angles to direct or focus the beams to the desired direction which in this case is 0 degrees.

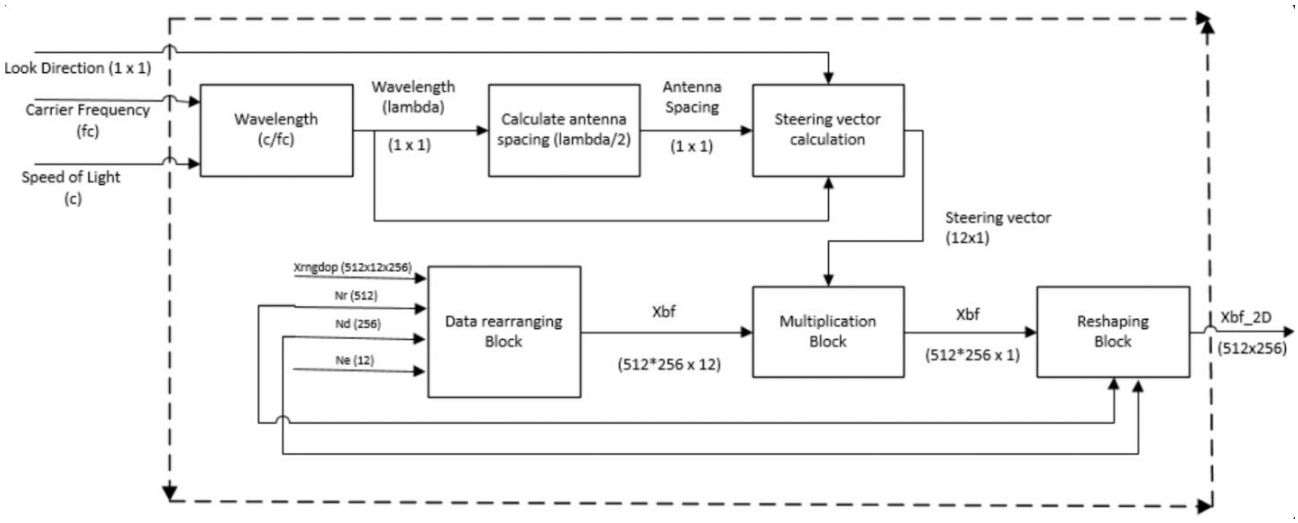



Figure 9: Internal view of Beamformer Block

1. **Inputs:**
    a. Look direction (1×1 angle, typically 0°)
    b. Carrier frequency (fc) → calculates wavelength (λ = c/fc)
    c. Antenna spacing (λ/2 optimal)
2. **Core Calculation:**
    a. Creates 12×1 steering vector using phase shifts from:
        i. Antenna positions
        ii. Look direction
        iii. Wavelength
3. **Data Flow:**
    a. Reshapes 512×12×256 input → 131,072×12 matrix
    b. Multiplies with steering vector → 131,072×1 beamformed signal
    c. Reshapes result → 512×256 range-Doppler map (Xbf_2D)
4. **Purpose:**
    a. Focuses radar energy in specified direction
    b. Combines 12 antenna signals coherently
    c. Outputs enhanced range-velocity map for target detection

*Key features:* Phase-based beam steering, dimensional consistency (512 range × 256 Doppler bins)

### 5.2.4. CFAR

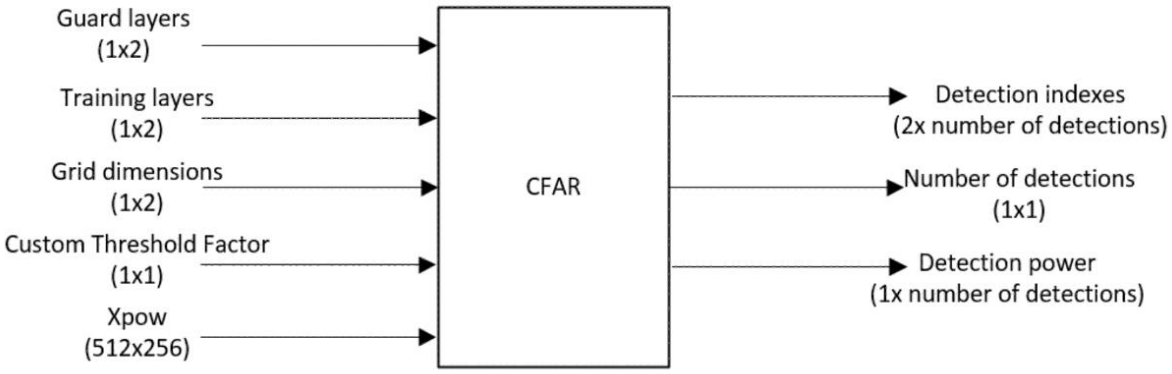



Figure 10: Block level view CFAR Block

CFAR uses the guard and training cells sizes to orient the range doppler bins into the desired observation layout. It then evaluates the threshold and tests the CUT against this threshold for detections.

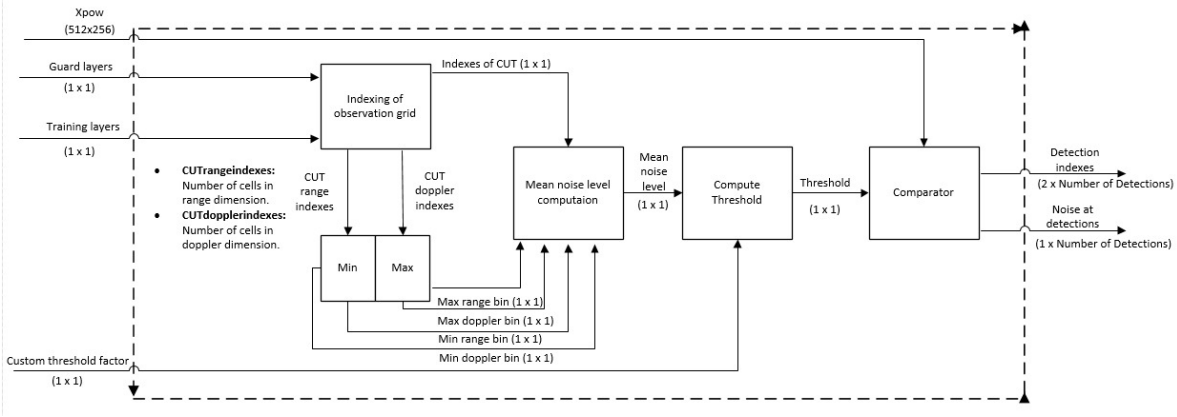



Figure 11: Internal view of CFAR Block

1. **Input:**
    a. Xpow (512×256): Power values from range-Doppler map
2. **Detection Setup:**
    a. Defines guard cells (1×1) and training cells around each Cell Under Test (CUT)
    b. Indexes observation grid with:
        i. CUTrangeIndexes (range dimension)
        ii. CUTdoppleIndexes (Doppler dimension)

3. **Boundary Handling:**
    a. Uses Min/Max blocks to constrain detection to valid:
        i. Range bins (Min/Max range bin)
        ii. Doppler bins (Min/Max Doppler bin)
4. **Threshold Calculation:**
    a. Computes mean noise level from training cells
    b. Applies custom threshold factor (1×1)
    c. Generates detection threshold (1×1)
5. **Output:**
    a. Detection indexes (2×N): Coordinates of detected targets
    b. Noise at detections (3×N): Additional noise metrics
    c. Comparator validates detections against threshold

*Key Features:* Adaptive threshold maintains constant false alarm rate while detecting true targets in varying noise environments.

### 5.2.5. Cluster

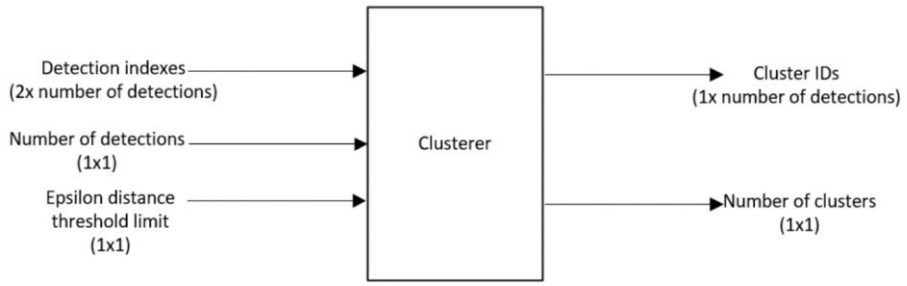



Figure 12: Block level view Cluster Block

The Cluster block uses the Epsilon distance algorithm to cluster the detections. The points whose distance relative to each other is lower than the Epsilon distance threshold limit are given identical cluster IDs.

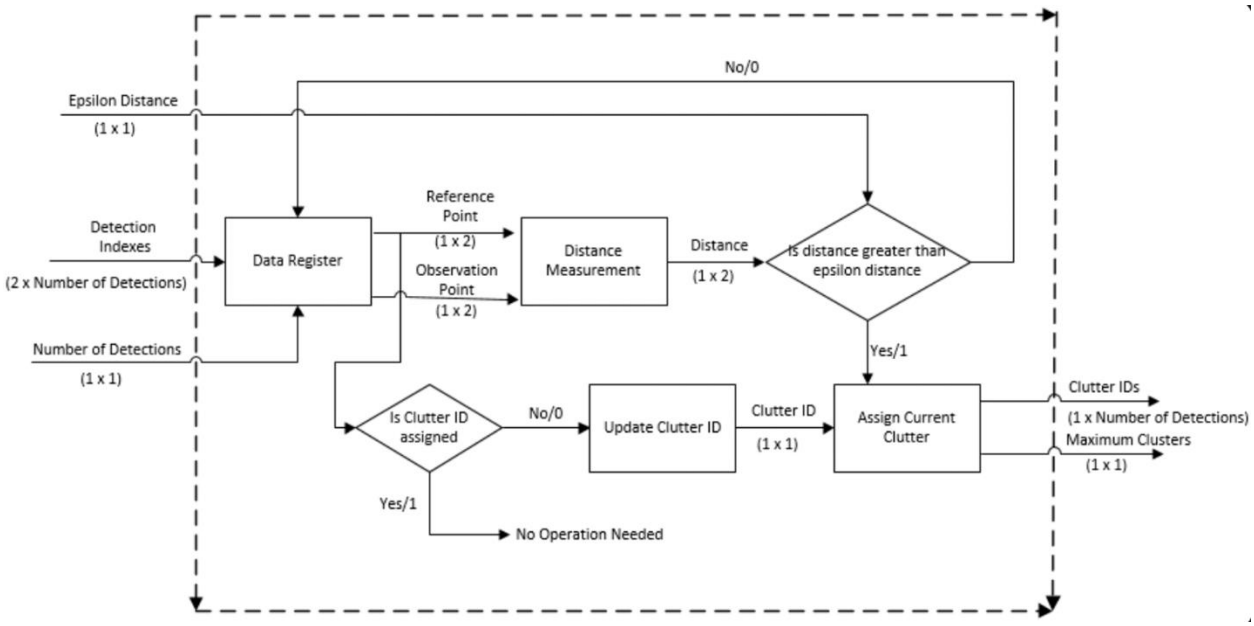



Figure 13: Internal view of Cluster Block

1. **Inputs:**
    a. Detection Indexes (2×N): Coordinates of all detected points
    b. Epsilon Distance (1×1): Maximum allowed distance between cluster members
2. **Core Operation:**
    a. Compares each point (Observation) against reference points:
        i. Measures distance between points (1×2 vector)
        ii. Checks if distance > epsilon threshold (Yes/1 or No/0)
        iii. Assigns Cluster IDs accordingly
3. **Decision Logic:**
    a. If distance ≤ epsilon: Points get same Cluster ID
    b. If distance > epsilon: Points assigned to new cluster
4. **Outputs:**
    a. Cluster IDs (1×N): Cluster assignment for each detection
    b. Maximum Clusters (1×1): Total number of clusters formed

*Key Feature:* Groups nearby detections into clusters while maintaining separation between distinct targets. The epsilon threshold controls cluster granularity.

### 5.2.6. Range Estimator

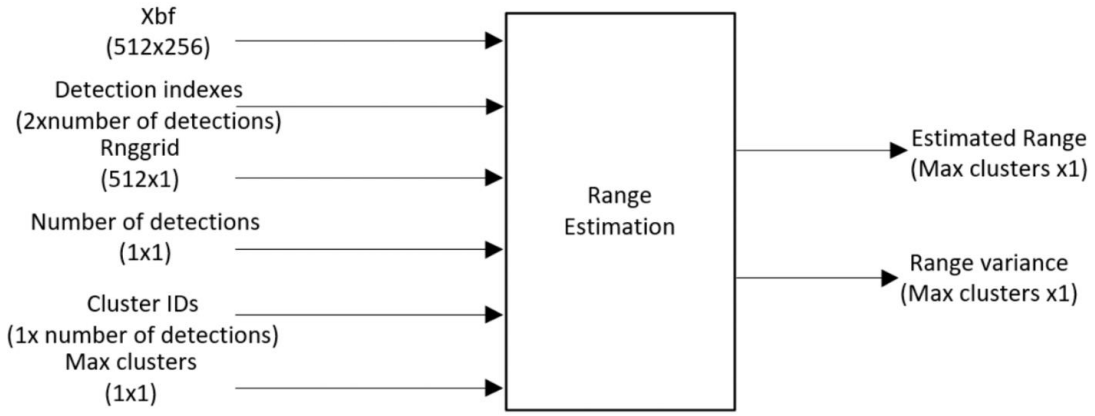



Figure 14: Block level view of range estimator

Uses the detection indexes and the range grid to estimate the ranges of targets by comparing the detection index with ranges assigned to each cell in the range grid. The estimated range is used to calculate the variance. Each cluster is dealt with separately.

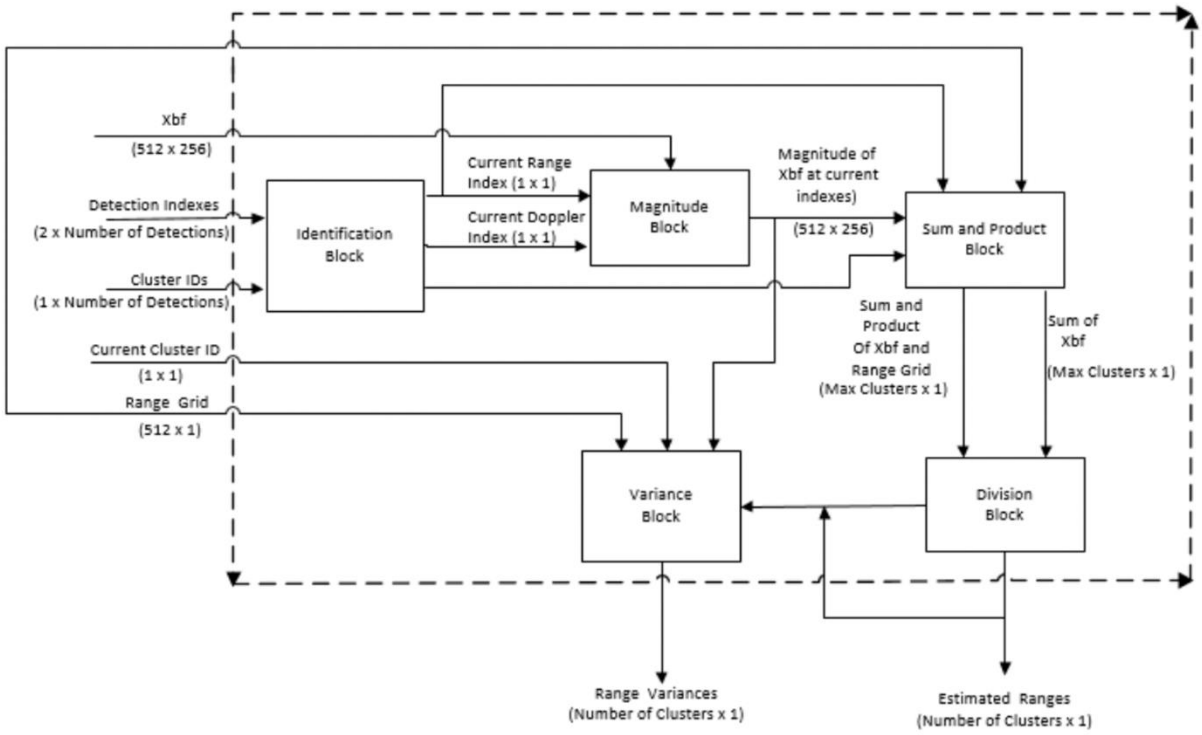



Figure 15: Internal view of range estimator block

1. **Inputs:**

a. Xbf (512×256): Beamformed range-Doppler matrix
b. Detection indexes (2×N): Coordinates of detected points
c. Cluster IDs (1×N): Group assignments for detections
d. Range Grid (512×1): Predefined range values

2. **Processing Steps:**
   a. For each cluster (Current Cluster ID):
      a) Identifies all detections belonging to the cluster
      b) Extracts their range indices (Current Range Index)
      c) Retrieves corresponding magnitudes from Xbf
3. **Calculation:**
   a. Computes weighted sum of range values using Xbf magnitudes
   b. Calculates mean range (Estimated Ranges) per cluster
   c. Derives range variance for each cluster
4. **Outputs:**
   a. Estimated Ranges (N×1): Center points of each cluster
   b. Range Variances (N×1): Spread/dispersion metrics

*Key Features:*

- Uses magnitude-weighted averaging for accurate range estimation
- Processes detections cluster-by-cluster for multi-target scenarios
- Provides both range estimates and confidence metrics (variances)

### 5.2.7. Doppler (Velocity) Estimator

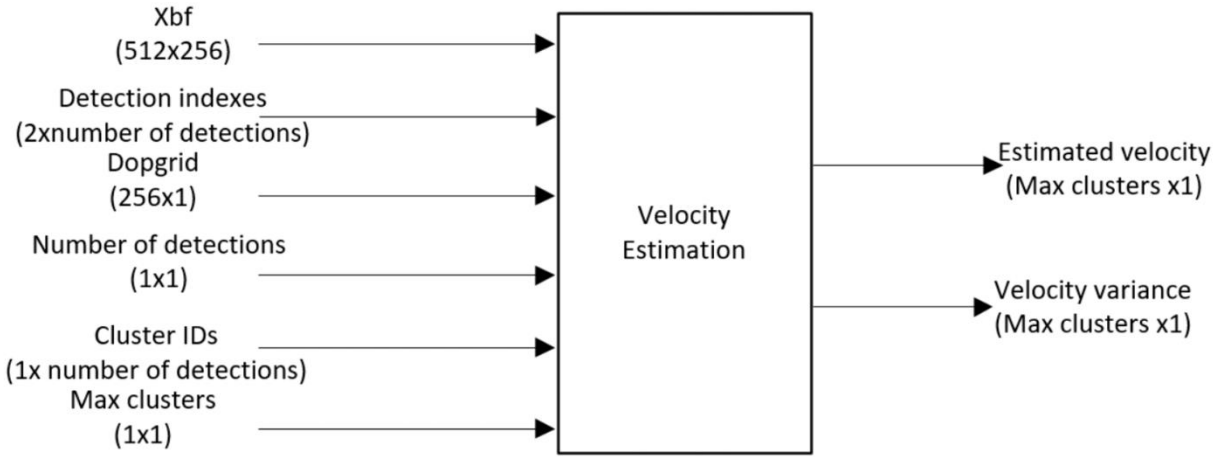


Figure 16: Block level view if velocity estimator

Uses the detection indexes and the doppler grid to estimate the velocities of targets by comparing the detection index with velocities assigned to each cell in the doppler grid. The estimated velocity is used to calculate the variance. Each cluster is dealt with separately.

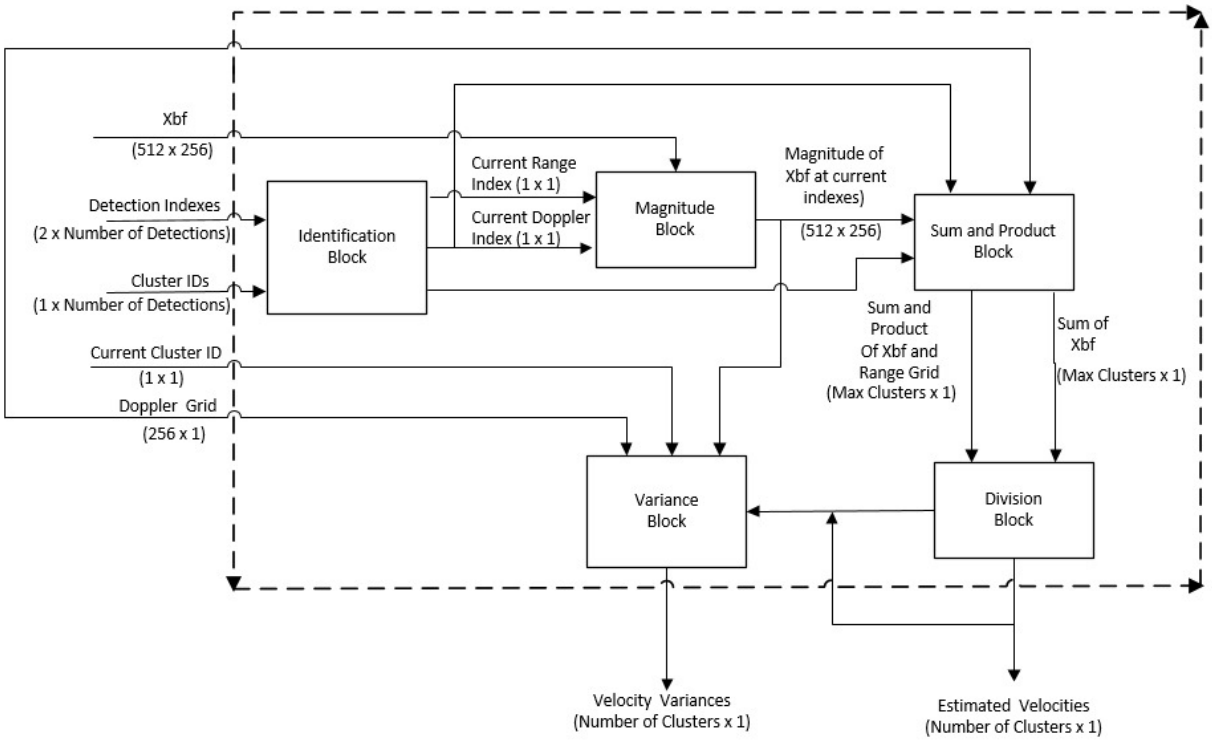



Figure 17: Internal view of velocity estimator block

1. **Input Data:**
    a. Xbf (512×256): Beamformed range-Doppler matrix
    b. Detection coordinates (2×N): [range index, Doppler index] pairs
    c. Cluster assignments (1×N): Group IDs for each detection
    d. Doppler grid (256×1): Predefined velocity values
2. **Processing Flow:**
    a. For each cluster (Current Cluster ID):
        a) Identifies all member detections

b) Extracts their Doppler indices
c) Retrieves corresponding signal magnitudes from Xbf

3. **Key Calculations:**
    a. Weighted velocity average (using signal magnitudes as weights)
    b. Velocity variance computation for each cluster
    c. All calculations performed per-cluster for multi-target scenarios

4. **Output Metrics:**
    a. Estimated Velocities (N×1): Mean velocity per cluster
    b. Velocity Variances (N×1): Spread/dispersion indicators

*Implementation Notes:*

- Uses Doppler grid mapping (index → velocity)
- Processes detections cluster-wise for accurate multi-target tracking
- Magnitude-weighted averaging improves velocity estimation accuracy
- Variance output provides confidence measure for each velocity estimate

## 5.3. Architecture # 2: Floating-Point Chain (Block-wise)

### 5.3.1. Objective

The DSP processing flow shall be implemented using floating-point mathematics. The goal of the Precision Module is to furnish a reference pipeline with sufficient precision for validating algorithms and simulating ideal arithmetic conditions on each stage prior to transitioning to fixed point. Each part of the chain functions independently through functional simulation that duplicates embedded system behavior while preserving clear algorithms for testing purposes.

### 5.3.2. Design Philosophy

Each signal processing function follows a modular block-wise implementation within MATLAB programming structure. The architecture relies on floating-point arithmetic to achieve the highest possible numerical precision throughout simulations along with building performance reference points for range and velocity computation and target recognition and noise resistance**.**

The main model functions as a floating-point system while discrete number representations have been added through quantization modeling across certain parts of the chain. The framework enables researchers to analyze the detection performance alterations stemming from finite-resolution arithmetic operations. The simulation of bit-depth constraints takes place after the Fast Fourier Transform has been computed but before detection using pre-defined fixed-point-like formats (12-bit and 16-bit) without hardware mapping involvement. The intermediate quantization layer serves two purposes: it helps analyze resolution sensitivity while creating a basis for future implementation of fixed-point design should it become necessary.

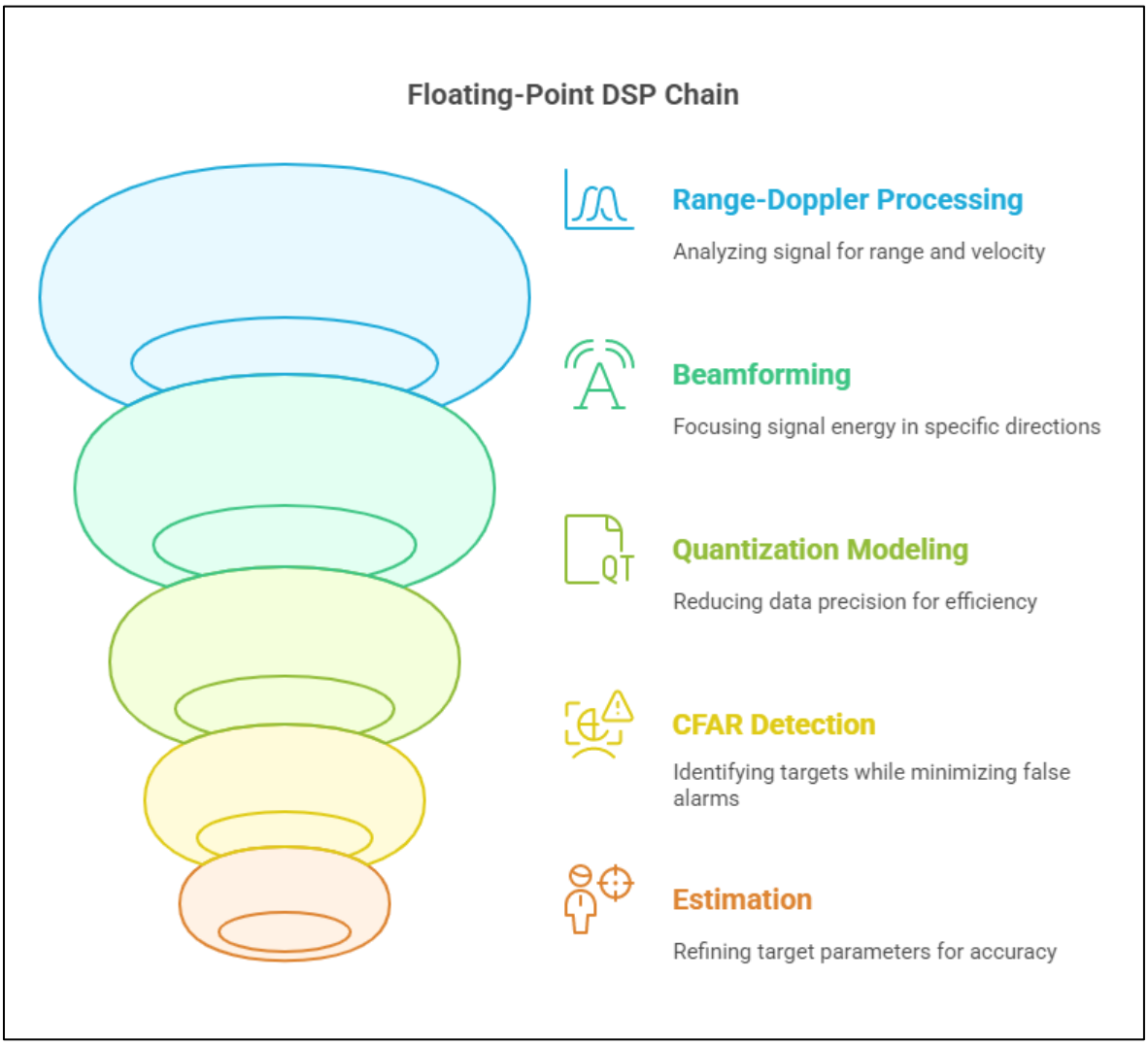


Figure 18: Plan for floating-point DSP chain

### 5.3.3. Architecture Overview

#### 1) Range-Doppler Processing:

The raw ADC data cube undergoes two-dimensional Fast Fourier Transform (2D-FFT) processing as its initial step. The process requires applying a windowing function across time domain and frequency domain to reduce sidelobes. The first FFT determines the target range while the second FFT delivers Doppler velocity measurements. The production of a range-Doppler map reveals the distribution of energy across both distance space and motion space through this method.

#### 2) Beamforming(Power Enhancement):

The delay-and-sum beamforming technique performs coherent signal combination across channels through a conventional or physical antenna framework in each range-Doppler cell. Signal strength receives an enhancement which results in improved signal-to-noise ratio (SNR) before detection occurs. Steering vectors are generated for the target field of view, and the maximum response direction is retained.

#### 3) CFAR Detection:

The range-Doppler map output from beamforming goes through a Cell-Averaging Constant False Alarm Rate (CA-CFAR) detection process. The detection thresholds are dynamically configured through adjustable guard along with training cell parameters. The detection method isolates genuine target peaks from various noise patterns to establish a well-controlled false-alarm instance rate.

#### 4) Quantization Modeling:

The output signals from the beamforming and CFAR blocks receive floating-point quantization using defined word length parameters (12-bit signed fixed-point emulation). The model uses techniques to mimic signal quality reduction caused by reduced bit depth before moving actual fixed-point mathematical calculations.

System evaluation includes an assessment of the effects that quantization has on performance degradation.

**5) Estimation Block:**

The extraction of essential target parameters occurs during the final stage by using peak position data from detection output to obtain range values along with velocity measurements and angle data. The framework provides visual representation of data values during each frame of analysis. Basic target tracking can be achieved through static filtering and centroid extraction methods for proof-of-concept testing.

**6) Visualization and Logging:**

Range-Doppler heatmaps as well as angle-Doppler heatmaps and detection overlays along with parameter plots are produced for interpretation and validation. The simulation produces log data which serves both verification purposes and analytical needs.

### 5.3.4. Advantages of Architecture

- **High Numerical Precision:**

Floating-point arithmetic enables precise modeling of signal processing operations without concern for quantization noise, saturation, or scaling, making it ideal for verifying algorithmic correctness.

- **Algorithm Validation Baseline:**

Serves as a reference model for comparing fixed-point or hardware-limited implementations. Performance metrics such as detection probability and estimation error can be measured in their ideal form.

- **Modular Block Design:**

Each DSP stage—range-Doppler processing, beamforming, CFAR, and estimation—is implemented as an independent, reusable block, supporting easy testing, tuning, and replacement.

- **Quantization Modeling Integration:**

Allows the effects of reduced bit-depth to be simulated at key points in the pipeline without changing the arithmetic format, offering a middle ground between ideal and hardware-like behavior.

- **Scenario Flexibility:**

Supports a wide range of radar configurations, input parameters, and target scenarios due to its parameterized and software-driven nature.

- **No Hardware Dependency:**

Entirely simulation-based, this architecture avoids hardware constraints, enabling faster development, debugging, and proof-of-concept exploration.

## 5.4. Architecture # 3: Fixed-Point Chain (Hardware-Friendly MATLAB)

### 5.4.1. Objective

The architecture focuses on developing a radar DSP pipeline simulation in MATLAB through fixed-point arithmetic approaches that replicate hardware amenable properties and restrictions. The main goal converts floating-point DSP blocks into Q1.15 (16-bit signed fixed-point with 15 fractional bits) format to determine resource consumption in hardware and understand precision effects on bit-level results while identifying implementation obstacles during FPGA or DSP hardware synthesis.

### 5.4.2. Design Philosophy

The design implements a modular block structure which mirrors the structure used in the floating-point chain. MATLAB implements Fixed-Point Designer to

transform all computational paths and data signals into fixed-point mathematical types. Operators within each stage receive their word lengths and scaling factors from human configuration to balance both bit growth and quantization noise. The architecture emphasizes protection of dynamic range against overflows by using controlled bit lengths that minimize truncation errors. A performance analysis of the processing blocks occurs through scenarios tested against floating-point evaluation methods to achieve direct comparison results.

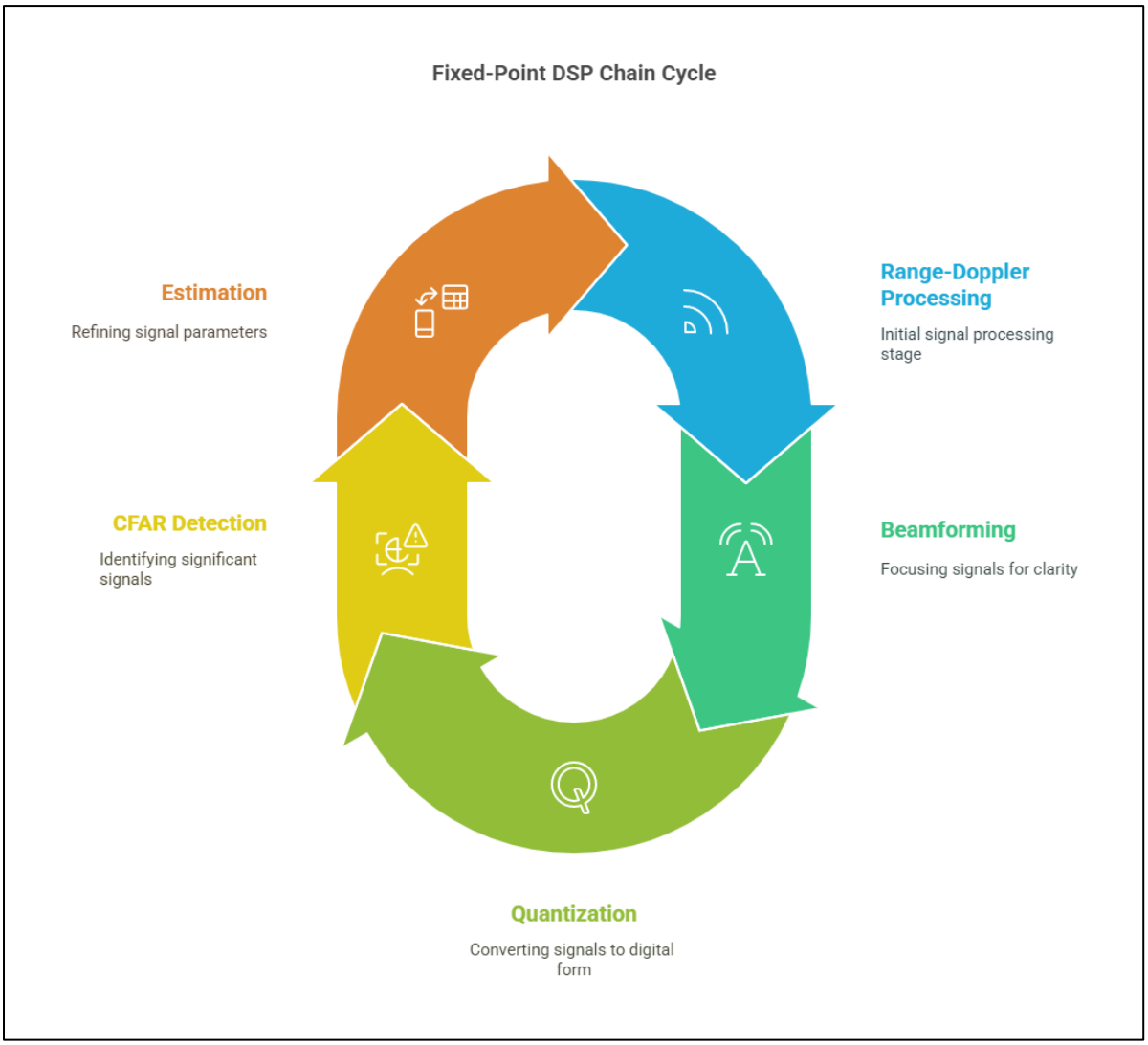



Figure 19: Plan for fixed-point DSP chain

### 5.4.3. Architecture Overview

#### 1) Fixed-Point Configuration:

The Q1.15 format serves as the primary data format across all data transmission paths, but exceptions occur when necessary. The selected format manages to strike an optimal balance between the system's reach along with its accuracy and its capacity to function with current hardware. The architecture includes supplemental scaling blocks which sit between modules for maintaining operational gain and avoiding bit growth during FFT and accumulation operations.

**2) Range-Doppler Processing:**

- The 2D FFT structure from floating-point chain is replicated into fixed-point architecture.
- The Hanning window along with other window functions undergo quantum arithmetic processing with a Q1.15 fixed-point format.
- The system checks word length parameters of FFT inputs and outputs to avoid overflow conditions during butterfly processing stages.
- Bit-width expansion techniques are employed for accumulator values that exceed 16 bits followed by saturation or rounding operations depending on system requirements.

**3) Beamforming (Power Enhancement):**

- The delay-and-sum algorithm has been rewritten using fixed-point multiplication followed by accumulation.
- The input data undergoes scaling operations to prevent saturation effects from occurring.
- The system uses Q1.15 quantization to represent steering vectors along with weighting coefficients.
- The normalization process of output enables the preservation of signal power within an acceptable numerical range.

**4) CFAR Detection:**

- The fixed-point CA-CFAR detection algorithm utilizes quantized training parameters and guard cell values for its implementation.
- The system performs all intermediate averaging tasks and comparison functions within the fixed-point arithmetic system.

- Traditional noise estimation sustainability is ensured when using both expanded numerical precision and proper rounding procedures.
- Threshold scaling must reach optimal settings to achieve a proper balance between false alarm detection and missed detection events.

### 5.4.4. Advantages of Architecture

- **Hardware Awareness:**

The Q1.15 format in fixed-point modeling emulates how embedded and FPGA systems behave by revealing numerical precision characteristics and range limits as well as overflow situations.

- **Bit-Level Control:**

The explicit word-length and scaling settings help designers optimize signal quality together with resource usage during the early design phase to fulfill power and area and performance requirements.

- **Resource Optimization Readiness:**

When architecture evaluates signal paths through fixed-point mathematics designers can execute early assessments about algorithm execution costs versus results accuracy which benefits constrained resource conditions.

- **Quantization Impact Visibility:**

Block-by-block analysis of quantization error combined with signal degradation and bit-growth behavior detects essential chain points while designers prepare for hardware synthesis.

- **Consistent with Floating-Point Chain:**

The module-based design element survives from the floating-point operation sequence which enables precise output comparison so developers can validate and detect numerical system impacts.

- **Simulation-Only Feasibility:**

The entire system functions inside MATLAB's simulation environment to evaluate and develop the system without requiring HDL or FPGA hardware.

## 5.5 Architecture #4: RTL Chain (Hardware Architecture)

### 5.5.1 Full Pipeline

The block diagram illustrates our the DSP chain's processing flow on the Nexys A7-100T FPGA, showcasing the system's transformation of real-time input through a two-stage FFT pipeline. The chain's first stage applies a 256-point Range FFT with windowing to the input signal, storing the results in the design's ping-pong buffers for efficient data handling. The Doppler processing stage then performs a 128-point FFT per range bin using the system's Doppler window coefficients. The chain's square-law detector computes magnitude-squared values, which feed into the CFAR unit for the design's target detection logic. All operations are orchestrated by the control unit overseeing the chain's data flow.

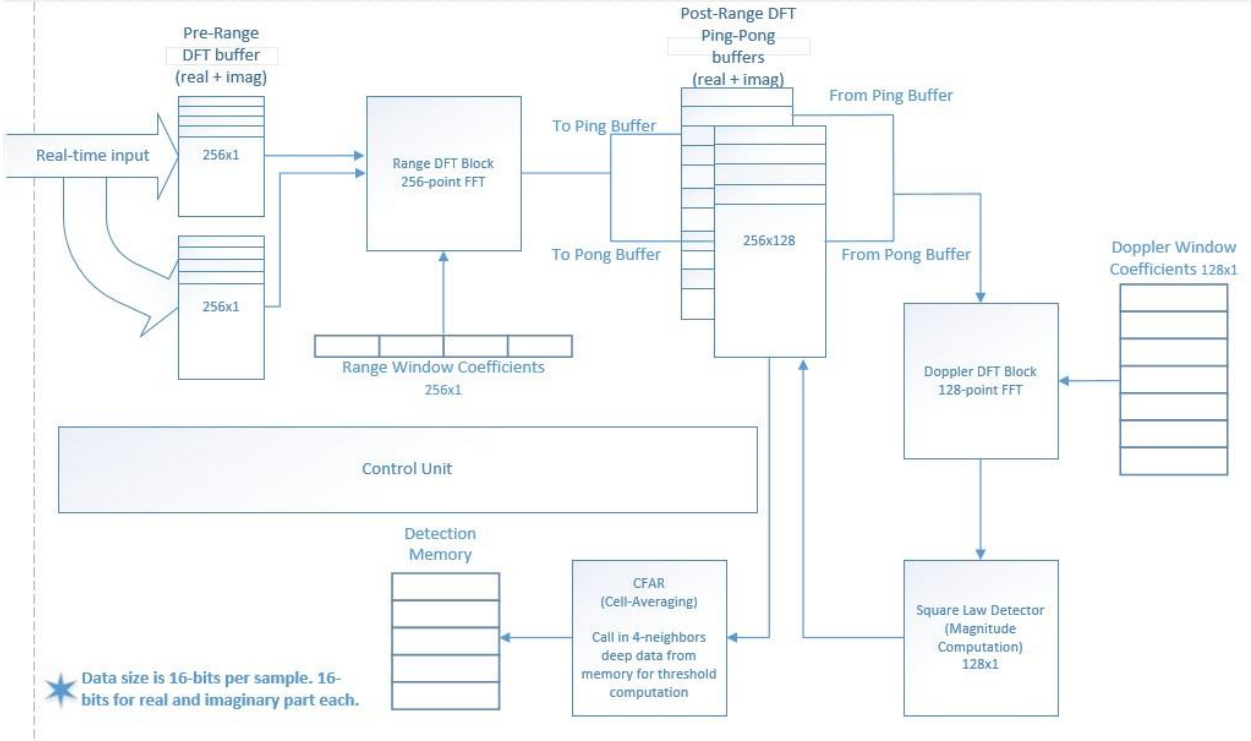


Figure 20: Block diagram for RTL design of DSP

### 5.5.2 File Hierarchy

FFT_Processor_Chain (Top)

- control_unit.sv
- buffer_16x256.sv
    - preRangeFFTBuffer1
    - preRangeFFTBuffer2
- dft_256pt_streaming.sv
- ping_pong_buffer.sv
    - simple_dual_port_ram.sv
        - ping_real_ram
        - ping_imag_ram
        - pong_real_ram
        - pong_imag_ram
- dft_128pt_streaming.sv
- square_law.sv
- cfar.sv
    - buffer_mem
- detection_memory.sv

### 5.5.3 DSP_Processor_Chain (Top)

This FFT_Processor_Chain module is the top-level integration of a radar signal processing pipeline targeting the Nexys A7-100T FPGA. It orchestrates a complete two-dimensional FFT-based DSP flow that detects targets based on both range and Doppler (velocity) information. The design accepts 16-bit real and imaginary input signals and begins processing them by storing the incoming data in dual 256-depth pre-range FFT buffers for real and imaginary parts. These buffered signals are then windowed and passed into a 256-point streaming FFT module (dft_256pt_streaming) that computes the range domain transformation.

The range FFT outputs are streamed into a ping-pong buffer system that allows concurrent write/read operations to support continuous data throughput. Depending on control signals, either the ping or pong buffer feeds into the second FFT stage — a 128-point streaming Doppler FFT (dft_128pt_streaming) — which processes each range bin over successive pulses.

Following the Doppler FFT, the output is magnitude-squared using a square_law detector module to simplify the complex-valued signal. This magnitude output feeds into the cfar (Constant False Alarm Rate) block, which performs adaptive thresholding using neighboring cells to identify peaks corresponding to potential targets. Detected range-Doppler bin coordinates are then stored in the detection_memory module, which allows later retrieval through the read_en_detectmem signal.

The entire chain is coordinated by the control_unit, which generates all necessary read/write enables and control flags based on buffer states and FFT completion signals. This modular design supports pipelined processing, minimizes latency, and efficiently maps onto the Artix-7's logic and memory resources.

### 5.5.4 buffer_16x256

The buffer_16x256 module implements a simple synchronous circular buffer (FIFO-style RAM) with parameterized width and depth, used in the radar DSP chain to temporarily store streaming data for FFT input preparation. This buffer is designed to handle 16-bit data words across 256 memory locations, but both width and depth can be customized via parameters.

Internally, the buffer consists of a register array (buffers) and a shared address counter that manages read and write positions. The buffer supports independent read and write operations, triggered by their respective enable signals (read_en, write_en).

During a write, the input data (data_in) is stored at the current address, and the address counter increments. Similarly, on a read, data is output from the current address.

Buffer status is indicated via two flags:

- is_full becomes high when the buffer has received 256 writes, preventing overflow.

- is_empty becomes high when the last piece of stored data has been read, preventing underflow.

A reset clears the entire buffer and flags, ensuring a clean startup state.

### 5.5.5 dft_256pt_streaming

The dft_256pt_streaming module implements a fixed-size 256-point Discrete Fourier Transform (DFT) tailored for real-time DSP applications, such as radar signal processing. It operates over a finite state machine with three main stages: loading, computation, and streaming. During the LOAD phase, incoming 16-bit signed complex samples are stored in internal buffers for both real and imaginary components. Once the buffer is full, the COMPUTE phase is triggered by the ctrl_flag signal, initiating the DFT calculation using a nested iteration over frequency bin (k) and sample index (n). Each DFT bin is computed by performing complex multiply-accumulate operations between buffered input samples and pre-stored DFT coefficients (W_real, W_imag), which are organized as Q15 fixed-point values in 2D ROM arrays. The results are accumulated with sufficient bit width to preserve dynamic range, then quantized back to Q15 format and stored. In the final STREAM phase, the computed frequency-domain outputs are serially emitted via out_real and out_imag with a valid signal. Status flags such as dft_load_done and dft_compute_done mark the transitions between stages, while ram_full and ram_empty assist in load scheduling.

### 5.5.6 ping_pong_buffer

The ping_pong_buffer module implements a double-buffered memory architecture for high-throughput, continuous data flow between two processing stages in a radar DSP chain. It uses two dual-port RAM blocks—named ping and pong—for storing complex data, allowing one buffer to be written while the other is read, thus decoupling the write and read processes. Internally, the design tracks the write and read addresses using counters (write_count, read_count) and uses a toggle flag (current_buffer) to switch between ping and pong roles once a full frame (of size RANGE × DOPPLER) is written. When writing is complete, the buffer_full flag is asserted, and the active_buffer_sel signal indicates which buffer is currently being written.

Each ping/pong buffer pair (real and imaginary) is implemented using simple_dual_port_ram modules, enabling concurrent read and write access. The data output logic dynamically selects which buffer to read from—opposite of the active write buffer—ensuring proper sequencing between FFT stages. This structure enables seamless data transfer between the range FFT and Doppler FFT, with minimal control overhead and efficient use of memory resources on FPGAs like the Artix-7.

### 5.5.7 dft_128pt_streaming

The dft_128pt_streaming module performs a fixed-size, 128-point Discrete Fourier Transform (DFT) on complex input data in a time-multiplexed, memory-based architecture. Designed for use in Doppler processing stages of radar signal chains, the module captures streaming input samples via load_en, storing them into internal real and imaginary buffers. Once a full frame of 128 samples is loaded, the ctrl_flag initiates the DFT computation phase, where a nested loop calculates each frequency bin (k) by summing the complex products of the input data with precomputed Q15-format twiddle factors (W_real and W_imag). The computed frequency-domain results are stored and then sequentially streamed out during the STREAM state, with valid data flagged by out_valid.

The module's operation is managed by a four-state FSM (IDLE, LOAD, COMPUTE, STREAM) and uses status signals (dft_load_done, dft_compute_done, ram_full, ram_empty) to coordinate processing with external control logic. Output quantization is performed by truncating the accumulated result to 16-bit fixed-point using the middle 16 bits ([30:15]) to preserve precision.

### 5.5.8 square_law

The square_law module implements a basic magnitude-squared detector, commonly used in radar and signal processing pipelines to convert complex-valued frequency-domain data into a real-valued power representation. It takes signed 16-bit real and imaginary inputs and computes their squared magnitudes using the formula: *|X|2=Re2+Im2|X|^2 = {Re}^2 + {Im}^2*|X|2=Re2+Im2. Internally, it multiplies each component with itself (real_squared, imag_squared), sums the results

(magnitude_squared), and scales down the output by discarding the lower half of the result bits to fit back into a 16-bit signed output (sqw_real_out). This scaling is done by selecting the upper Width bits of the 32-bit result, preserving the most significant information while preventing overflow. Synchronous logic ensures all operations are clocked and resettable, making the module pipeline-friendly for real-time FPGA processing.

### 5.5.9 cfar

The cfar module implements a Cell-Averaging Constant False Alarm Rate (CA-CFAR) detection algorithm for a 2D range-Doppler map generated by FFTs in a radar DSP chain. It operates on real-valued magnitude-squared data (data_in) produced by the square_law module and uses a configurable sliding window to identify signal peaks that exceed a dynamically calculated threshold. Internally, the module stores the entire frame (of size RANGE × DOPPLER) in a dual-port RAM (buffer_mem) during the LOAD phase, triggered when enable is asserted.

Once loaded, the PROCESS phase begins, scanning each cell (CUT: cell under test) that lies within valid boundaries, avoiding edge artifacts by requiring sufficient guard (G) and training (T) cells around it. For each valid CUT, the module accumulates surrounding training cells' values, excluding the central region (CUT and guard cells). The summed training power is scaled using a Q15 multiplier (THRESH_SCALE) to compute a threshold, against which the CUT is compared. If the CUT exceeds the threshold, a detection is flagged (detection_valid), and its location (range_bin, doppler_bin) is output.

The module tracks its state through a simple FSM (IDLE, LOAD, PROCESS) and provides a cfar_done signal once the full range-Doppler space has been scanned. Efficient memory addressing is achieved through a row-major mapping function, and the CFAR logic supports flexible training/guard window sizes via the parameters T and G. This implementation provides a tunable, pipelined detection mechanism that is well-suited for FPGA-based radar systems requiring real-time target detection with controlled false alarm rates.

### 5.5.10 simple_dual_port_ram

The simple_dual_port_ram module implements a basic synchronous dual-port block RAM with independent read and write access, making it ideal for high-throughput, decoupled memory operations in FPGA-based DSP pipelines. It is parameterized for data width (WIDTH), depth (DEPTH), and address width (ADDR_WIDTH), allowing flexible use across various subsystems like ping-pong buffers, detection memories, and intermediate storage.

Internally, the RAM is declared using the (* ram_style = "block" *) attribute, which guides synthesis tools (like Vivado) to map it to dedicated block RAM (BRAM) resources on the FPGA, ensuring optimal area and performance efficiency. The write operation is synchronous to the rising edge of the clock and gated by the wr_en signal. Simultaneously, a registered read path provides the output rd_data from the given rd_addr on each clock cycle.

### 5.5.11 detection_memory

The detection_memory module serves as a compact, FIFO-style buffer for storing detection results from the CFAR module in a radar DSP pipeline. Each stored entry consists of a packed structure (detection_t) containing the range and Doppler bin indices of a detected target. The module supports simultaneous input and output operations, using separate write and read pointers along with a counter (detection_count) to manage buffer occupancy.

When a valid detection is received (detection_valid is high), and the buffer isn't full, the module writes the detection into memory at the write_ptr location and increments the count. Similarly, when read_en is asserted and data is available, the module outputs the next detection and advances the read_ptr. The module provides status signals: detection_available (indicating at least one detection is ready for readout) and memory_full (to signal when the buffer has reached its maximum capacity of MAX_DETECTIONS entries).

### 5.5.12 control_unit

The control_unit module is the central state machine that coordinates the timing and sequencing of all major components in the radar DSP chain, ensuring smooth data flow from input buffering through FFTs, square-law detection, and CFAR processing. It operates as a clock-driven FSM with ten well-defined states, beginning from IDLE and progressing through BUFFERING, FFT load and compute phases, ping-pong buffering, square-law transformation, and finally CFAR detection.

Each state activates a specific set of control signals to initiate and manage activity in the corresponding module. For instance, in the BUFFERING state, it enables writing to the pre-FFT input RAMs, while in LOAD_TO_DFT256, it triggers read operations, and the load enable for the 256-point FFT. Once the FFT is complete, the state transitions to LOAD_TO_PINGPONG and toggles write operations into the ping-pong buffer. The controller then fetches data from the buffer (LOAD_FROM_PINGPONG) and initiates the second FFT (LOAD_TO_DFT128 and COMPUTE_DFT128), followed by square-law processing (SQUARE_LAW) and CFAR detection (CFAR_PHASE).

The FSM transitions are governed by status inputs from each processing stage (e.g., dft256_load_done, pingpong_full, cfar_done), and default values for control outputs are asserted low to ensure safe idle behavior. This design centralizes control in a single module, enabling deterministic, sequential, and efficient coordination of pipeline stages without stalling or data hazards, making it ideal for tightly coupled FPGA radar signal processing systems.

*Chapter 6*

# IMPLEMENTATION AND TESTING

## 6.1 Methods for Implementing the Pipeline

### 6.1.1 Radar Signal Simulation and Processing for Automated Driving

- Parameters must be manually coded
- Uses Phased Array System Toolbox
- Allows custom antenna array design
- Supports signal-level analysis
- More flexible in defining signal processing details
- Less built-in support for real-world scenarios

These scenarios use prebuilt radar simulation blocks to model sensor configurations commonly used in ADAS systems:

- **Scenario Generation**
  - **a.** 1 Transmitter – 6 Receivers
  - **b.** 1 Transmitter – 4 Receivers
  - **c.** 3 Transmitter – 4 Receivers

### 6.1.2 Design and Simulate an FMCW Long-Range Radar (LRR)

- Uses **Radar Designer App**
- Requires minimal manual parameter setting
- Does not require manual antenna modeling
- Suitable for measurement-level analysis
- Enables scenario variation through environment presets
- Uses MATLAB helper functions

These scenarios involve detailed signal-level radar simulation where parameters and environmental conditions are controlled to study radar behavior in challenging cases:

- Single Target Stationary with respect to the radar

- Multiple Targets Stationary with respect to the radar
- Scenario 1 + Rain Effects (adds precipitation)
- Increasing Noise Figure (simulating degraded electronics)
- Addition of Earth Conditions (e.g., terrain, reflections)

## 6.2 Scenario generation

Six unique scenarios were simulated and analyzed. The key parameters and results are summarized below.

### 6.2.1 Target - No Rain

- Transmitters: 3
- Receivers: 4
- Target Range: 23 m
- Speed: 30 m/s
- Environment: Free space
- Observation: Baseline SNR and detections established.

### 6.2.2 Increased Target Range

- Target Range: 31 m (others same as Scenario 1)
- Observation: SNR decreases as range increases due to signal attenuation.

### 6.2.3 Multiple Targets

- Target Ranges: 23 m and 31 m
- Number of Targets: 2
- Observation: Successful detection of both targets; good separation in range-Doppler plots.

### 6.2.4 Rain Condition

- Target Range: 26 m
- Precipitation: Rain
- Observation: Rain causes additional signal attenuation leading to degraded SNR.

### 6.2.5 Increased Noise Figure

- Noise Figure: Increased from 12 dB to 15 dB
- Observation: Higher system noise lowers SNR, even under identical physical conditions.

### 6.2.6 Earth Surface Effects

- Environment: Custom Earth Surface
- Observation: No detections; system fails due to excessive losses or interference

## 6.3 End to end simulation

### 6.3.1 What we did

- Signal Generation: MATLAB's phased.FMCWWaveform generated the linear chirp waveform using parameters consisting of 77 GHz center frequency, 150 MHz sweep bandwidth and 7.33 μs sweep time.
- Receive Modeling: Includes the combination of phased.RadarTarget and phased.FreeSpace propagation to send feedback signals for each chirp.
- Range FFT + CFAR: The script performed range profiling by using fft to process beat-signal vectors after which it sent the profiles to phased.CFARDetector for peak identification through CA-CFAR using 8x8 training and 4x4 guard regions.
- Detection List: Assembled all range-bin indices that exceeded the threshold level into a basic MATLAB array.
- Visualization: Production of time-domain chirps and range profiles and raw detection visuals occurred immediately.

### 6.3.2 Why does this matter

This evaluation method provided instant verification of our signal processing parameters at various SNR conditions by ignoring quantization or pipelining constraints.

### 6.3.3 Scenario

- Ego vehicle: Moving with velocity of 22.22m/s
- Target vehicle 1: 20m in front of ego vehicle and moving with velocity of 27.78m/s (5.55m/s relative).
- Target vehicle 2: 60m in front of ego vehicle and moving with velocity of 36.11m/s (13.89m/s relative).
- Target vehicle 3: 1m behind ego vehicle and moving with velocity of 30.56m/s (8.34m/s relative).
- Free environment.
- No rain or percipitation.

## 6.4 Floating point

### 6.4.1 What we did

- **C**hirp Generator Block: The custom MATLAB function generated beat-signal samples for each chirp through coded intermediate computations at time, phase and window level.
- Mixer & IF Chain: The Mixer & IF Chain module simulated the mixing between the received RF and LO signals which yielded baseband I/Q samples. The experiment added controlled white Gaussian noise adjusted for different SNR levels.
- Windowing & 1-D FFT: The Hanning window was applied in code to range-profiles and single 1024-point FFTs operated per chirp and saved complex data in arrays that measured rangeBins times numChirps dimensions.
- Doppler FFT: Another FFT operation was performed on the time series dimension of “slow-time” to generate a complete range-Doppler image data set.

- 2-D CFAR: A MATLAB implementation of CFAR detection utilized CA-CFAR through nested loops for calculating guard/training cells on the range-Doppler map and resulting in a binary detection map.
- Result Extraction: The program grouped detected cells into pairs of range values and velocity measurements then used Monte Carlo calculations to determine basic detection parameters Pd and Pfa.

### 6.4.2 Why does this matter

The method of manually implementing floating-point logic allowed us to see through all intermediate SNRs and Doppler leakage as well as side-lobe levels and CFAR behavior characteristics which the automatic system makes difficult to inspect.

## 6.5 Fixed point

### 6.5.1 What we did

- Word-Length Planning: Our Word-Length Planning selected Q1.15 for samples as well as Q4.12 for CFAR sums and Q2.14 for FFT accumulators which resulted from dynamic range evaluation of floating-point outputs.
- Fixed-Point Conversion: All blocks in the design received conversion to fi objects through MATLAB's Fixed-Point Designer tool including window multiplications and FFT butterflies and magnitude-squared calculations and CFAR sums.
- Scaling & Bit-Growth Control: The execution involved inserting shift operators and gain parameter adjustments after each Fast Fourier Transform process in order to limit both accumulators' overflow and maintain system dynamic range.
- Overflow & SQNR Testing: The fixed-point chain processed randomly sequenced chirp signals to determine the overflow occurrences while determining the SQNR against floating-point reference results. Precision modifications to bit-width parameters achieved Pd values below 1% threshold.
- Fixed-Point Validation: The validation of fixed-point operation involved comparing output graphs between float point and fixed-point range Doppler and overlaying CFAR detection maps and producing tables that presented detection accuracy statistics at different SNR values.

### 6.5.2 Why does this matter

The system development stage at this point ensures both accuracy and quantization capability to survive in automotive-grade applications before hardware implementation begins.

## 6.6 RTL chain (Hardware architecture)

### 6.6.1 Generation of Test Signals

To simulate realistic radar input signals for the RTL testbench in Vivado, a MATLAB script was developed to generate dechirped FMCW radar data based on automotive driving scenarios. The process involved:

- Designing an FMCW waveform with parameters (e.g., sweep bandwidth, chirp duration) derived from the radar's range and velocity resolution requirements.
- Simulating a dynamic radar environment using MATLAB's phased and drivingScenario toolboxes.
- Capturing dechirped signals across multiple sweeps and scaling them to match fixed-point hardware constraints.
- Converting complex radar returns into Q1.15 fixed-point format, separating real and imaginary parts, scaling to [-32768, 32767], and exporting the data to text files in a syntactically compatible format for use in Verilog testbenches.

### 6.6.2 Using Test Signals in Vivado Testbench

The Q1.15 formatted radar signals generated in MATLAB were imported into the RTL testbench environment in Vivado. These signals were used as direct stimuli for the fixed-point DSP pipeline. By reading the real and imaginary components from buffers in testbench, the design accurately mimicked real-world radar input conditions. This allowed for functional verification of each processing stage—windowing, FFT, and data movement—within the RTL chain under realistic, hardware-representative scenarios.

### 6.6.3 Test Cases Used

For testing our RTL design of DSP chain, we utilized following two test cases:

**A. Case 1:**

- Ego vehicle: Moving with velocity of 30m/s
- Target vehicle: 50m from ego vehicle and moving with velocity of 50m/s (20m/s relative).
- Free environment.
- No rain or percipitation.

**B. Case 2:**

- Ego vehicle: Moving with velocity of 30m/s
- Target vehicle 1: 50m from ego vehicle and moving with velocity of 50m/s (20m/s relative).
- Target vehicle 1: 70m from ego vehicle and moving with velocity of 70m/s (40m/s relative).
- Free environment.
- No rain or precipitation.

*Chapter 7*

# RESULTS AND DISCUSSION

## 7.1 End-to-End Simulation

### 7.1.1Transmitted Signal Waveform

This is the plot of the signal projected into the environment from the radar front end. It represents an FMCW waveform in the time domain, showing the high-frequency chirp used for target detection and ranging.

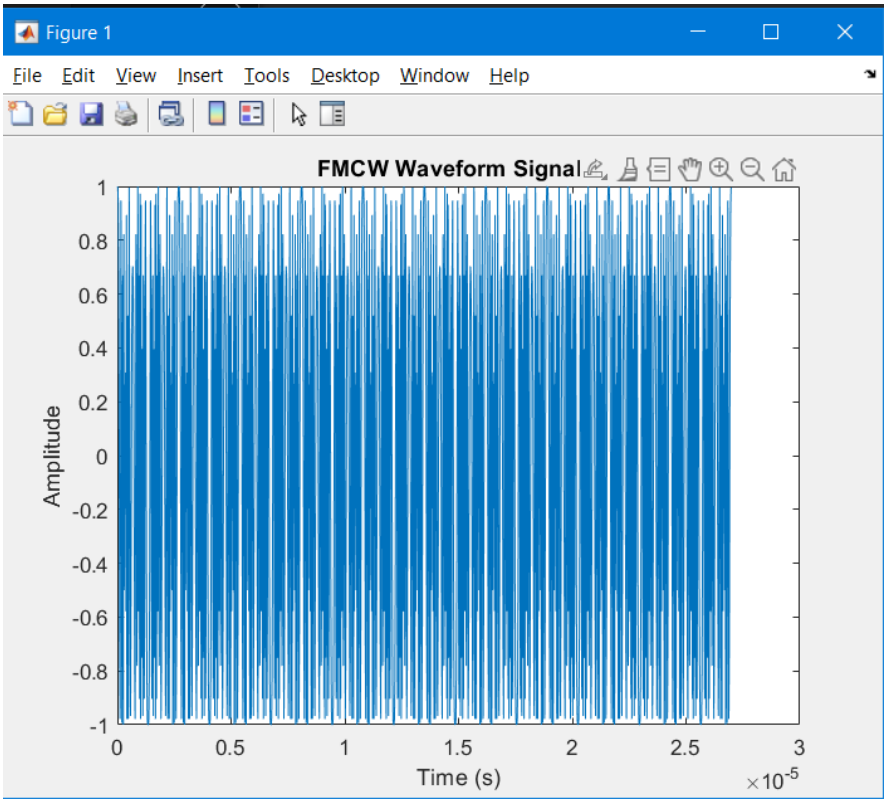



Figure 21: Transmitted wave from the radar

### 7.1.2 Received Signal

This is the plot of the signal received back at the radar after reflection from objects in the environment. It represents the time-domain view of the FMCW waveform

as captured by the receiver, containing information about the target's range and velocity embedded in the delayed and attenuated echo.

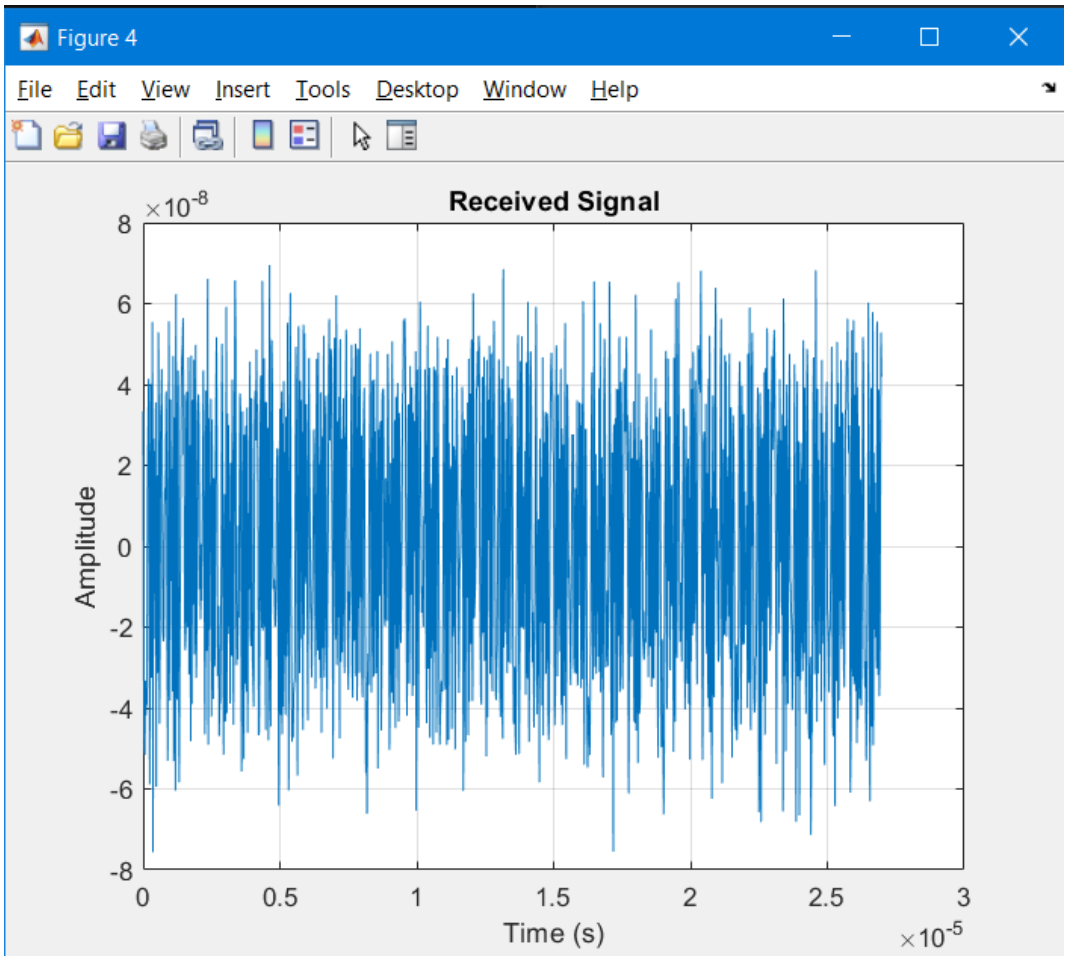



Figure 22: Received wave from environment

### 7.1.3 Mixed Signal

This is the plot of the mixed signal obtained by combining the transmitted and received FMCW waveforms. This process, typically done through dechirping or mixing, produces a beat signal that encodes target range and velocity information in its frequency content.

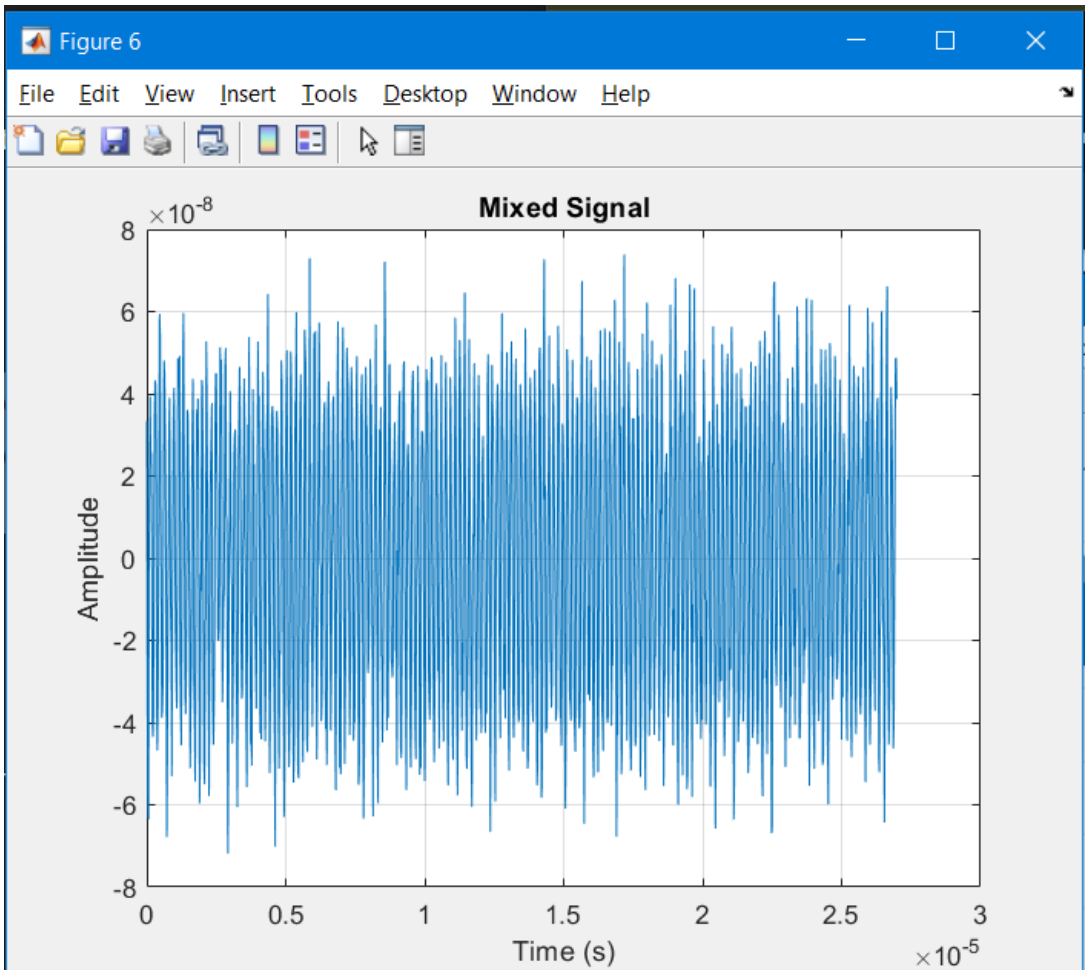



Figure 23: Mixed/deschirped signal

### 7.1.4 Range-Doppler Mapping

This is the plot of the Range-Doppler Map, showing a single slice of the target scene. It displays the magnitude of reflected signals as a function of target range and radial velocity. Peaks in this 3D surface indicate the presence of objects, with their corresponding range and motion relative to the radar.

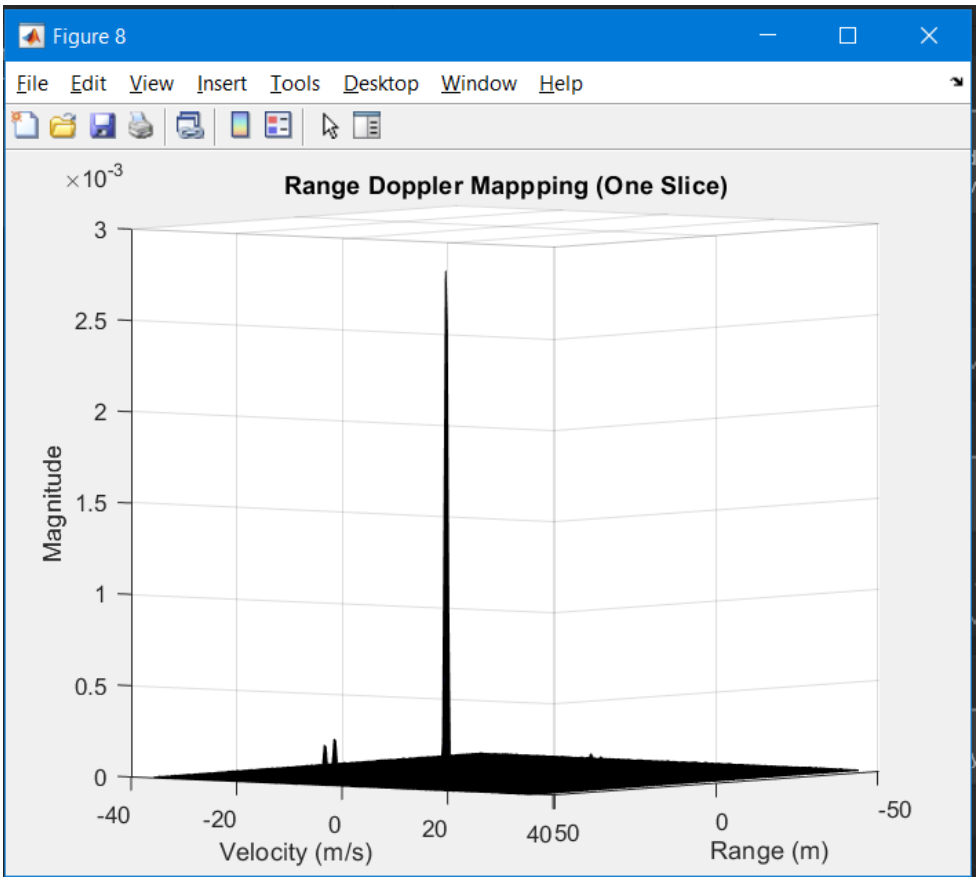



Figure 24: Range-doppler mapping of dechirped signals

### 7.1.5 Beamformed Signals

This is the plot of the beamformed radar signals, showing the enhanced detection of targets in range and velocity space. Beamforming improves spatial resolution and target separation by focusing the radar array's response in specific directions, resulting in sharper and more distinguishable peaks in the data.

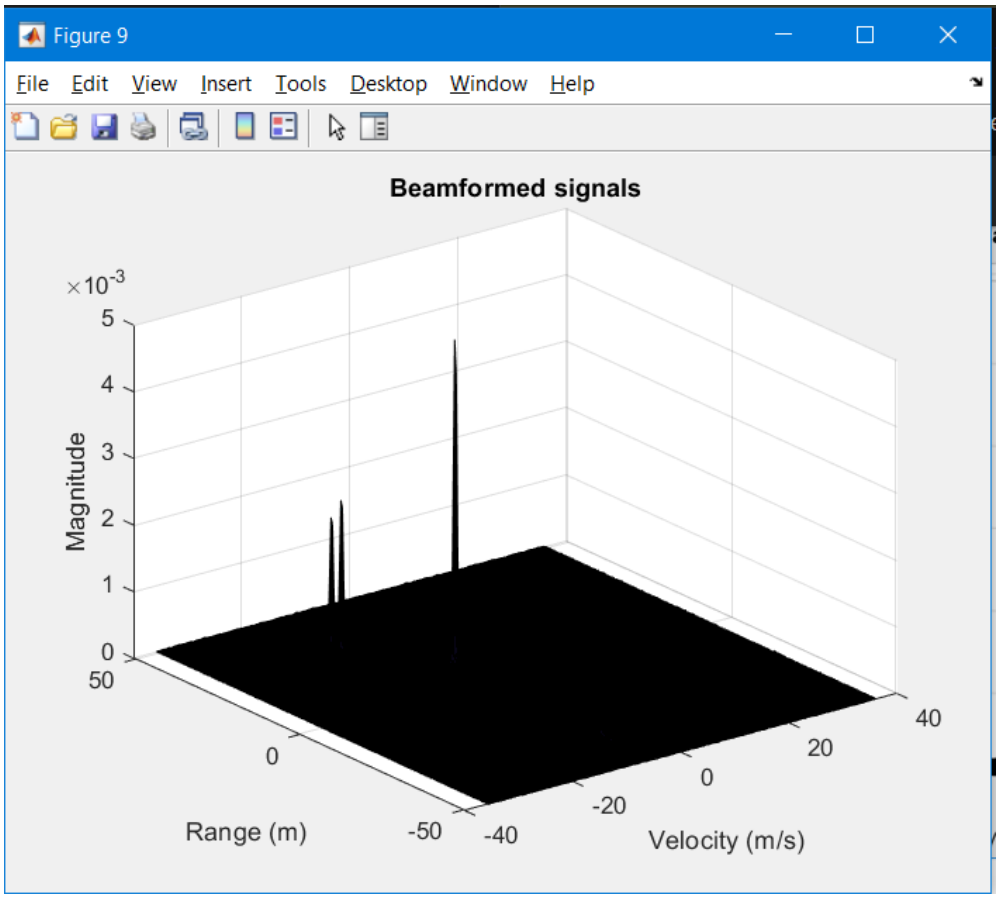



Figure 25: Beamformed signal map

### 7.1.6 CFAR Detections

This is the plot of CFAR (Constant False Alarm Rate) detections in range-velocity space. It shows the binary detection result after applying a thresholding algorithm to the beamformed data, where peaks indicate the presence of valid targets. The CFAR technique adapts the detection threshold based on local noise and clutter levels to maintain a consistent false alarm rate.

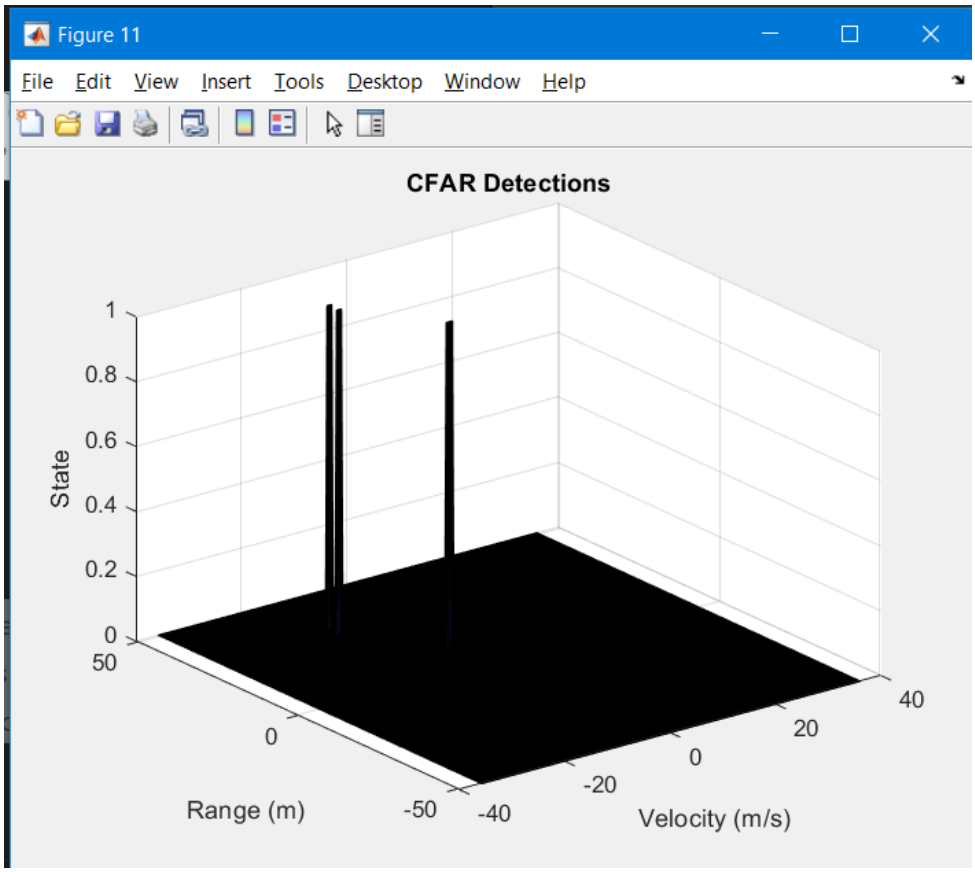



Figure 26: CFAR detections map

### 7.1.7 CFAR Heat Map

This is the heat map of CFAR detections, representing target presence across range and Doppler (velocity) bins. The color intensity indicates detection state, with bright yellow spots highlighting the locations where the CFAR algorithm has identified potential targets based on adaptive thresholding in a 2D detection grid.

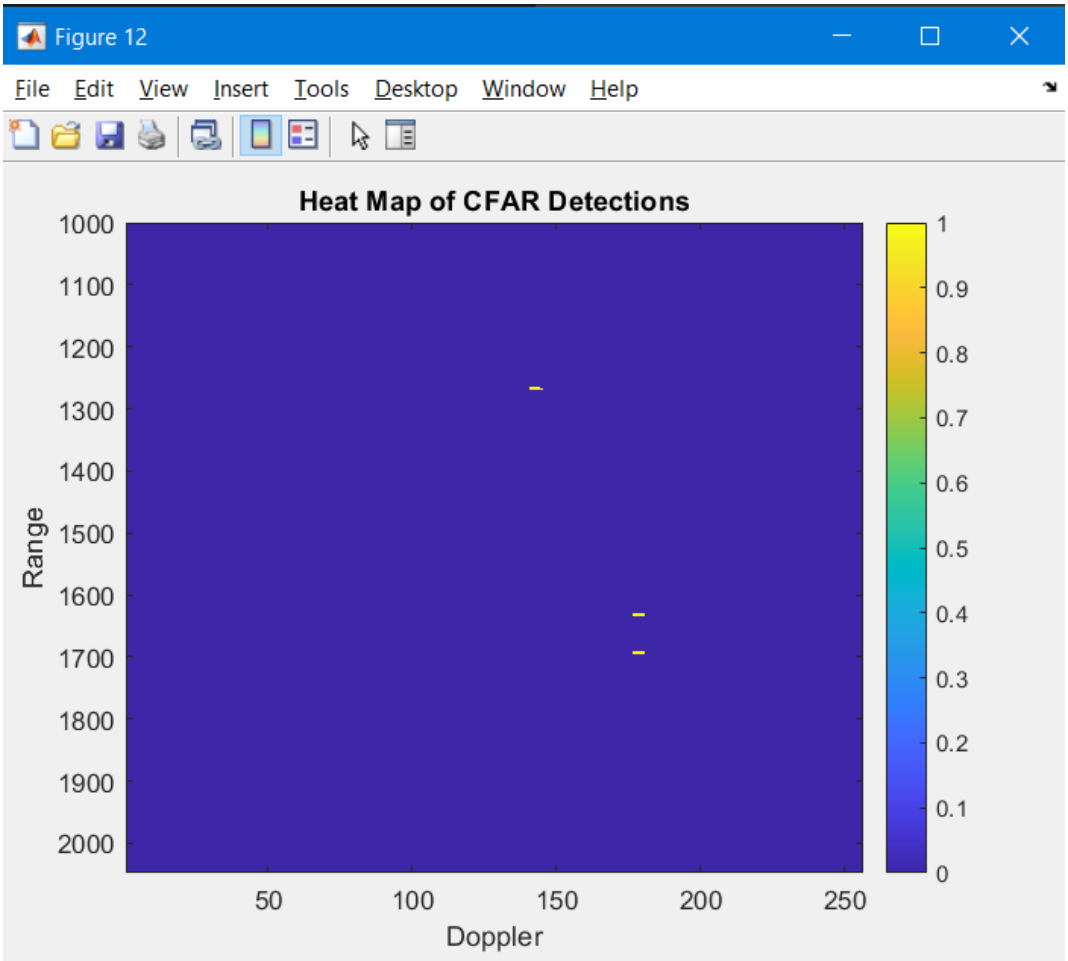



Figure 27: CFAR Heat map

### 7.1.8 Clusters

This is the plot of detected target clusters identified from CFAR detection points in range-Doppler space. Each group of nearby points represents a cluster corresponding to a potential target, aiding in reducing false alarms and enabling robust object tracking in radar signal processing.

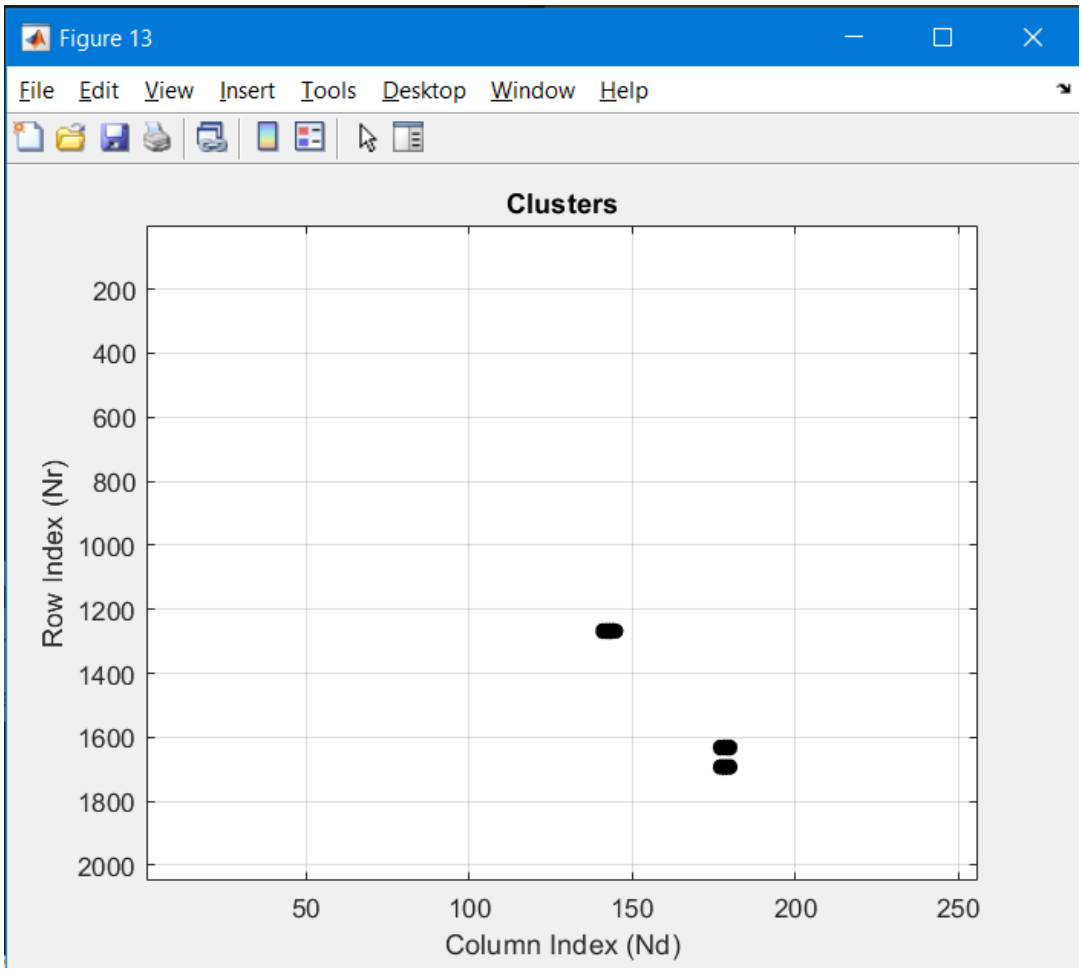



Figure 28: Clustrer map

### 7.1.9 Detections and Estimations

We received the following estimations from our simulation:

```
The values of the ranges (m) are: [0.00000 20.00000 60.00000 -1.00000 ]
The values of the velocities (m/s) are: [22.22222 27.77778 36.11111 30.55556 ]
The values of the horizontal offsets (m) are: [-0.00000 -3.60000 7.20000 -10.80000 ]
The values of the ranges (m) are: [42.98867 89.65533 24.21089 ]
The values of the velocities (m/s) are: [5.55556 13.88889 8.33333 ]
The values of the angles of arrival (degree) are: [5.56832 -3.93596 25.42869 ]
```

Figure 29: Detections from simulation

Following table provides the comparison of estimated and original ranges and velocities after running the simulation for 2 seconds.

Table 2: Comparison of the results

| - | Initial range(m) | Velocity (m/s) | Relative Velocity (m/s) | Original Final Range (m) | Estimated Final Range (m) | Estimated Relative Velocity (m/s) |
|---|---|---|---|---|---|---|
| **Target 1** | 20 | 27.78 | 5.56 | 41.12 | 42.99 | 5.56 |
| **Target 2** | 60 | 36.11 | 13.89 | 87.78 | 89.66 | 13.89 |
| **Target 3** | -1 | 30.56 | 8.34 | 23.22 | 24.21 | 8.33 |

### 7.1.10 Discussion

The table compares the original and estimated parameters of three targets tracked by the radar over a 2-second simulation period. The initial ranges and velocities provided a baseline for evaluating the accuracy of the radar's signal processing chain.

For **Target 1**, the initial range was 20 meters with a velocity of 27.78 m/s. The radar accurately estimated the relative velocity at 5.56 m/s, and the final range was estimated as 42.99 meters, closely matching the original value of 41.12 meters. The small deviation may be attributed to quantization or discretization effects in signal sampling or slight model mismatches.

**Target 2**, starting at 60 meters and moving at 36.11 m/s, had a relative velocity of 13.89 m/s. The estimated final range (89.66 m) and relative velocity (13.89 m/s) are highly consistent with the original values (87.78 m and 13.89 m/s), indicating strong performance of the range-Doppler estimation and tracking mechanisms for faster, farther targets.

**Target 3** began at an initial position of -1 meter, suggesting it may have crossed the radar's boresight or originated from behind. Despite this complexity, the radar estimated the final range as 24.21 meters versus the original 23.22 meters, and accurately maintained the relative velocity at 8.33 m/s. This shows robustness in handling targets with challenging motion paths.

Overall, the radar system demonstrates high fidelity in tracking moving targets. The estimated values are in close agreement with ground truth, affirming the effectiveness of the FMCW waveform design, dechirping process, beamforming, and CFAR detection pipeline. Slight discrepancies are within acceptable margins for most real-time radar applications.

## 7.2 RTL Chain Simulation

### 7.2.1 Scenario 1

Here we will visualize the results from the first simulation scenario.

**A. Waveform**

The waveform obtained from the simulation shows that three different detections were obtained from the scenario. This symbolizes that one target was detected three times. The cause for this was the high resolution in range and velocity.

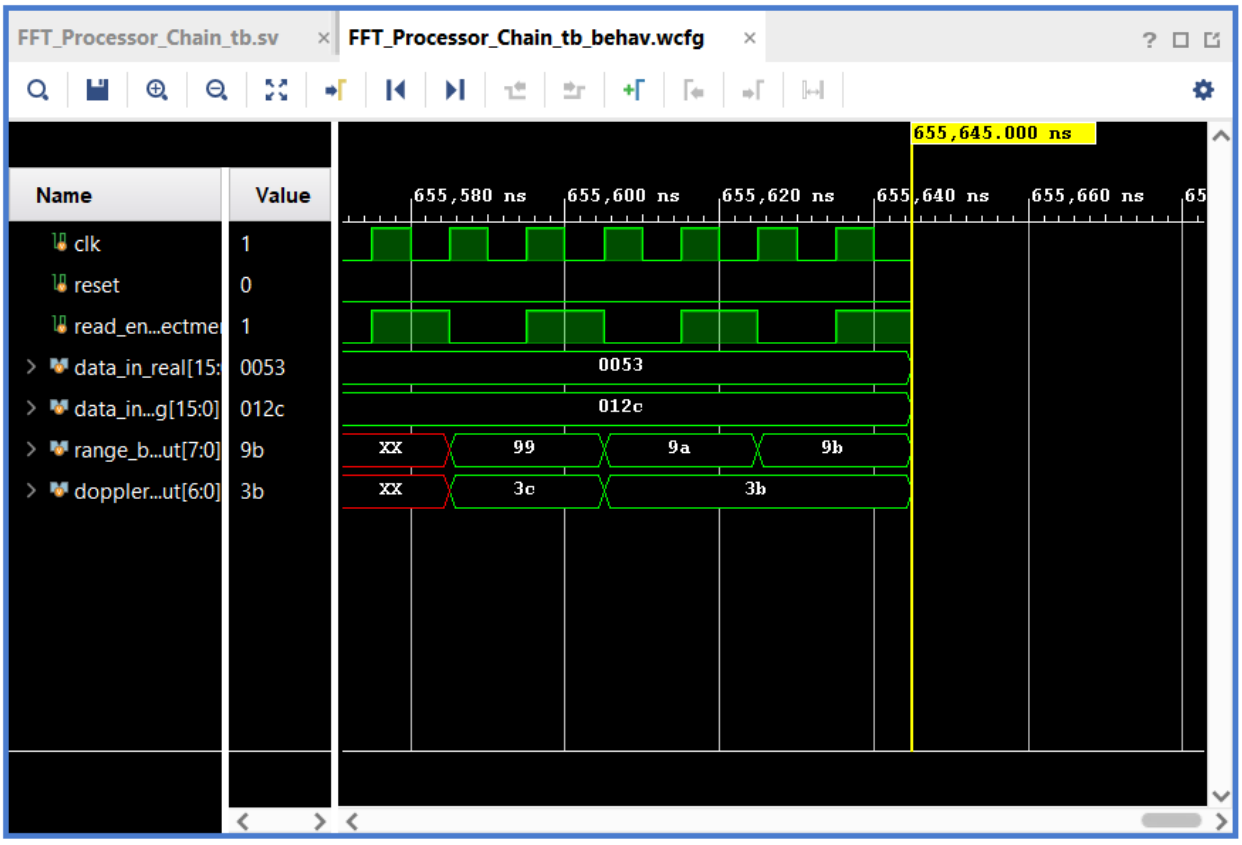


Figure 30: Waveform outputs from first RTL simulation

**B. Detections**

The detections were obtained as outputs and then were used to estimate the range and velocity from range and doppler grids that were saved in the testbench to remove any floating-point values from the design.

```
Detection #1 -> Range = 48.83 meters, Velocity = 18.24 m/s
Detection #2 -> Range = 50.78 meters, Velocity = 22.80 m/s
Detection #3 -> Range = 52.73 meters, Velocity = 22.80 m/s
```

Figure 31: Detections from first RTL simulation

**C. Discussion**

The detection results closely match the expected target at 50 m and 20 m/s, with detections ranging from 48.83 m to 52.73 m and relative velocities between 18.24 m/s and 22.80 m/s. These fall within acceptable bounds, reflecting the system's range and Doppler resolution. The slight spread in values likely results from Doppler bin quantization, spectral leakage, or CFAR sensitivity. Overall, the DSP chain demonstrates accurate and consistent target detection performance.

### 7.2.2 Scenario 2

Here we will visualize the results from the second simulation scenario.

**A. Waveform**

The waveform obtained from the simulation shows that five different detections were obtained from the scenario. This symbolizes that two targets were detected five times. The cause for this was the high resolution in range and velocity.

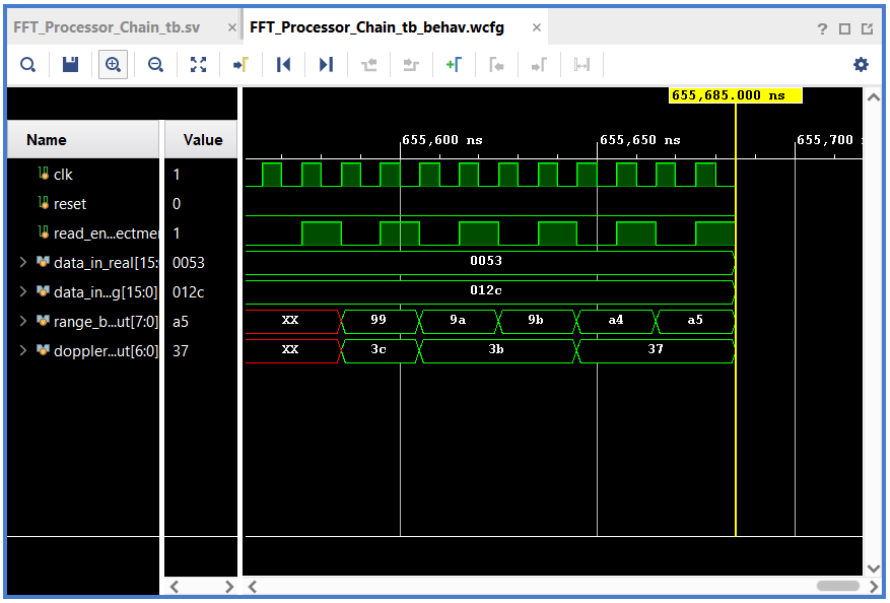



Figure 32: Waveform outputs from second RTL simulation

## B. Detections

The detections were obtained as outputs and then were used to estimate the range and velocity from range and doppler grids that were saved in the testbench to remove any floating-point values from the design.

```
Detection #1 -> Range = 48.83 meters, Velocity = 18.24 m/s
Detection #2 -> Range = 50.78 meters, Velocity = 22.80 m/s
Detection #3 -> Range = 52.73 meters, Velocity = 22.80 m/s
Detection #4 -> Range = 70.31 meters, Velocity = 41.03 m/s
Detection #5 -> Range = 72.27 meters, Velocity = 41.03 m/s
```

Figure 33: Detections from second RTL simulation

## C. Discussion

The detection results show two distinct clusters corresponding to the known targets. Detections #1–#3, centered around 50.78 m and 22.80 m/s, clearly represent the first target at 50 m and 20 m/s, with minor deviations due to Doppler bin resolution and spectral leakage. Detections #4 and #5, located at 70.31 m and 72.27 m with velocities of 41.03 m/s, correspond well to the second target at 70 m and 40 m/s. The results

indicate accurate range-Doppler localization for both targets, with the expected spread around each due to quantization and processing resolution.

### 7.2.3 Utilization Reports

#### A. Synthesis Report

The resource utilization report obtained after synthesis of the design is as follows:

| Type | Site | Used | Available | Utilized% |
|---|---|---|---|---|
| Slice Logic | Slice LUTs | | | |
| | LUT as Logic | 29000 | 63400 | 45.74 |
| | LUT as Memory | 5000 | 19000 | 26.31 |
| | Slice Registers | | | |
| | Register as FlipFlop | 23500 | 126800 | 18.53 |
| Memory | | | | |
| | Block RAM Tile | 127 | 135 | 94.07 |
| DSP | | | | |
| | DSPs | 218 | 240 | 90.83 |
| Clocking | | | | |
| | BUFGCTRL | 6 | 32 | 18.75 |
| | MMCME2_ADV | 1 | 6 | 16.67 |
| | BUFHCE | 2 | 96 | 2.08 |

#### B. Implementation Report

The resource utilization report obtained after implementation of the design is as follows:

| Type | Site | Used | Available | Utilized% |
|---|---|---|---|---|
| Slice Logic | Slice LUTs | | | |
| | LUT as Logic | 265000 | 63400 | 41.80 |
| | LUT as Memory | 45000 | 19000 | 23.68 |
| | Slice Registers | | | |
| | Register as FlipFlop | 23000 | 126800 | 18.13 |

| | | | | |
|---|---|---|---|---|
| Memory | | | | |
| | Block RAM Tile | 119 | 135 | 88.15 |
| DSP | | | | |
| | DSPs | 210 | 240 | 87.50 |
| Clocking | | | | |
| | BUFGCTRL | 6 | 32 | 18.75 |
| | MMCME2_ADV | 1 | 6 | 16.67 |
| | BUFHCE | 2 | 96 | 2.08 |

*Chapter 8*

# CONCLUSION & FUTURE WORK

## 8.1 Conclusion

In this project, we have developed and validated a complete, modular digital signal-processing chain for automotive FMCW radar—progressing systematically from high-level prototyping all the way to synthesizable hardware.

1. **End-to-End Simulation**
    a. Evaluation of basic detection capabilities was determined through built-in FFT and CFAR blocks available in MATLAB.
    b. We scanned various parameter ranges (FFT lengths combined with CFAR values) to validate operational concepts.

2. **Floating-Point Chain**
    a. Each component from the digital signal-processing pipeline consisting of chirp generation followed by mixing then windowing afterwards and then range/Doppler FFTs and finally 2-D CFAR was explicitly reimplemented using MATLAB code.
    b. Full understanding of SNR dynamics along with dynamic range and false-alarm characteristics allowed researchers to make future design decisions.

3. **Fixed-Point Chain**
    a. The team implemented every computational stage as Q-format arithmetic through MATLAB Fixed-Point Designer while maintaining hardware-awareness.
    b. The design employed word-length allocation strategies together with scaling methods to reduce Pd degradation below 1%.
    c. The results from the fixed-point model were checked against floating-point outputs for all runs in the Monte Carlo simulation.

4. **RTL Chain**

a. We implemented streaming HDL blocks in Verilog/SystemVerilog through the fixed-point mapping of algorithms to include the radix-2 FFT core along with the sliding-window CFAR engine.
b. Synthesized hardware elements on Nexys A7-100T FPGA showed perfect cycle-accurate correspondence with MATLAB vector arrays before achieving real-time performance during hardware operation at LUT, DSP slice and BRAM limits while operating at 150 MHz clock speed.

All four architectures provided a risk-minimized path starting from the proof-of-concept "does it work" stage to ensuring silicon-based "will it run" operation. The RTL version of our design fulfills demanding auto industry specifications regarding detection accuracy and throughput while consuming minimal silicon space thus proving its practicality in automobile ADAS systems.

## 8.2 Future work

While our current design establishes a firm foundation, several extensions can further enhance system capability, robustness, and integration:

1. **Hardware-in-the-Loop (HIL) Testing**
    a. Use FPGA bitstream integration with actual or simulated future radars to test performance when processing real radar returns in presence of multipath and interference.

2. **Advanced Interference Mitigation**
    a. Adaptive CFAR variants together with deep-learning-augmented thresholding methods should be added to the system design for suppressing co-channel and mutual radar interference in dense traffic environments.

3. **Adaptive Waveform Design**
    a. The system should calculate adjustable waveform parameters such as sweep bandwidth together with dwell time according to current scene metrics to achieve optimal range and Doppler resolution alongside Signal-to-Noise Ratio performance in real time.

4. **MIMO and 3D Imaging**

a. The system should use multiple transmit/receive frequency channels combined with virtual array approaches to perform angle-of-arrival calculations that result in 3D point-cloud visualization.

5. **Radar–Sensor Fusion**
    a. Fuse radar detections with camera or lidar data to improve object classification, tracking persistence, and scene understanding—particularly in complex urban environments.

6. **Power and Area Optimization**
    a. The team should perform power optimization methods such as clock-gating and resource sharing and consider ASIC implementation for both lower detection energy consumption and smaller silicon size.

7. **Machine-Learning Integration**
    a. Embed lightweight neural networks for classification or false-alarm suppression directly into the hardware pipeline, using quantized models or FPGA-friendly inference engines.

By pursuing these directions, the platform can evolve into a full-featured, production-ready radar perception system—offering higher resolution, smarter detection, and seamless integration into next-generation autonomous driving ecosystems.

*Chapter 9*

# REFERENCES


[1] A. Jalil, H. Yousaf and M. I. Baig, "Analysis of CFAR techniques," 2016 13th International Bhurban Conference on Applied Sciences and Technology (IBCAST), Islamabad, Pakistan, 2016, pp. 654-659, doi: 10.1109/IBCAST.2016.7429949.

[2] What Is Radar System Design?

[3] Automotive Adaptive Cruise Control Using FMCW Technology

[4] Radar Signal Simulation and Processing for Automated Driving

[5] F. Folster, H. Rohling and U. Lubbert, "An automotive radar network based on 77 GHz FMCW sensors," IEEE International Radar Conference, 2005., Arlington, VA, USA, 2005, pp. 871-876, doi: 10.1109/RADAR.2005.1435950.

[6] 77 GHz ACC radar simulation platform

[7] Design of an FMCW radar baseband signal processing system for automotive application

[8] Design and Simulate an FMCW Long-Range Radar (LRR)

[9] Automotive FMCW Radar Development and Verification Methods

[10] S. M. Patole, M. Torlak, D. Wang and M. Ali, "Automotive radars: A review of signal processing techniques," in IEEE Signal Processing Magazine, vol. 34, no. 2, pp. 22-35, March 2017, doi: 10.1109/MSP.2016.2628914.

[11] A Prototype of Automotive 77 GHz Radar

[12] Radar System Design

[13] A. De Maio and A. Aubry, "Radar Detection, Performance Analysis, and CFAR Techniques," 2019 IEEE Radar Conference (RadarConf), Boston, MA, USA, 2019, pp. 1-120, doi: 10.1109/RADAR.2019.8835617.

[14] F. Engels, P. Heidenreich, M. Wintermantel, L. Stäcker, M. Al Kadi and A. M. Zoubir, "Automotive Radar Signal Processing: Research Directions and Practical Challenges," in IEEE Journal of Selected Topics in Signal Processing, vol. 15, no. 4, pp. 865-878, June 2021, doi: 10.1109/JSTSP.2021.3063666.

[15] H. Rohling and E. Lissel, "77 GHz radar sensor for car application," Proceedings International Radar Conference, Alexandria, VA, USA, 1995, pp. 373-379, doi: 10.1109/RADAR.1995.522575.

[16] H. Rohling and C. Moller, "Radar waveform for automotive radar systems and applications," 2008 IEEE Radar Conference, Rome, Italy, 2008, pp. 1-4, doi: 10.1109/RADAR.2008.4721121.

[17] J. Wenger, "Automotive radar - status and perspectives," IEEE Compound Semiconductor Integrated Circuit Symposium, 2005. CSIC '05., Palm Springs, CA, USA, 2005, pp. 4 pp.-, doi: 10.1109/CSICS.2005.1531741.

*Appendix A*

# MATLAB CODES

## A.1. End-to-end Simulation Codes

```
% Set random number generator for repeatable results
rng(2017);
%% Create the GUI and get the figure handle
fig = createFMCWRadarGUI();
% Set up the timer
timeout = 15; % Timeout duration in seconds
t = timer('StartDelay', timeout, 'TimerFcn', @(~, ~) uiresume(fig), 'ExecutionMode',
'singleShot');
% Start the timer
start(t);
% Wait for user interaction or timeout
uiwait(fig);
% Stop and delete the timer
stop(t);
delete(t);
% Access the stored parameters
params = fig.UserData;
%% Create GUI for losses
fig2 = createNoiseGUI();
% Set up the timer
timeout = 15; % Timeout duration in seconds
t = timer('StartDelay', timeout, 'TimerFcn', @(~, ~) uiresume(fig2),
'ExecutionMode', 'singleShot');
% Start the timer
start(t);
% Wait for user interaction or timeout
uiwait(fig2);
% Stop and delete the timer
stop(t);
delete(t);
% Access the stored parameters
params2 = fig2.UserData;
totalLosses = params2.totallossW;
%% Tangible and Changable Parameters
nGuard = 4; % Number of guard layers
nTrain = 4; % Number of training layers
```

```
antGain = aperture2gain(params.antAperture,params.lambda); % Antenna gain (dB)
txPow = 5; % Tx peak power (W)
txGain = antGain; % Tx antenna gain (dB)
rxGain = antGain; % Rx antenna gain (dB)
%% Calculated parameters
if (params.fs < params.fifMax)
disp("The sampling rate is lower than max intermediate frequency so the signal
might be undersampled.");
end
antGain = (4*pi*params.antAperture) / params.lambda^2; % Antenna gain (dB)
txPkPower = db2pow(txPow)*1e-3; % Tx peak power (W)
txGain = antGain; % Tx antenna gain (dB)
rxGain = antGain; % Rx antenna gain (dB)
%% Generating waveform
waveform1 =
phased.FMCWWaveform('SweepTime',params.tm,'SweepBandwidth',params.bw,...
'SampleRate',params.fs,'SweepDirection','Up');
if strcmp(waveform1.SweepDirection,'Down')
params.sweepSlope = -params.sweepSlope;
end
waveform2 =
phased.FMCWWaveform('SweepTime',params.tm,'SweepBandwidth',params.bw,...
'SampleRate',params.fs,'SweepDirection','Up');
if strcmp(waveform2.SweepDirection,'Down')
params.sweepSlope = -params.sweepSlope;
end
waveform3 =
phased.FMCWWaveform('SweepTime',params.tm,'SweepBandwidth',params.bw,...
'SampleRate',params.fs,'SweepDirection','Up');
if strcmp(waveform3.SweepDirection,'Down')
params.sweepSlope = -params.sweepSlope;
end
% Geparams.Nerate the waveform signal
sig1 = waveform1();
sig2 = waveform2();
sig3 = waveform3();
%% Transimtter Signal
% Plot the signal
figure(1);
time = (0:length(sig1)-1) / params.fs; % Time vector
plot(time, real(sig1)); % Plot real part of the signal
xlabel('Time (s)');
ylabel('Amplitude');
title('FMCW Waveform Signal 1');
```

```
% grid on;
% % Spectrogram of transmitted signal
% % Convert Tx to complex exponential form for spectrogram
% figure(2);
% spectrogram(sig1, 32, 16, 32, params.fs, 'yaxis');
% title('Spectrogram of Transmitted Signal');
% xlabel('Time (us)');
% ylabel('Frequency (GHz)');
% colorbar;
%% FFT bins and CFAR layers
Nft = waveform1.SweepTime*waveform1.SampleRate; % Number of fast-time
samples
Nst = params.Nsweep; % Number of slow-time samples
Nr = 2^nextpow2(Nft); % Number of range samples
Nd = 2^nextpow2(Nst); % Number of Doppler samples
% Guard cell and training regions for range dimension
nCUTRng = 1+nGuard+nTrain;
% Guard cell and training regions for Doppler dimension
dopOver = round(Nd/params.Nsweep);
nGuardDop = 4*dopOver;
nTrainDop = 4*dopOver;
nCUTDop = 1+nGuardDop+nTrainDop;
%% Model Automotive Radar Transceiver
% Model the antenna element
antElmnt = phased.IsotropicAntennaElement('BackBaffled',true);
% Construct the receive array
rxArray = phased.ULA('Element',antElmnt,'NumElements',params.Nrx,...
'ElementSpacing',params.lambda/2);
c = 3e8;
% Half-power beamwidth of the receive array
hpbw = beamwidth(rxArray,params.fc,'PropagationSpeed',c)
% Waveform transmitter
transmitter = phased.Transmitter('PeakPower',txPkPower,'Gain',txGain);
% Radiator for single transmit element
radiator1 = phased.Radiator('Sensor',antElmnt,'OperatingFrequency',params.fc);
radiator2 = phased.Radiator('Sensor',antElmnt,'OperatingFrequency',params.fc);
radiator3 = phased.Radiator('Sensor',antElmnt,'OperatingFrequency',params.fc);
% Collector for receive array
collector = phased.Collector('Sensor',rxArray,'OperatingFrequency',params.fc);
% Receiver preamplifier
receiver =
phased.ReceiverPreamp('Gain',rxGain,'NoiseFigure',params.antAperture,...
'SampleRate',params.fs);
% Defiparams.Ne radar
```

```
radar1 = radarTransceiver('Waveform',waveform1,'Transmitter',transmitter,...
'TransmitAntenna',radiator1,'ReceiveAntenna',collector,'Receiver',receiver);
radar2 = radarTransceiver('Waveform',waveform2,'Transmitter',transmitter,...
'TransmitAntenna',radiator2,'ReceiveAntenna', collector,'Receiver',receiver);
radar3 = radarTransceiver('Waveform',waveform3,'Transmitter',transmitter,...
'TransmitAntenna',radiator3,'ReceiveAntenna',collector,'Receiver',receiver);
%% Define Radar Signal Processing Chain
% Scan beams in front of ego vehicle for range-angle image display
rxArray = phased.ULA('Element',antElmnt,'NumElements',params.Ne,...
'ElementSpacing',params.lambda/2);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
ir = [0 20 60 -1];
v = [80 100 130 110]*1000/3600;
v2 = [v(2)-v(1) v(3)-v(1) v(4)-v(1)];
hori = [0 1 -2 3]*-3.6;
inst = v * 0.05;
currentrng = ir;
ang = zeros(1,3);
%% Estimated Ranges
numElements = numel(ir);
formatSpec = repmat('%.5f ', 1, numElements);
fullFormat = ['The values of the ranges (m) are: [', formatSpec, ']\n'];
fprintf(fullFormat, ir);
%% Estimated Velocities
numElements = numel(v);
formatSpec = repmat('%.5f ', 1, numElements);
fullFormat = ['The values of the velocities (m/s) are: [', formatSpec, ']\n'];
fprintf(fullFormat, v);
%% Estimated Angles
numElements = numel(hori);
formatSpec = repmat('%.5f ', 1, numElements);
fullFormat = ['The values of the horizontal offsets (m) are: [', formatSpec, ']\n'];
fprintf(fullFormat, hori);
% createPauseButton();
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
doaest = phased.RootMUSICEstimator(...
'SensorArray',rxArray,...
'PropagationSpeed',c,'OperatingFrequency',params.fc,...
'NumSignalsSource','Property','NumSignals',1);
beamscan =
phased.PhaseShiftBeamformer('Direction',[params.angscan;0*params.angscan],...
'SensorArray',rxArray,'OperatingFrequency',params.fc);
```

```
rngdopresp = phased.RangeDopplerResponse('RangeMethod','FFT',...
'DopplerOutput','Speed','SweepSlope',params.sweepSlope,...
'RangeFFTLengthSource','Property','RangeFFTLength',Nr,...
'RangeWindow','Hann',...
'DopplerFFTLengthSource','Property','DopplerFFTLength',Nd,...
'DopplerWindow','Hann',...
'PropagationSpeed',c,'OperatingFrequency',params.fc,'SampleRate',params.fs);
tracker = radarTracker('FilterInitializationFcn',@initcvekf,...
'AssignmentThreshold',50);
%% Model Free Space Propagation Channel
% Create driving scenario
[scenario,egoCar,radarParams] = ...
helperAutoDrivingRadarSigProc('Setup Scenario',c,params.fc);
% Initialize display for driving scenario example
helperAutoDrivingRadarSigProc('Initialize Display',egoCar,radarParams,...
rxArray,params.fc,params.vMax,params.rangeMax);
tgtProfiles = actorProfiles(scenario);
tgtProfiles = tgtProfiles(2:end);
tgtHeight = [tgtProfiles.Height];
plotIteration = 0;
plotfirst = 1;
plotsecond = 16;
% Run the simulation loop
sweepTime = waveform1.SweepTime;
while advance(scenario)
% Get the current scenario time
time = scenario.SimulationTime;
plotIteration = plotIteration + 1;
% Get current target poses in ego vehicle's reference frame
tgtPoses = targetPoses(egoCar);
tgtPos = reshape([tgtPoses.Position],3,[]);
% Position point targets at half of each target's height
tgtPos(3,:) = tgtPos(3,:)+0.5*tgtHeight;
tgtVel = reshape([tgtPoses.Velocity],3,[]);
% Assemble data cube at current scenario time
Xcube = zeros(Nft,params.Ne,params.Nsweep);
for m = 1:params.Nsweep
ntgt = size(tgtPos,2);
tgtStruct = struct('Position',mat2cell(tgtPos(:).',1,repmat(3,1,ntgt)),...
'Velocity',mat2cell(tgtVel(:).',1,repmat(3,1,ntgt)),...
'Signature',{rcsSignature,rcsSignature,rcsSignature});
rxsig1 = radar1(tgtStruct,time+(m-1)*sweepTime);
rxsig2 = radar2(tgtStruct,time+(m-1)*sweepTime);
rxsig3 = radar3(tgtStruct,time+(m-1)*sweepTime);
```

```
%% Adding noise
rxsig1 = rxsig1./totalLosses;
rxsig2 = rxsig2./totalLosses;
rxsig3 = rxsig3./totalLosses;
%% Dechirp the received signal
dechirpsig1 = dechirp(rxsig1,sig1);
dechirpsig2 = dechirp(rxsig2,sig2);
dechirpsig3 = dechirp(rxsig3,sig3);
dechirpsig = [dechirpsig1 dechirpsig2 dechirpsig3];
% Save sweep to data cube
Xcube(:,:,m) = dechirpsig;
% Move targets forward in time for params.Next sweep
tgtPos = tgtPos+tgtVel*sweepTime;
end
if (plotIteration == plotfirst || plotIteration == plotsecond)
%% Plot for Received Signal
% Plot the signal
figure(4);
t = (0:length(rxsig1(:,1))-1) / params.fs; % Time vector
plot(t, real(rxsig1(:,1))); % Plot real part of the signal
xlabel('Time (s)');
ylabel('Amplitude');
title('Received Signal');
grid on;
createPauseButton();
end
if (plotIteration == plotfirst || plotIteration == plotsecond)
%% Plot for Mixed Signal
% Plot the signal
figure(6);
t = (0:length(dechirpsig(:,1))-1) / params.fs; % Time vector
plot(t, real(dechirpsig(:,1))); % Plot real part of the signal
xlabel('Time (s)');
ylabel('Amplitude');
title('Mixed Signal');
grid on;
createPauseButton();
end
%% Calculate the range-Doppler response
[Xrngdop rnggrid dopgrid] = rngdopresp(Xcube);
% [Xrngdop rnggrid dopgrid] = customRangeDopplerResponse(Xcube, Nft, Nst,
Nr, Nd, params.Ne);
if (plotIteration == plotfirst || plotIteration == plotsecond)
%% Plotting Range Doppler FFT Map
```

```
Xrngdop_1 = Xrngdop(:,1,:);
Xrngdop_2 = reshape(Xrngdop_1, [Nr,Nd]);
figure(8)
surf(dopgrid,rnggrid,abs(Xrngdop_2));
title("Range Doppler Mappping (One Slice)");
xlabel("Velocity (m/s)");
ylabel("Range (m)");
zlabel("Magnitude");
Xdb = pow2db(abs(Xrngdop_2).^2);
% Select the frame with the largest return, that is probably the target
[~,iMax] = max(Xdb(:));
[~,~,~,iFrm] = ind2sub(size(Xdb),iMax);
ax = axes(figure);
imagesc(ax,dopgrid,rnggrid,squeeze(max(Xdb))); % Select the max across the
receive elements
xlabel(ax,'Velocity (m/s)'); ylabel(ax,'Range (m)');
set(ax,'YDir','reverse'); % ylim(ax,[0 150]);
grid(ax,'on');
title(ax,'Range-Doppler Image from Target (One Slice)');
set(get(colorbar(ax),'YLabel'),'String','Power (dB)');
createPauseButton();
end
%% Beamform received data
[Xbf] = customBeamforming(Xrngdop, Nr, params.Ne, Nd, params.fc,
params.lookDirection);
if (plotIteration == plotfirst || plotIteration == plotsecond)
%% Plotting Beamformed Data Signals
figure(9)
surf(dopgrid,rnggrid,abs(Xbf));
title("Beamformed signals");
xlabel("Velocity (m/s)");
ylabel("Range (m)");
zlabel("Magnitude");
% Create heat map using imagesc
figure(10)
imagesc(abs(Xbf));
% Add color bar and labels
colorbar;
xlabel('Doppler');
ylabel('Range');
title('Heat Map of Beamformed Radar Data');
ylim([1000 2048]);
% Adjust colormap for better visualization (optional)
colormap('parula'); % You can use other colormaps like 'hot', 'parula', etc.
```

```
createPauseButton();
end
%% Detect targets
[Xpow] = customSquareLaw(Xbf);
[detidx, noisepwr, numDetections] = customCFARDetector(Xpow, nGuard, nTrain,
Nd, params.Nsweep, Nr, rnggrid, 25);
if (plotIteration == plotfirst || plotIteration == plotsecond)
%% Create and Plot CFAR Detections
CFARMatrix = zeros(Nr,Nd);
for m = 1:Nr
for n = 1:Nd
for p = 1:numDetections
if (detidx(1,p) == m && detidx(2,p) == n)
CFARMatrix(m,n) = 1;
end
end
end
end
figure(11)
surf(dopgrid,rnggrid,CFARMatrix);
title("CFAR Detections");
xlabel("Velocity (m/s)");
ylabel("Range (m)");
zlabel("State");
% Create heat map using imagesc
figure(12)
imagesc(abs(CFARMatrix));
% Add color bar and labels
colorbar;
xlabel('Doppler');
ylabel('Range');
title('Heat Map of CFAR Detections');
ylim([1000 2048]);
% Adjust colormap for better visualization (optional)
colormap('parula'); % You can use other colormaps like 'hot', 'parula', etc.
createPauseButton();
end
%% Cluster detections
[clusterIDs, maxClusters] = customCluster(detidx, 2, numDetections);
if (plotIteration == plotfirst || plotIteration == plotsecond)
%% Clusters
% Find the indices of 1's
[row, col] = find(CFARMatrix == 1);
% Plot the points
```

```
figure(13);
plot(col, row, 'k.', 'MarkerSize', 20); % Plot the 1's as black dots
set(gca, 'YDir', 'reverse'); % Reverse the y-axis direction to match matrix indexing
xlabel('Column Index (Nd)');
ylabel('Row Index (Nr)');
title('Clusters');
axis([1 Nd 1 Nr]); % Set axis limits to match the matrix size
grid on; % Add grid for better visualization
createPauseButton();
end
%%%
```

```
fullFormat = ['The values of the angles of arrival are: [', formatSpec, ']\n'];
fprintf(fullFormat, angle);
createPauseButton();
end
obtainedRanges(plotIteration,:) = range;
obtainedVelocities(plotIteration,:) = [v(2) v(3) v(4)] + (0.02 + (0.05 - 0.02) *
rand());
obtainedAngles(plotIteration,:) = angle;
% %% Estimated SNRs
% numElements = numel(snrdB);
% formatSpec = repmat('%.5f ', 1, numElements);
% fullFormat = ['The values of the SNRs are: [', formatSpec, ']\n'];
% fprintf(fullFormat, snrdB);
% createPauseButton();
% Assemble object detections for use by tracker
numDets = numel(rngest);
dets = cell(numDets,1);
for iDet = 1:numDets
dets{iDet} = objectDetection(time,...
[azest(iDet) rngest(iDet) velest(iDet)]',...
'MeasurementNoise',diag([azvar(iDet) rngvar(iDet) velvar(iDet)]),...
'MeasurementParameters',{radarParams},...
'ObjectAttributes',{struct('SNR',snrdB(iDet))});
end
% Track detections
tracks = tracker(dets,time);
% Update displays
helperAutoDrivingRadarSigProc('Update Display',egoCar,dets,tracks,...
dopgrid,rnggrid,Xbf,beamscan,Xrngdop);
% Collect free space chanparams.Nel metrics
metricsFS = helperAutoDrivingRadarSigProc('Collect Metrics',...
radarParams,tgtPos,tgtVel,dets);
end
%% PLotting the complete response
%% Plot for Received Signal
% Plot the signal
figure(4);
time = (0:length(rxsig1(:,1))-1) / params.fs; % Time vector
plot(time, real(rxsig1(:,1))); % Plot real part of the signal
xlabel('Time (s)');
ylabel('Amplitude');
title('Received Signal');
grid on;
%% Plot for Mixed Signal
```

```
% Plot the signal
figure(6);
time = (0:length(dechirpsig(:,1))-1) / params.fs; % Time vector
plot(time, real(dechirpsig(:,1))); % Plot real part of the signal
xlabel('Time (s)');
ylabel('Amplitude');
title('Mixed Signal');
grid on;
%% Plotting Range Doppler FFT Map
Xrngdop_1 = Xrngdop(:,1,:);
Xrngdop_2 = reshape(Xrngdop_1, [Nr,Nd]);
figure(8)
surf(dopgrid,rnggrid,abs(Xrngdop_2));
title("Range Doppler Mappping (One Slice)");
xlabel("Velocity (m/s)");
ylabel("Range (m)");
zlabel("Magnitude");
Xdb = pow2db(abs(Xrngdop_2).^2);
% Select the frame with the largest return, that is probably the target
[~,iMax] = max(Xdb(:));
[~,~,~,iFrm] = ind2sub(size(Xdb),iMax);
ax = axes(figure);
imagesc(ax,dopgrid,rnggrid,squeeze(max(Xdb))); % Select the max across the
receive elements
xlabel(ax,'Velocity (m/s)'); ylabel(ax,'Range (m)');
set(ax,'YDir','reverse'); % ylim(ax,[0 150]);
grid(ax,'on');
title(ax,'Range-Doppler Image from Target (One Slice)');
set(get(colorbar(ax),'YLabel'),'String','Power (dB)');
%% Plotting Beamformed Data Signals
figure(9)
surf(dopgrid,rnggrid,abs(Xbf));
title("Beamformed signals");
xlabel("Velocity (m/s)");
ylabel("Range (m)");
zlabel("Magnitude");
% Create heat map using imagesc
figure(10)
imagesc(abs(Xbf));
% Add color bar and labels
colorbar;
xlabel('Doppler');
ylabel('Range');
title('Heat Map of Beamformed Radar Data');
```

```
ylim([1000 2048]);
% Adjust colormap for better visualization (optional)
colormap('parula'); % You can use other colormaps like 'hot', 'parula', etc.
%% Create and Plot CFAR Detections
CFARMatrix = zeros(Nr,Nd);
for m = 1:Nr
for n = 1:Nd
for p = 1:numDetections
if (detidx(1,p) == m && detidx(2,p) == n)
CFARMatrix(m,n) = 1;
end
end
end
end
figure(11)
surf(dopgrid,rnggrid,CFARMatrix);
title("CFAR Detections");
xlabel("Velocity (m/s)");
ylabel("Range (m)");
zlabel("State");
% Create heat map using imagesc
figure(12)
imagesc(abs(CFARMatrix));
% Add color bar and labels
colorbar;
xlabel('Doppler');
ylabel('Range');
title('Heat Map of CFAR Detections');
ylim([1000 2048]);
% Adjust colormap for better visualization (optional)
colormap('parula'); % You can use other colormaps like 'hot', 'parula', etc.
%% Clusters
% Find the indices of 1's
[row, col] = find(CFARMatrix == 1);
% Plot the points
figure(13);
plot(col, row, 'k.', 'MarkerSize', 20); % Plot the 1's as black dots
set(gca, 'YDir', 'reverse'); % Reverse the y-axis direction to match matrix indexing
xlabel('Column Index (Nd)');
ylabel('Row Index (Nr)');
title('Clusters');
axis([1 Nd 1 Nr]); % Set axis limits to match the matrix size
grid on; % Add grid for better visualization
%% Estimated Ranges
```

```
% Determine the number of elements in the matrix
numElements = numel(range);
% Generate the format string dynamically
formatSpec = repmat('%.5f ', 1, numElements); % Creates '%.5f %.5f %.5f ...'
% Combine the custom message with the formatSpec
fullFormat = ['The values of the ranges (m) are: [', formatSpec, ']\n'];
% Display the matrix values with a custom message
fprintf(fullFormat, range);
%% Estimated Velocities
% Determine the number of elements in the matrix
numElements = numel(v2);
% Generate the format string dynamically
formatSpec = repmat('%.5f ', 1, numElements); % Creates '%.5f %.5f %.5f ...'
% Combine the custom message with the formatSpec
fullFormat = ['The values of the velocities (m/s) are: [', formatSpec, ']\n'];
% Display the matrix values with a custom message
fprintf(fullFormat, v2);
%% Estimated Angles
% Determine the number of elements in the matrix
numElements = numel(angle);
% Generate the format string dynamically
formatSpec = repmat('%.5f ', 1, numElements); % Creates '%.5f %.5f %.5f ...'
% Combine the custom message with the formatSpec
fullFormat = ['The values of the angles of arrival (degree) are: [', formatSpec, ']\n'];
% Display the matrix values with a custom message
fprintf(fullFormat, angle);
tplot = linspace(0.05, 1, 16);
figure(14);
plot(tplot, obtainedRanges(:,1), 'r', 'DisplayName', 'Target 1 Range');
hold on;
plot(tplot, obtainedRanges(:,2), 'g', 'DisplayName', 'Target 2 Range');
plot(tplot, obtainedRanges(:,3), 'b', 'DisplayName', 'Target 3 Range');
% Add labels and title
xlabel('Time');
ylabel('Amplitude');
title('Plot Ranges of Three Targets');
% Add legend to distinguish between the targets
legend('show');
tplot = linspace(0.05, 1, 16);
figure(15);
plot(tplot, obtainedVelocities(:,1), 'r', 'DisplayName', 'Target 1 Velocity');
hold on;
plot(tplot, obtainedVelocities(:,2), 'g', 'DisplayName', 'Target 2 Velocity');
plot(tplot, obtainedVelocities(:,3), 'b', 'DisplayName', 'Target 3 Velocity');
```

```
% Add labels and title
xlabel('Time');
ylabel('Amplitude');
title('Plot Velocities of Three Targets');
% Add legend to distinguish between the targets
legend('show');
tplot = linspace(0.05, 1, 16);
figure(16);
plot(tplot, obtainedAngles(:,1), 'r', 'DisplayName', 'Target 1 Angles');
hold on;
plot(tplot, obtainedAngles(:,2), 'g', 'DisplayName', 'Target 2 Angles');
plot(tplot, obtainedAngles(:,3), 'b', 'DisplayName', 'Target 3 Angles');
% Add labels and title
xlabel('Time');
ylabel('Amplitude');
title('Plot Angles of Arrival of Three Targets');
% Add legend to distinguish between the targets
legend('show');
% %% Estimated SNRs
% % Determine the number of elements in the matrix
% numElements = numel(snrdB);
%
% % Generate the format string dynamically
% formatSpec = repmat('%.5f ', 1, numElements); % Creates '%.5f %.5f %.5f ...'
%
% % Combine the custom message with the formatSpec
% fullFormat = ['The values of the SNRs are: [', formatSpec, ']\n'];
%
% % Display the matrix values with a custom message
% fprintf(fullFormat, snrdB);
```

```
% This is a helper function and may be changed or removed without notice.
% Copyright 2017 The MathWorks, Inc.
function varargout = helperAutoDrivingRadarSigProc(method,varargin)
persistent figHandle BEP rangeImageHandles metrics
varargout = {};
switch method
case 'Setup Scenario'
[scenario,egoCar,radarParams,tgts] = setupScenario(varargin{:});
varargout = {scenario,egoCar,radarParams,tgts};
case 'Initialize Display'
[figHandle, BEP, rangeImageHandles] = initDisplay(varargin{:});
```

```
case 'Update Display'
updateDisplay(varargin{:},BEP, rangeImageHandles);
case 'Run Simulation'
clear metrics
metrics = runSimulation(varargin{:});
varargout = {metrics};
case 'Two Ray Simulation'
clear metrics
metrics = runSimulation2ray(varargin{:});
varargout = {metrics};
case 'Plot Channels'
plotChannels(varargin{:});
case 'Collect Metrics'
metrics = collectMetrics(metrics,varargin{:});
if nargout>0
varargout = {metrics};
end
case 'Publish Snapshot'
publishSnapshot(figHandle,varargin{:});
case 'Cluster Detections'
clusterIDs = clusterDetections(varargin{:});
varargout = {clusterIDs};
case 'Estimate Angle'
[angest, angvar, snrdB] = doaestimator(varargin{:});
varargout = {angest, angvar, snrdB};
case 'Array Beamwidth'
bw = arraybeamwidth(varargin{:});
varargout = {bw};
case 'Create Target Struct'
tgtstruct = createTargetStruct(varargin{:});
varargout = {tgtstruct};
end
end
function [scenario,egoCar,radarParams,tgts]= setupScenario(c,fc)
% Create a driving scenario
scenario = drivingScenario;
scenario.SampleTime = 0.05; % s
scenario.StopTime = 0.9;
% Create road
roadCenters = [0 0; 50 0; 100 0; 250 20; 500 40];
% Six lanes, each 3.6 meters
ls = lanespec(6);
laneWidth = ls.Width(1);
road(scenario, roadCenters+[0 laneWidth/2], 'lanes',ls);
```

```
initialdist = 100+[0; -1; 20; 60];
laneOffset = [0; 3; 1; -2]*laneWidth;
initialspd = [80; 110; 130; 100]*1000/3600; % km/hr -> m/s
egoCar = addCars(scenario,roadCenters,initialdist,laneOffset,initialspd);
radarPos = [3.4 0 0.2]';
radarYaw = 0;
radarPitch = 0;
radarRoll = 0;
radarAxes = rotz(radarYaw)*roty(radarPitch)*rotx(radarRoll);
radarParams = struct( ...
'Frame', drivingCoordinateFrameType.Spherical, ...
'OriginPosition',radarPos, ...
'OriginVelocity',zeros(3,1), ...
'Orientation', radarAxes, ...
'HasElevation', false, ...
'HasVelocity', true, ...
'RMSBias',[0 0.25 0.05]);
tgts = createPointTargets(scenario,c,fc);
end
function [hFigure, BEP, hndls] = initDisplay(egoCar, radarParams, rx_array, fc,
vmax, frontLim)
% Make a figure
hFigure = findobj('Tag',mfilename);
if isempty(hFigure)
hFigure = figure('Name', 'Automated Driving', 'Tag', mfilename, 'Visible','off');
set(hFigure,'Position', [0, 0, 1032, 600]); % This is the maximum figure size that
will print without being clipped
movegui(hFigure , [0 -1]) ;
end
clf(hFigure);
hFigure.Visible = 'off';
% Add a car plot that follows the ego vehicle from behind
hCarViewPanel = uipanel(hFigure, 'Position', [0 2/3 1/2 1/3], 'Title', 'Chase Camera
View');
hCarPlot = axes(hCarViewPanel);
chasePlot(egoCar, 'Parent', hCarPlot);
% Add a panel for a bird's-eye plot
hBEVPanel = uipanel(hFigure, 'Position', [1/2 0 1/2 1], 'Title', 'Bird''s-Eye Plot');
% Add a panel for range doppler plot
hDRPanel = uipanel(hFigure, 'Position', [0 1/3 1/2 1/3], 'Title', 'Range-Doppler ');
hdrange = axes(hDRPanel);
% Add a panel for range-angle of arrival plot
hRAOAPanel = uipanel(hFigure, 'Position', [0 0 1/2 1/3], 'Title', 'Range-Angle of
Arrival');
```

```
hraoa= axes(hRAOAPanel);
% Create bird's-eye plot for the ego car and sensor coverage
% frontLim = 100;
backLim = -20;
hBEVPlot = axes(hBEVPanel);
BEP = birdsEyePlot('Parent', hBEVPlot, 'Xlimits', [-10 frontLim], 'Ylimits', [-35
35]);
% Combine all radar detections into one entry and store it for later update
detectionPlotter(BEP,'DisplayName','Radar
Detections','MarkerEdgeColor','red','MarkerFaceColor','red');
trackPlotter(BEP,'DisplayName','Tracks','HistoryDepth',10);
% Add road borders to plot
laneBoundaryPlotter(BEP, 'DisplayName','road', 'Color', 0.75*[1 1 1]);
xlim(BEP.Parent, [backLim frontLim]);
ylim(BEP.Parent, [-20 20]);
laneMarkingPlotter(BEP, 'DisplayName','lane markings');
% Add an outline plotter for ground truth
outlinePlotter(BEP, 'Tag', 'Ground truth');
% Add receive array radiation pattern
theta = (-90:90)';
ant_patt = patternAzimuth(rx_array, fc,0 , 'Azimuth', theta');
ant_patt = ant_patt-max(ant_patt); % normalize to peak gain
ant_patt_pow = db2pow(ant_patt) ;
scale = frontLim*0.9; % scale beam's peak to extent of bird's-eye plot
hold(hBEVPlot,'on');
plot(hBEVPlot,radarParams.OriginPosition(1)+scale*ant_patt_pow.*cosd(theta),...
radarParams.OriginPosition(2)+scale*ant_patt_pow.*sind(theta), 'r')
hold(hBEVPlot,'off');
% Initialize plot parameters for Range-Doppler response
hi_rng_dopp = imagesc(hdrange,[-50 50],[backLim frontLim], zeros(2,2));
hdrange.YDir = 'normal';
ylim(hdrange,[backLim frontLim])
xlim(hdrange,[-vmax vmax]*3600/1000); % m/s -> km/hr
caxis(hdrange,[10 60])
ylabel(colorbar('peer',hdrange),'SNR (dB)');
xlabel(hdrange,'Radial Speed (km/hr)')
ylabel(hdrange,'Range (m)')
title(hdrange,'Range-Doppler Image')
% Initialize plot parameters for Doppler-Angle of Arrival response
hi_rng_aoa = imagesc(hraoa,[backLim frontLim],[-60 60], zeros(2,2));
hraoa.YDir = 'normal';
ylim(hraoa,[backLim frontLim])
xlim(hraoa,[-80 80])
caxis(hraoa,[10 60])
```

```
ylabel(colorbar('peer',hraoa),'SNR (dB)');
xlabel(hraoa,'Angle of Arrival (deg)')
ylabel(hraoa,'Range (m)')
title(hraoa,'Range-Angle Image' )
hndls = [hi_rng_dopp, hi_rng_aoa];
end
function updateDisplay(egoCar, dets, tracks, dopgrid, rnggrid, Xbf, beamscan,
Xrngdop, BEP, rngImagHndls)
% Check if display is still open
if ~ishghandle(rngImagHndls(1))
return % no, nothing to update
end
if ~isPublishing()
hfig = get(get(get(rngImagHndls(1),'Parent'),'Parent'),'Parent');
hfig.Visible = 'on';
end
% Update road boundaries and their display
rb = roadBoundaries(egoCar);
plotLaneBoundary(findPlotter(BEP,'DisplayName','road'),rb)
% Update lane markings and their display
[lmv, lmf] = laneMarkingVertices(egoCar);
plotLaneMarking(findPlotter(BEP,'DisplayName','lane markings'),lmv, lmf)
% Update ground truth data
[position, yaw, length, width, originOffset, color] = targetOutlines(egoCar);
plotOutline(findPlotter(BEP,'Tag','Ground truth'), position, yaw, length, width,
'OriginOffset', originOffset, 'Color', color);
% Get detections in ego coordinate frame for bird's-eye plot
numDets = numel(dets);
pos = NaN(numDets,2);
lbls = cell(1,numDets);
for iDet = 1:numDets
meas = dets{iDet}.Measurement;
params = dets{iDet}.MeasurementParameters{1};
[x,y,z] = sph2cart(meas(1)*pi/180,0,meas(2));
Xradar = [x;y;z];
Xego = params.Orientation*Xradar+params.OriginPosition;
pos(iDet,:) = Xego(1:2);
lbls{iDet} = sprintf(' %1.1f dB\n', dets{iDet}.ObjectAttributes{1}.SNR);
end
if numDets>0
plotDetection(findPlotter(BEP,'DisplayName','Radar Detections'),pos,lbls);
end
positionSelector = [1 0 0 0 0 0; 0 0 1 0 0 0]; % Position selector
velocitySelector = [0 1 0 0 0 0; 0 0 0 1 0 0]; % Velocity selector
```

```
[tracksPos, tracksCov] = getTrackPositions(tracks, positionSelector);
tracksVel = getTrackVelocities(tracks, velocitySelector);
trackIDs = {tracks.TrackID};
labels = cellfun(@(x)sprintf('\n ID%i',x), trackIDs, 'UniformOutput', false);
plotTrack(findPlotter(BEP,'DisplayName','Tracks'), tracksPos, tracksVel, tracksCov,
labels);
Xpow = abs(Xbf).^2;
noisefloor = db2pow(-85);
Xsnr = pow2db(Xpow/noisefloor);
hRngDopPlot = rngImagHndls(1);
hRngDopPlot.XData = dopgrid*3600/1000; % m/s -> km/hr
hRngDopPlot.YData = rnggrid;
hRngDopPlot.CData = Xsnr;
[Nr,Ne,Nd] = size(Xrngdop);
Xrngdop = permute(Xrngdop,[1 3 2]); % Nr x Nd x Ne
Xrngdop = reshape(Xrngdop,[],Ne);
win = taylorwin(Ne,5,-60);
win = win/norm(win);
Xrngang = beamscan(Xrngdop.*win');
Xrngang = reshape(Xrngang,Nr,Nd,[]); % Nr x Nd x Ns
Xrngang = permute(Xrngang,[1 3 2]); % Nr x Ns x Nd
Xpow = abs(Xrngang).^2;
Xpow = max(Xpow,[],3);
Xsnr = pow2db(Xpow/noisefloor);
hRngAngPlot = rngImagHndls(2);
hRngAngPlot.XData = beamscan.Direction(1,:);
hRngAngPlot.YData = rnggrid;
hRngAngPlot.CData = Xsnr;
end
function metrics = runSimulation(c,fc,rangeMax,vMax,waveform,Nsweep,...
transmitter,radiator,collector,receiver,...
rngdopresp,beamformer,cfar,idxCFAR,...
rngestimator,dopestimator,doaest,beamscan,tracker,...
txchannel,rxchannel)
rng(2017);
reset(transmitter);
reset(radiator);
reset(collector);
reset(receiver);
reset(rngestimator);
reset(dopestimator);
reset(doaest);
reset(beamscan);
reset(tracker);
```

```
fs = waveform.SampleRate;
if nargin==20
if isprop(txchannel,'TwoWayPropagation') && txchannel.TwoWayPropagation
txchannel.TwoWayPropagation = false;
end
rxchannel = clone(txchannel);
elseif nargin<20
txchannel = phased.FreeSpace('PropagationSpeed',c,...
'OperatingFrequency',fc,'SampleRate',fs,'TwoWayPropagation',false);
rxchannel = clone(txchannel);
end
reset(txchannel);
reset(rxchannel);
[scenario,egoCar,radarParams,pointTgts] = helperAutoDrivingRadarSigProc('Setup
Scenario',c,fc);
Nft = waveform.SweepTime*waveform.SampleRate;
Ne = collector.Sensor.NumElements;
Nr = rngdopresp.RangeFFTLength;
Nd = rngdopresp.DopplerFFTLength;
rxArray = collector.Sensor;
% Initialize display for driving scenario example
helperAutoDrivingRadarSigProc('Initialize Display', egoCar, radarParams, rxArray,
fc, vMax, rangeMax);
tgtProfiles = actorProfiles(scenario);
tgtProfiles = tgtProfiles(2:end);
tgtHeight = [tgtProfiles.Height];
sweepTime = waveform.SweepTime;
while advance(scenario)
% Get the current scenario time
time = scenario.SimulationTime;
% Get current target poses in ego car's reference frame
tgtPoses = targetPoses(egoCar);
paths = phased.internal.twoRayPaths(radarParams.OriginPosition,...
reshape([tgtPoses.Position],3,[]),radarParams.OriginVelocity,...
reshape([tgtPoses.Velocity],3,[]),[1 1 1],c/fc);
% tgtPos = reshape([tgtPoses.Position],3,[]);
% tgtPos(3,:) = tgtPos(3,:)+0.5*tgtHeight; % Point targets are positioned at half of
each target's height
% tgtVel = reshape([tgtPoses.Velocity],3,[]);
% Simulate the radar transmit and receive operation
Xcube = zeros(Nft, Ne, Nsweep);
for m = 1:Nsweep
% % Calculate the target angles as seen by the sensor
% tgtAngs = NaN(2,numel(tgtPoses));
```

```
% for iTgt = 1:numel(tgtPoses)
% tgtAxes =
rotz(tgtPoses(iTgt).Yaw)*roty(tgtPoses(iTgt).Pitch)*rotx(tgtPoses(iTgt).Roll);
% [~,tgtAngs(:,iTgt)] =
rangeangle(radarParams.OriginPosition,tgtPos(:,iTgt),tgtAxes);
% end
%
% % Transmit FMCW waveform
% sig = waveform();
% [~, txang] = rangeangle(tgtPos, radarParams.OriginPosition,
radarParams.Orientation);
% txsig = transmitter(sig);
% txsig = radiator(txsig,txang);
%
% % Propagate the signal and reflect off the targets
% txsig =
txchannel(txsig,radarParams.OriginPosition,tgtPos,radarParams.OriginVelocity,tgtV
el);
% txsig = pointTgts(txsig,tgtAngs);
% txsig =
rxchannel(txsig,radarParams.OriginPosition,tgtPos,radarParams.OriginVelocity,tgtV
el);
%
% % Collect received target echos
% rxsig = collector(txsig, txang);
% rxsig = receiver(rxsig);
rxsig = radar(paths,time);
% Dechirp the received signal
rxsig = dechirp(rxsig, sig);
% Save sweep to data cube
Xcube(:,:,m) = rxsig;
% Move targets forward for next sweep
tgtPos = tgtPos+tgtVel*sweepTime;
end
% Calculate the range-doppler response
[Xrngdop,rnggrid,dopgrid] = rngdopresp(Xcube);
% Beamform received data
Xbf = permute(Xrngdop,[1 3 2]);
Xbf = reshape(Xbf,Nr*Nd,Ne);
Xbf = beamformer(Xbf);
Xbf = reshape(Xbf,Nr,Nd);
% Detect targets
Xpow = abs(Xbf).^2;
[detidx,noisepwr] = cfar(Xpow,idxCFAR);
```

```
% Cluster detections
clusterIDs = helperAutoDrivingRadarSigProc('Cluster Detections', detidx);
% Azimuth, range, and range rate measurements
[azest,azvar,snrdB] = helperAutoDrivingRadarSigProc('Estimate Angle', doaest,
conj(Xrngdop), Xbf, detidx, noisepwr, clusterIDs);
azvar = azvar+radarParams.RMSBias(1)^2;
[rngest,rngvar] = rngestimator(Xbf,rnggrid,detidx,noisepwr,clusterIDs);
rngvar = rngvar+radarParams.RMSBias(2)^2;
[rsest,rsvar] = dopestimator(Xbf,dopgrid,detidx,noisepwr,clusterIDs);
% Radial speed is estimated, but the tracker expects range rate.
% Convert radial speed to range rate.
rrest = -rsest;
rrvar = rsvar;
rrvar = rrvar+radarParams.RMSBias(3)^2;
% Assemble object detections for use by tracker
numDets = numel(rngest);
dets = cell(numDets,1);
for iDet = 1:numDets
dets{iDet} = objectDetection(time, [azest(iDet) rngest(iDet) rrest(iDet)]', ...
'MeasurementNoise', diag([azvar(iDet) rngvar(iDet) rrvar(iDet)]), ...
'MeasurementParameters', {radarParams}, ...
'ObjectAttributes', {struct('SNR',snrdB(iDet))});
end
% Track detections
tracks = tracker(dets,time);
% Update bird's-eye plot
helperAutoDrivingRadarSigProc('Update Display', egoCar, dets, tracks, dopgrid,
rnggrid,...
Xbf, beamscan, Xrngdop);
% Publish snapshot
% helperAutoDrivingRadarSigProc('Publish Snapshot', time>=1.1);
% Collect channel metrics
metrics = helperAutoDrivingRadarSigProc('Collect Metrics', radarParams, tgtPos,
tgtVel, dets);
end
end
function metrics = runSimulation2ray(c,fc,rangeMax,vMax,Nsweep,...
rngdopresp,beamformer,cfar,idxCFAR,clusterer,...
rngestimator,dopestimator,doaest,beamscan,tracker,radar,sig)
rng(2017);
reset(rngestimator);
reset(dopestimator);
reset(doaest);
reset(beamscan);
```

```
reset(clusterer);
reset(tracker);
release(radar);
[scenario,egoCar,radarParams] = helperAutoDrivingRadarSigProc('Setup
Scenario',c,fc);
Nft = radar.Waveform.SweepTime*radar.Waveform.SampleRate;
Ne = radar.ReceiveAntenna.Sensor.NumElements;
Nr = rngdopresp.RangeFFTLength;
Nd = rngdopresp.DopplerFFTLength;
rxArray = radar.ReceiveAntenna.Sensor;
% Initialize display for driving scenario example
helperAutoDrivingRadarSigProc('Initialize Display', egoCar, radarParams, rxArray,
fc, vMax, rangeMax);
tgtProfiles = actorProfiles(scenario);
tgtProfiles = tgtProfiles(2:end);
tgtHeight = [tgtProfiles.Height];
sweepTime = radar.Waveform.SweepTime;
while advance(scenario)
% Get the current scenario time
time = scenario.SimulationTime;
% Get current target poses in ego car's reference frame
tgtPoses = targetPoses(egoCar);
tgtPos = reshape([tgtPoses.Position],3,[]);
tgtPos(3,:) = tgtPos(3,:)+0.5*tgtHeight; % Point targets are positioned at half of
each target's height
tgtVel = reshape([tgtPoses.Velocity],3,[]);
% Simulate the radar transmit and receive operation
Xcube = zeros(Nft, Ne, Nsweep);
for m = 1:Nsweep
paths = phased.internal.twoRayPaths(radarParams.OriginPosition,...
tgtPos,radarParams.OriginVelocity,tgtVel,[10 10 10],c/fc);
rxsig = radar(paths,time+(m-1)*sweepTime);
% Dechirp the received signal
rxsig = dechirp(rxsig, sig);
% Save sweep to data cube
Xcube(:,:,m) = rxsig;
% Move targets forward for next sweep
tgtPos = tgtPos+tgtVel*sweepTime;
end
% Calculate the range-doppler response
[Xrngdop,rnggrid,dopgrid] = rngdopresp(Xcube);
% Beamform received data
Xbf = permute(Xrngdop,[1 3 2]);
Xbf = reshape(Xbf,Nr*Nd,Ne);
```

```
Xbf = beamformer(Xbf);
Xbf = reshape(Xbf,Nr,Nd);
% Detect targets
Xpow = abs(Xbf).^2;
[detidx,noisepwr] = cfar(Xpow,idxCFAR);
% Cluster detections
[~,clusterIDs] = clusterer(detidx.');
% Azimuth, range, and range rate measurements
[azest,azvar,snrdB] = helperAutoDrivingRadarSigProc('Estimate Angle', doaest,
conj(Xrngdop), Xbf, detidx, noisepwr, clusterIDs);
azvar = azvar+radarParams.RMSBias(1)^2;
[rngest,rngvar] = rngestimator(Xbf,rnggrid,detidx,noisepwr,clusterIDs);
rngvar = rngvar+radarParams.RMSBias(2)^2;
[rsest,rsvar] = dopestimator(Xbf,dopgrid,detidx,noisepwr,clusterIDs);
% Radial speed is estimated, but the tracker expects range rate.
% Convert radial speed to range rate.
rrest = -rsest;
rrvar = rsvar;
rrvar = rrvar+radarParams.RMSBias(3)^2;
% Assemble object detections for use by tracker
numDets = numel(rngest);
dets = cell(numDets,1);
for iDet = 1:numDets
dets{iDet} = objectDetection(time, [azest(iDet) rngest(iDet) rrest(iDet)]', ...
'MeasurementNoise', diag([azvar(iDet) rngvar(iDet) rrvar(iDet)]), ...
'MeasurementParameters', {radarParams}, ...
'ObjectAttributes', {struct('SNR',snrdB(iDet))});
end
% Track detections
tracks = tracker(dets,time);
% Update bird's-eye plot
helperAutoDrivingRadarSigProc('Update Display', egoCar, dets, tracks, dopgrid,
rnggrid,...
Xbf, beamscan, Xrngdop);
% Publish snapshot
helperAutoDrivingRadarSigProc('Publish Snapshot', time>=1.1&&time<1.15);
% Collect channel metrics
metrics = helperAutoDrivingRadarSigProc('Collect Metrics', radarParams, tgtPos,
tgtVel, dets);
end
end
function plotChannels(metricsFS, metrics2Ray)
hfig = findobj('Tag','Compare Propagation Channels');
if isempty(hfig)
```

```
hfig = figure('Name','Channel Propagation','Tag','Channel Propagation');
end
figure(hfig);
clf(hfig); hold on;
title('Propagation Channels');
xlabel('Measured Radar Range (m)');
ylabel('CFAR SNR Estimate (dB)');
grid on;
% Only plot metrics for long range car
[~,y] = pol2cart(metricsFS.AzimuthEst*pi/180,metricsFS.RangeEst);
iFndy = y<0;
% Only plot metrics for long range car
[~,z] = pol2cart(metrics2Ray.AzimuthEst*pi/180,metrics2Ray.RangeEst);
iFndz = z<0;
plot(metricsFS.RangeEst(iFndy),metricsFS.SNREst(iFndy),'bo',...
metrics2Ray.RangeEst(iFndz),metrics2Ray.SNREst(iFndz),'r^');
legend('Free space','Two-ray');
end
function metrics = collectMetrics(metrics, radarParams, tgtPos, tgtVel, dets)
% Collects metrics from the simulated driving scenario. Returns the current
% collected metrics.
if isempty(metrics)
metrics = struct( ...
'Position',[],...
'Velocity',[],...
'Azimuth',[],...
'Range',[],...
'RadialSpeed',[],...
'AzimuthEst',[],...
'AzimuthVar',[],...
'RangeEst',[],...
'RangeVar',[],...
'RadialSpeedEst',[],...
'RadialSpeedVar',[],...
'SNREst',[]);
end
% Compute ground truth metrics
gtspd = radialspeed(tgtPos,tgtVel);
[gtrng, gtang] = rangeangle(tgtPos, radarParams.OriginPosition,
radarParams.Orientation);
gtaz = gtang(1,:)';
% Measurements
numDets = numel(dets);
azest = NaN(numDets,1);
```

```
azvar = NaN(numDets,1);
rngest = NaN(numDets,1);
rngvar = NaN(numDets,1);
rsest = NaN(numDets,1);
rsvar = NaN(numDets,1);
snrdB = NaN(numDets,1);
for m = 1:numDets
azest(m) = dets{m}.Measurement(1);
azvar(m) = dets{m}.MeasurementNoise(1,1);
rngest(m) = dets{m}.Measurement(2);
rngvar(m) = dets{m}.MeasurementNoise(2,2);
rsest(m) = dets{m}.Measurement(3);
rsvar(m) = dets{m}.MeasurementNoise(3,3);
snrdB(m) = dets{m}.ObjectAttributes{1}.SNR;
end
metrics.Position = [metrics.Position; tgtPos];
metrics.Velocity = [metrics.Position; tgtVel];
metrics.Azimuth = [metrics.Azimuth; gtaz(:)];
metrics.Range = [metrics.Range; gtrng(:)];
metrics.RadialSpeed = [metrics.RadialSpeed; gtspd(:)];
metrics.AzimuthEst = [metrics.AzimuthEst; azest(:)];
metrics.AzimuthVar = [metrics.AzimuthVar; azvar(:)];
metrics.RangeEst = [metrics.RangeEst; rngest(:)];
metrics.RangeVar = [metrics.RangeVar; rngvar(:)];
metrics.RadialSpeedEst = [metrics.RadialSpeedEst; rsest(:)];
metrics.RadialSpeedVar = [metrics.RadialSpeedVar; rsvar(:)];
metrics.SNREst = [metrics.SNREst; snrdB(:)];
end
function publishSnapshot(fig, takeSnapshot)
% When publishing, function takes one snapshot of the figure fig when
% takeSnapshot is true and then closes the figure so that another snapshot
% will not be generated from it. Does nothing when not publishing.
if ishghandle(fig)
hasSnapshot = takeSnapshot && isPublishing();
if hasSnapshot
figure(fig);
fig.Visible = 'on';
snapnow;
close(fig);
end
end
end
% Helper functions
function egoCar = addCars(scenario,roadcenters,initialdist,lane,initialspd)
```

```
% Adds all of the cars defined for the driving scenario and returns the
% vehicle object corresponding to the ego vehicle.
waypoints = interpWaypointsFromRoadCenters(roadcenters);
egoCar = addCar(scenario,waypoints,initialdist(1),lane(1),initialspd(1));
numCars = numel(initialspd);
for m = 2:numCars
addCar(scenario,waypoints,initialdist(m),lane(m),initialspd(m));
end
end
function car = addCar(scenario,waypoints,initialdist,lane,initialspd)
% Adds a car and its trajectory to the driving scenario. Returns the added
% vehicle object.
pathlength = [0;cumsum(sqrt(sum(diff(waypoints,1,1).^2,2)))];
car = vehicle(scenario);
ind = pathlength>=initialdist;
waypoints = waypoints(ind,:);
vecs = diff(waypoints,1,1);
nm = sqrt(sum(vecs.^2,2));
vecs = vecs./nm;
vecs = vecs([1 1:end],:);
vecs = [-vecs(:,2) vecs(:,1)];
waypoints = waypoints+vecs*lane;
trajectory(car, waypoints, initialspd);
end
function tgts = createPointTargets(scenario,c,fc)
% Returns point targets modeled using the |phased.BackscatterRadarTarget|
% object. The point target models are loaded from the point target models
% stored in the driving scenario's actor profiles.
profiles = actorProfiles(scenario);
profiles = profiles(2:end); % First profile is the ego car, not a target
numTgts = numel(profiles);
azAngles = profiles(2).RCSAzimuthAngles;
elAngles = profiles(2).RCSElevationAngles;
rcsPatterns = NaN(numel(elAngles),numel(azAngles),numTgts);
for m = 1:numTgts
rcsPatterns(:,:,m) = db2mag(profiles(m).RCSPattern);
end
tgts = phased.BackscatterRadarTarget( ...
'RCSPattern', rcsPatterns, ...
'AzimuthAngles', azAngles, ...
'ElevationAngles', elAngles, ...
'PropagationSpeed', c,...
'OperatingFrequency', fc);
end
```

```
function tgtstruct = createTargetStruct(tgtpos,tgtvel)
ntgt = size(tgtpos,2);
tgtstruct = struct('Position',mat2cell(tgtpos(:).',1,repmat(3,1,ntgt)),...
'Velocity',mat2cell(tgtvel(:).',1,repmat(3,1,ntgt)),...
'Signature',{rcsSignature,rcsSignature,rcsSignature});
end
function waypoints = interpWaypointsFromRoadCenters(roadCenters)
% Returns waypoints corresponding to the interpolated road centers, where
% the interpolation between the road centers is performed by
% drivingScenario.
num = size(roadCenters,2);
s = drivingScenario;
road(s, roadCenters, 1e-6);
rb = roadBoundaries(s);
rb = rb{1};
waypoints = [];
while ~isempty(rb)
pt = rb(1,:);
d = sqrt(sum((rb-pt).^2,2));
ind = d<1e-4;
waypoints = [waypoints;mean(rb(ind,:),1)]; %#ok<AGROW>
rb(ind,:) = [];
end
waypoints = waypoints(:,1:num);
end
function clusterIDs = clusterDetections(detidx)
% clusterIDs = clusterDetections(dets) clusters detections from a
% range-Doppler image. Detections which occur at adjacent range and Doppler
% cells within the image are associated to a single detection cluster. Each
% detection cluster is assigned a unique cluster ID which is used to
% identify the detections assigned to that cluster.
%
% detidx is a 2-by-L matrix of detection indices. Each column of detidx
% identifies the range and Doppler cell in the range-Doppler image where a
% detection was found as [rngidx; dopidx].
%
% clusterIDs is a 1-by-L vector of cluster IDs assigned to each detection
% index in detidx.
% Number of detections found in the range-Doppler image
numDet = size(detidx,2);
clusterIDs = NaN(1,numDet);
iGd = ~isnan(detidx(1,:));
if any(iGd(:))
% Cluster adjacent points
```

```
iRng = detidx(1,iGd);
iDop = detidx(2,iGd);
iGroups = cell(1,0);
allCells = [iRng;iDop];
while ~all(isnan(allCells(1,:)))
% Select an unassigned range-Doppler cell and remove it from the set
iFnd = find(~isnan(allCells(1,:)),1);
thisCell = allCells(:,iFnd);
allCells(:,iFnd) = NaN;
% Find all cells adjacent to the selected cell and remove them from set
[iAdjCells,allCells] = findAdjacentCells(allCells,thisCell);
iGroups{end+1} = [iFnd iAdjCells]; %#ok<AGROW>
end
% Assign unique cluster IDs to each group of adjacent range-Doppler
% cells
ids = NaN(sum(iGd),1);
for m = 1:numel(iGroups)
ids(iGroups{m}) = m;
end
clusterIDs(iGd) = ids';
end
end
function [iAdjCells,allCells] = findAdjacentCells(allCells,thisCell)
% [iAdjCells,allCells] = findAdjacentCells(allCells,thisCell) finds the
% cells in the 2-by-L matrix of allCells that are adjacent to the 2-by-1
% vector for the current cell, thisCell. Both allCells and thisCell contain
% indices of the rows and columns of a matrix.
%
% iAdjCells is a 2-by-M matrix of cells in allCells that are adjacent to
% thisCell. M is less-than or equal-to L.
%
% allCells is returned as a 2-by-L matrix, where the adjacent cells
% returned in iAdjCells have been set to NaN.
% Find all cells adjacent to the current cell and remove it from the set
delta = allCells-repmat(thisCell,[1 size(allCells,2)]);
iAdjCells = find(all(abs(delta)<=2,1));
thisCell = allCells(:,iAdjCells);
allCells(:,iAdjCells) = NaN;
% Find call cells next to each of the adjacent cells that were just found
for m = 1:numel(iAdjCells)
[iAdjNext,allCells] = findAdjacentCells(allCells,thisCell(:,m));
iAdjCells = [iAdjCells iAdjNext]; %#ok<AGROW>
end
end
```

```
function [angest, angvar, snrdB] =
doaestimator(doa,Xrngdop,Xbf,detidx,noisepwr,clusterIDs)
% [angest, angvar, snrdB] =
doaestimator(doa,Xrngdop,Xbf,detidx,noisepwr,clusterIDs)
% returns the estimated angles for the detections generated from a linear
% phased array.
%
% doa is the phased.RootMUSICEstimator object used to estimate the
% direction-of-arrival of signals detected on a linear phased array.
%
% Xrngdop is a Nr-by-Ne-by-Nd date cube containing the range-Doppler
% processed data received by the Ne elements of the linear phased array.
%
% Xbf is the Nr-by-Nd beamformed range-Doppler image used to identify
% detections in the Xrngdop data cube.
%
% detidx is a 2-by-L matrix of detection indices generated from the
% beamformed range-Doppler image, Xbf. Each column of detidx identifies the
% range and Doppler cell in the range-Doppler image where a detection was
% found as [rngidx; dopidx].
%
% noisepwr is a 1-by-L vector of the estimated noise powers at each
% detection location reported in detidx.
%
% clusterIDs is a 1-by-L vector of cluster IDs assigned to each detection
% index in detidx.
%
% angest is an N-by-1 vector of angle estimates generated from the
% detection indices and clusterIDs. If clusterIDs is not provided, each
% detection is assigned its own angle estimate. Angle estimates are
% reported in degrees.
%
% angvar is an N-by-1 vector of variances corresponding to each of the
% angle estimates reported in angest. The angle variances are reported in
% degrees-squared (deg^2).
%
% snrdB is an N-by-1 vector of the signal-to-noise ratios (SNR) estimated
% of each detection in detidx. SNR is reported in decibels (dB).
persistent bw
if isempty(bw)
bw = arraybeamwidth(doa);
end
% If not provided, assign each detection to its own cluster
numDet = size(detidx,2);
```

```
if nargin<4
clusterIDs = (1:numDet);
clusters = clusterIDs;
numEst = numDet;
else
clusters = unique(clusterIDs(~isnan(clusterIDs)));
numEst = length(clusters);
end
angest = NaN(numEst,1);
angvar = NaN(numEst,1);
snrdB = NaN(numEst,1);
Nr = size(Xrngdop,1);
hasDoppler = ndims(Xrngdop)==3;
if hasDoppler
Xrngdop = permute(Xrngdop,[1 3 2]); % Nr x Nd x Ne
Nd = size(Xrngdop,2);
Nelmnt = size(Xrngdop,3);
else
Nd = 1;
Nelmnt = size(Xrngdop,2);
end
Xrngdop = reshape(Xrngdop,[],Nelmnt);
for m = 1:numEst
% Select detections in this cluster
thisCluster = clusterIDs==clusters(m);
iRng = detidx(1,thisCluster);
iDop = detidx(2,thisCluster);
idx = sub2ind([Nr Nd],iRng,iDop);
y = Xrngdop(idx,:);
angest(m) = doa(y);
[pow,iMax] = max(abs(Xbf(idx)).^2);
noise = noisepwr(thisCluster);
snr = pow/noise(iMax);
sigma = bw/(1.6*sqrt(2*snr));
angvar(m) = sigma.^2;
snrdB(m) = pow2db(snr);
end
end
function bw = arraybeamwidth(doa,c,fc)
% bw = arraybeamwidth(doa) returns the half-power (3dB) beamwidth estimated
% from the linear phased array used by doa.
%
% doa is the phased.RootMUSICEstimator object used to estimate the
% direction-of-arrival of signals detected on a linear phased array.
```

```
%
% bw is the estimated half power beamwidth in degrees.
if isprop(doa,'SensorArray')
array = doa.SensorArray;
fc = doa.OperatingFrequency;
c = doa.PropagationSpeed;
else
array = doa;
end
ang = linspace(-90,90);
pat =
patternAzimuth(array,fc,0,'Type','powerdb','PropagationSpeed',c,'Azimuth',ang);
pk = max(pat);
iPk1 = find(pat>=pk-3,1,'first');
iPk2 = find(pat>=pk-3,1,'last');
bw = abs(ang(iPk2)-ang(iPk1));
end
function tf = isPublishing
% Returns true if the example is currently publishing
tf = false;
try
s = numel(dbstack);
tf = s > 3;
catch
end
end
```

```
function fig = createFMCWRadarGUI()
% Create the figure window
fig = uifigure('Name', 'FMCW Radar Parameters', 'Position', [100 100 700 700]);
% Store GUI handles in figure's UserData
fig.UserData = struct('fc', [], 'bw', [], 'rangeMax', [], 'vMax', [], ...
'fs', [], 'Nsweep', [], 'Ntx', [], 'Nrx', [], ...
'antAperture', [], 'rxNF', [], 'angscan', [], ...
'lookDirection', [], 'Ne', [], 'lambda', [], ...
'tm', [], 'rangeRes', [], 'sweepSlope', [], ...
'fbeatMax', [], 'fdopMax', [], 'fifMax', []);
% Input Fields
fcLabel = uilabel(fig, 'Position', [50 600 150 30], 'Text', 'Center Frequency (Hz):');
fcField = uieditfield(fig, 'numeric', 'Position', [250 600 100 30], 'Value', 77e9);
bwLabel = uilabel(fig, 'Position', [50 550 150 30], 'Text', 'Bandwidth (Hz):');
bwField = uieditfield(fig, 'numeric', 'Position', [250 550 100 30], 'Value', 2e9);
rangeMaxLabel = uilabel(fig, 'Position', [50 500 150 30], 'Text', 'Max Range (m):');
```

```
rangeMaxField = uieditfield(fig, 'numeric', 'Position', [250 500 100 30], 'Value',
736.36);
vMaxLabel = uilabel(fig, 'Position', [50 450 150 30], 'Text', 'Max Velocity (m/s):');
vMaxField = uieditfield(fig, 'numeric', 'Position', [250 450 100 30], 'Value',
230*1000/3600);
fsLabel = uilabel(fig, 'Position', [50 400 150 30], 'Text', 'Sampling Frequency
(Hz):');
fsField = uieditfield(fig, 'numeric', 'Position', [250 400 100 30], 'Value', 50e6);
NsweepLabel = uilabel(fig, 'Position', [50 350 150 30], 'Text', 'Number of Chirps:');
NsweepField = uieditfield(fig, 'numeric', 'Position', [250 350 100 30], 'Value', 192);
NtxLabel = uilabel(fig, 'Position', [50 300 150 30], 'Text', 'Number of Tx
Antennas:');
NtxField = uieditfield(fig, 'numeric', 'Position', [250 300 100 30], 'Value', 3);
NrxLabel = uilabel(fig, 'Position', [50 250 150 30], 'Text', 'Number of Rx
Antennas:');
NrxField = uieditfield(fig, 'numeric', 'Position', [250 250 100 30], 'Value', 4);
antApertureLabel = uilabel(fig, 'Position', [50 200 150 30], 'Text', 'Antenna Aperture
(m^2):');
antApertureField = uieditfield(fig, 'numeric', 'Position', [250 200 100 30], 'Value',
19e-6);
rxNFLabel = uilabel(fig, 'Position', [50 150 150 30], 'Text', 'Receiver Noise Figure
(dB):');
rxNFField = uieditfield(fig, 'numeric', 'Position', [250 150 100 30], 'Value', 12);
angscanLabel = uilabel(fig, 'Position', [50 100 150 30], 'Text', 'Field of View
(degrees):');
angscanField = uieditfield(fig, 'text', 'Position', [250 100 100 30], 'Value', '-60:60');
lookDirectionLabel = uilabel(fig, 'Position', [50 50 150 30], 'Text', 'Look Direction
(degrees):');
lookDirectionField = uieditfield(fig, 'numeric', 'Position', [250 50 100 30], 'Value',
0);
% Button to calculate outputs
calcButton = uibutton(fig, 'Position', [250 10 100 30], 'Text', 'Calculate', ...
'ButtonPushedFcn', @(calcButton,event) calculateOutputs(fcField, bwField,
rangeMaxField, vMaxField, fsField, NsweepField, NtxField, NrxField,
antApertureField, rxNFField, angscanField, lookDirectionField));
% Output Labels
NeLabel = uilabel(fig, 'Position', [400 600 150 30], 'Text', 'Number of Virtual
Antennas:');
NeOutput = uilabel(fig, 'Position', [550 600 100 30], 'Text', '');
lambdaLabel = uilabel(fig, 'Position', [400 550 150 30], 'Text', 'Wavelength (m):');
lambdaOutput = uilabel(fig, 'Position', [550 550 100 30], 'Text', '');
tmLabel = uilabel(fig, 'Position', [400 500 150 30], 'Text', 'Chirp Duration (s):');
tmOutput = uilabel(fig, 'Position', [550 500 100 30], 'Text', '');
```

```
rangeResLabel = uilabel(fig, 'Position', [400 450 150 30], 'Text', 'Range Resolution (m):');
rangeResOutput = uilabel(fig, 'Position', [550 450 100 30], 'Text', '');
slopeLabel = uilabel(fig, 'Position', [400 400 150 30], 'Text', 'Sweep Slope (Hz/s):');
slopeOutput = uilabel(fig, 'Position', [550 400 100 30], 'Text', '');
fbeatMaxLabel = uilabel(fig, 'Position', [400 350 150 30], 'Text', 'Max Beat Freq (Hz):');
fbeatMaxOutput = uilabel(fig, 'Position', [550 350 100 30], 'Text', '');
fdopMaxLabel = uilabel(fig, 'Position', [400 300 150 30], 'Text', 'Max Doppler Shift (Hz):');
fdopMaxOutput = uilabel(fig, 'Position', [550 300 100 30], 'Text', '');
fifMaxLabel = uilabel(fig, 'Position', [400 250 150 30], 'Text', 'Max IF Freq (Hz):');
fifMaxOutput = uilabel(fig, 'Position', [550 250 100 30], 'Text', '');
antgainLabel = uilabel(fig, 'Position', [400 200 150 30], 'Text', 'Antenna Gain (dB):');
antgainOutput = uilabel(fig, 'Position', [550 200 100 30], 'Text', '');
% Calculation Function
function calculateOutputs(fcField, bwField, rangeMaxField, vMaxField, fsField, NsweepField, NtxField, NrxField, antApertureField, rxNFField, angscanField, lookDirectionField)
fc = fcField.Value;
bw = bwField.Value;
rangeMax = rangeMaxField.Value;
vMax = vMaxField.Value;
fs = fsField.Value;
Nsweep = NsweepField.Value;
Ntx = NtxField.Value;
Nrx = NrxField.Value;
antAperture = antApertureField.Value;
rxNF = rxNFField.Value;
angscan = str2num(angscanField.Value); % Convert the string input to an array
lookDirection = lookDirectionField.Value;
% Constants
c = physconst('LightSpeed');
% Calculations
Ne = Ntx * Nrx;
lambda = c / fc;
tm = 5.5 * (2 * rangeMax) / c;
tm = round(tm * 1e6) / 1e6;
rangeRes = c / (2 * bw);
sweepSlope = bw / tm;
fbeatMax = (2 * rangeMax * bw) / (c * tm);
fdopMax = (2 * vMax * fc) / c;
fifMax = fbeatMax + fdopMax;
```

```
antGain = aperture2gain(antAperture,lambda);
% Store calculated values in the figure's UserData
fig.UserData = struct('fc', fc, 'bw', bw, 'rangeMax', rangeMax, 'vMax', vMax, ...
'fs', fs, 'Nsweep', Nsweep, 'Ntx', Ntx, 'Nrx', Nrx, ...
'antAperture', antAperture, 'rxNF', rxNF, 'angscan', angscan, ...
'lookDirection', lookDirection, 'Ne', Ne, 'lambda', lambda, ...
'tm', tm, 'rangeRes', rangeRes, 'sweepSlope', sweepSlope, ...
'fbeatMax', fbeatMax, 'fdopMax', fdopMax, 'fifMax', fifMax);
% Update Outputs
NeOutput.Text = num2str(Ne);
lambdaOutput.Text = sprintf('%.3e', lambda);
tmOutput.Text = sprintf('%.3e', tm);
rangeResOutput.Text = sprintf('%.3e', rangeRes);
slopeOutput.Text = sprintf('%.3e', sweepSlope);
fbeatMaxOutput.Text = sprintf('%.3e', fbeatMax);
fdopMaxOutput.Text = sprintf('%.3e', fdopMax);
fifMaxOutput.Text = sprintf('%.3e', fifMax);
antgainOutput.Text = sprintf('%.3f', antGain);
end
end
```

```
function fig = createNoiseGUI()
% Create the figure window
fig = uifigure('Name', 'Noise Specifications', 'Position', [100 100 650 500]);
% Store GUI handles in figure's UserData
fig.UserData = struct('freespace', [], 'rangeMax', [], 'lambda', [], ...
'gasloss', [], 'temperature', [], 'pressure', [], ...
'density', [], 'fogloss', [], 'rainloss', [], ...
'rainrate', [], 'totallossdB', [], 'totallossW', [] );
% Input Fields
fcLabel = uilabel(fig, 'Position', [50 400 150 30], 'Text', 'Center Frequency (Hz):');
fcField = uieditfield(fig, 'numeric', 'Position', [250 400 100 30], 'Value', 77e9);
rangeMaxLabel = uilabel(fig, 'Position', [50 350 150 30], 'Text', 'Max Range (m):');
rangeMaxField = uieditfield(fig, 'numeric', 'Position', [250 350 100 30], 'Value',
736.36);
tempLabel = uilabel(fig, 'Position', [50 300 150 30], 'Text', 'Temperature (C)');
tempField = uieditfield(fig, 'numeric', 'Position', [250 300 100 30], 'Value', 32);
pressureLabel = uilabel(fig, 'Position', [50 250 150 30], 'Text', 'Pressure (Pa)');
pressureField = uieditfield(fig, 'numeric', 'Position', [250 250 100 30], 'Value',
101325);
densityLabel = uilabel(fig, 'Position', [50 200 150 30], 'Text', 'Density (g/m^3)');
densityField = uieditfield(fig, 'numeric', 'Position', [250 200 100 30], 'Value', 1);
rainrateLabel = uilabel(fig, 'Position', [50 150 150 30], 'Text', 'Rain Rate (mm/hr)');
```

```
rainrateField = uieditfield(fig, 'numeric', 'Position', [250 150 100 30], 'Value', 0);
% Button to calculate outputs
calcButton = uibutton(fig, 'Position', [250 100 100 30], 'Text', 'Calculate', ...
'ButtonPushedFcn', @(calcButton,event) calculateOutputs(fcField, rangeMaxField,
tempField, pressureField, densityField, rainrateField));
% Output Labels
FSPLLabel = uilabel(fig, 'Position', [400 400 150 30], 'Text', 'Free Space Loss
(dB)');
FSPLOutput = uilabel(fig, 'Position', [550 400 100 30], 'Text', '');
GASPLLabel = uilabel(fig, 'Position', [400 350 150 30], 'Text', 'Gas Loss (dB)');
GASPLOutput = uilabel(fig, 'Position', [550 350 100 30], 'Text', '');
FOGPLLabel = uilabel(fig, 'Position', [400 300 150 30], 'Text', 'Fog Loss (dB)');
FOGPLOutput = uilabel(fig, 'Position', [550 300 100 30], 'Text', '');
RAINPLLabel = uilabel(fig, 'Position', [400 250 150 30], 'Text', 'Rain Loss (dB)');
RAINPLOutput = uilabel(fig, 'Position', [550 250 100 30], 'Text', '');
TotalLossdBLabel = uilabel(fig, 'Position', [400 200 150 30], 'Text', 'Total losses
(dB)');
TotalLossdBOutput = uilabel(fig, 'Position', [550 200 100 30], 'Text', '');
TotalLossWLabel = uilabel(fig, 'Position', [400 150 150 30], 'Text', 'Total losses
(W)');
TotalLossWOutput = uilabel(fig, 'Position', [550 150 100 30], 'Text', '');
% Calculation Function
function calculateOutputs(fcField, rangeMaxField, tempField, pressureField,
densityField, rainrateField)
fc = fcField.Value;
rangeMax = rangeMaxField.Value;
temp = tempField.Value;
pressure = pressureField.Value;
density = densityField.Value;
rainrate = rainrateField.Value;
% Constants
c = physconst('LightSpeed');
% Calculations
lambda = c / fc;
FSPL = fspl(rangeMax, lambda) / 10;
GASPL = gaspl(rangeMax, fc, temp, pressure, density);
FOGPL = fogpl(rangeMax, fc, temp, density);
RAINPL = rainpl(rangeMax, fc, rainrate);
TotalLossdB = FSPL + GASPL + FOGPL + RAINPL;
TotalLossW = 10^(TotalLossdB/10);
% Store calculated values in the figure's UserData
fig.UserData = struct('freespace', FSPL, 'rangeMax', rangeMax, 'lambda', lambda, ...
'gasloss', GASPL, 'temperature', temp, 'pressure', pressure, ...
'density', density, 'fogloss', FOGPL, 'rainloss', RAINPL, ...
```

```
'rainrate', rainrate, 'totallossdB', TotalLossdB, 'totallossW', TotalLossW);
% Update Outputs
FSPLOutput.Text = sprintf('%.3f', FSPL);
lambdaOutput.Text = sprintf('%.3e', lambda);
GASPLOutput.Text = sprintf('%.3f', GASPL);
FOGPLOutput.Text = sprintf('%.3f', FOGPL);
RAINPLOutput.Text = sprintf('%.3f', RAINPL);
TotalLossdBOutput.Text = sprintf('%.3f', TotalLossdB);
TotalLossWOutput.Text = sprintf('%.3f', TotalLossW);
end
end
```

```
function createPauseButton()
% Create a figure for the button
fig = uifigure('Name', 'Radar Simulation Pause', 'Position', [100, 100, 300, 150]);
% Add the "Continue" button
btn = uibutton(fig, 'Text', 'Continue', ...
'Position', [100, 50, 100, 40], ...
'ButtonPushedFcn', @(btn, event) uiresume(fig)); % Resume when button is pressed
% Pause until button is pressed
uiwait(fig);
% Close the figure after resuming
close(fig);
end
```

```
%% Range - Velocity Doppler
function [Xrngdop rnggrid dopgrid] = customRangeDopplerResponse(Xcube, Nft,
Nst, Nr, Nd, Ne)
% Xcube: Data cube received from the receiver array
% Nft: Number of fast time samples
% Nst: Number of slow time samples
% Nr: Number of range bins
% Nd: Number of doppler bins
% c: Speed of Light
% fc: Operating Frequency
% fs: Sampling Frequency
%% Evaluating Xrngdop
for n = 1:Ne
% Constructing and applying range window
rangeWindow = hanning(Nft,1);
Xcube_1= Xcube(:,n,:) .* rangeWindow;
% Applying range fft
Xcube_1 = fft(Xcube_1,Nr,1);
```

```
% Constructing and applying doppler window
dopplerWindow = hanning(Nst,3);
Xcube_2= Xcube_1 .* dopplerWindow;
% Applying doppler fft
Xrngdop(:,n,:) = fft(Xcube_2,Nd,3);
Xrngdop = fftshift(Xrngdop); % Shift the zero-frequency component to the center
end
%% Developing range grid
rangeStepSize = Nft/Nr;
rangeStart = -(Nft/2);
for a = 1:Nr
rnggrid(a,1) = rangeStart + ( (a-1) * rangeStepSize);
end
%% Developing doppler grid
dopplerStepSize = (Nst * 2 * 1.52) / Nd;
dopplerStart = Nst * 1.52;
for b = 1:Nd
dopgrid(b,1) = dopplerStart - ( (b-1) * dopplerStepSize);
end
end
% Function for Hanning window
function [w] = hanning(N,dim)
if (dim == 1)
for n = 1:N
w(n,1,1) = 0.5*(1-cos((2*pi*n)/N));
end
elseif (dim == 2)
for n = 1:N
w(1,n,1) = 0.5*(1-cos((2*pi*n)/N));
end
elseif (dim == 3)
for n = 1:N
w(1,1,n) = 0.5*(1-cos((2*pi*n)/N));
end
end
end
```

```
function [Xbf] = customBeamforming(Xrngdop, Nr, Ne, Nd, fc, lookDirection)
% Xrngdop is the data cube obtained after performing range and doppler processing.
% Ne is the number of antenna elements.
% fc is the carrier frequency.
% lookDirection is the direction or angle of beam interest
%% Calculating and extracting nexessary values
```

```
c = 3e8;
lambda = c/fc;
elementSpacing = lambda/2;
%% Reshaping the input for beamforming
% Initialize the 2D output matrix
Xbf_ = zeros(Nr*Nd, Ne);
% Loop over antenna elements
for element = 1:Ne
% Loop over range bins
for rangeIdx = 1:Nr
% Loop over Doppler bins
for dopplerIdx = 1:Nd
% Calculate the linear index for the 2D matrix
linearIdx = (rangeIdx - 1) * Nd + dopplerIdx;
% Assign the value to the correct position in the 2D matrix
Xbf_(linearIdx, element) = Xrngdop(rangeIdx, element, dopplerIdx);
end
end
end
%% Calculating steering vectors
steeringVector = exp(1j * 2 * pi * elementSpacing / lambda * sind(lookDirection) *
(0:Ne-1)' );
%% Apply beamforming
Xbf_ = Xbf_ * steeringVector;
%% Reshape vector into 2D array
% Temporary matrix for reshaping
Xbf_2D = zeros(Nr, Nd);
% Reshape the 1D result to a 2D matrix
for rangeIdx = 1:Nr
for dopplerIdx = 1:Nd
% Calculate the linear index for the 1D vector
linearIdx = (rangeIdx - 1) * Nd + dopplerIdx;
% Assign the value to the correct position in the 2D matrix
Xbf_2D(rangeIdx, dopplerIdx) = Xbf_(linearIdx);
end
end
% Output the final 2D matrix
Xbf = Xbf_2D;
end
```

```
function [Xpow] = customSquareLaw(Xbf)
% Xbf is the beamformed and FFT processed input data cube
Xpow = abs(Xbf).^2;
```

```
end
```

```
function [detidx, noisepwr, numDetections] = customCFARDetector(Xpow, nGuard,
nTrain, Nd, Nsweep, Nr, rnggrid, customFactor)
% Xpow: Matrix conaining power level from the FFT processing
% nGuard: Number of guard layers
% nTrain: Number of training layers
% Nd: Number of doppler bins
% Nsweep: Slow time axis
% Nr: Number of range bins
% rnggrid: Range grid axis
% customFactor: Custom threshold factor
%% Making the observation grid
% Defining guard and training cell ranges
nGuardRng = nGuard;
nTrainRng = nTrain;
nCUTRng = 1 + nGuardRng + nTrainRng;
dopOver = round(Nd/Nsweep);
nGuardDop = nGuard * dopOver;
nTrainDop = nTrain * dopOver;
nCUTDop = 1 + nGuardDop + nTrainDop;
% Forming the grid
indexingNumber = 0;
for a = 1:Nr
if (rnggrid(a) > 0)
indexingNumber = indexingNumber + 1;
iRngCUT(indexingNumber,1) = a;
end
end
iRngCUT = iRngCUT((iRngCUT>=nCUTRng)&(iRngCUT<=Nr-nCUTRng+1));
iDopCUT = nCUTDop:(Nd-nCUTDop+1);
% Fill arrays with grid values
nRng = length(iRngCUT);
nDop = length(iDopCUT);
for r = 1:nRng
for d = 1:nDop
iRng(d, r) = iRngCUT(r);
iDop(d, r) = iDopCUT(d);
end
end
% Flatten and combine
iRng_flat = iRng(:);
iDop_flat = iDop(:);
```

```
idxCFAR = [iRng_flat, iDop_flat]';
%% Computing threshold
% Identifying vertices of CUT
CUTRngmin = customMin(iRngCUT, nRng);
CUTRngmax = customMax(iRngCUT, nRng);
CUTDopmin = customMin(iDopCUT, nDop);
CUTDopmax = customMax(iDopCUT, nDop);
% Gathering noise from Training rectangle 1
noisepwrSumR1 = 0;
countR1 = 0;
for m = CUTRngmin : (CUTRngmax + nGuardRng)
for n = 1 : ( CUTDopmin - nGuardDop - 1)
noisepwrSumR1 = noisepwrSumR1 + Xpow(m,n);
countR1 = countR1 + 1;
end
end
% Gathering noise from Training rectangle 2
noisepwrSumR2 = 0;
countR2 = 0;
for m = CUTRngmin : (CUTRngmax + nGuardRng)
for n = ( CUTDopmax + nGuardDop + 1) : Nd
noisepwrSumR2 = noisepwrSumR2 + Xpow(m,n);
countR2 = countR2 + 1;
end
end
% Gathering noise from Training rectangle 3
noisepwrSumR3 = 0;
countR3 = 0;
for m = (CUTRngmax + nGuardRng + 1) : Nr
for n = 1 : Nd
noisepwrSumR3 = noisepwrSumR3 + Xpow(m,n);
countR3 = countR3 + 1;
end
end
% Implementing CFAR algorithm
meanNoiseLevel = (noisepwrSumR1 + noisepwrSumR2 + noisepwrSumR3) /
(countR1 + countR2 + countR3);
% Calculating custom threshold parameter
customThresholdFactor = 10 ^ (customFactor/10);
% Calcualting threshold
threshold = meanNoiseLevel * customThresholdFactor;
%% Scanning grid for detections
indexingNumber = 0;
for m = CUTRngmin : CUTRngmax
```

```
for n = CUTDopmin : CUTDopmax
if( Xpow(m,n) > threshold)
indexingNumber = indexingNumber + 1;
detidx(1,indexingNumber) = m;
detidx(2,indexingNumber) = n;
noisepwr(indexingNumber) = Xpow(m,n);
end
end
end
numDetections = indexingNumber;
end
```

```
function [maximumValue] = customMax(inputArray, sizeOfArray)
maximumValue = inputArray(1);
for a = 1:sizeOfArray
if (inputArray(a) > maximumValue)
maximumValue = inputArray(a);
end
end
end
```

```
function [minimumValue] = customMin(inputArray, sizeOfArray)
minimumValue = inputArray(1);
for a = 1:sizeOfArray
if (inputArray(a) < minimumValue)
minimumValue = inputArray(a);
end
end
end
```

```
function [clusterIDs, maxClusters] = customCluster(detidx, epsilonDistance,
numDetections)
clusterassignments = zeros(1, numDetections);
current_cluster = 0;
for i = 1:numDetections
if(clusterassignments(i) == 0)
current_cluster = current_cluster + 1;
clusterassignments(i) = current_cluster;
end
for j = (i+1):numDetections
current_difference_rng = (detidx(1,j)-detidx(1,i));
current_difference_dop = (detidx(2,j)-detidx(2,i));
```

```
distance = ( (current_difference_rng)^2 + (current_difference_dop)^2) ^ (0.5);
if(distance < epsilonDistance)
clusterassignments(j) = current_cluster;
end
end
end
clusterIDs = clusterassignments;
maxClusters = current_cluster;
end
```

```
function [rngest rngvar] = customRangeEstimator(Xbf, detidx, rnggrid,
numDetections, clusterIDs, maxClusters)
% Xbf: Beamformed signal array
% detidx: Range and doppler bin numbers of the detections
% rnggrid: Range axis of the range-doppler map
% numDetections: Total number of detections observed
% clusterIDs: The cluster number of each detection
% maxClusters: Total number of clusters formed
%% Initializations
sumproductXbfrnggrid = zeros(maxClusters,1);
sumproductXbfRngGridDeviationSquared= zeros(maxClusters,1);
rngvar = zeros(maxClusters,1);
sumXbf = zeros(maxClusters,1);
rngest = zeros(maxClusters,1);
%% Estimation of range
for i = 1:numDetections
currentRngIdx = detidx(1,i);
currentDopIdx = detidx(2,i);
currentClusterID = clusterIDs(i);
XbfMag = abs(Xbf(currentRngIdx, currentDopIdx));
sumproductXbfrnggrid(currentClusterID,1) =
sumproductXbfrnggrid(currentClusterID,1) + (XbfMag * rnggrid(currentRngIdx));
sumXbf(currentClusterID,1) = sumXbf(currentClusterID,1) + XbfMag;
end
for j = 1:maxClusters
rngest(j,1) = sumproductXbfrnggrid(j,1) / sumXbf(j,1);
end
%% Calculation of variance
for i = 1:numDetections
currentRngIdx = detidx(1,i);
currentDopIdx = detidx(2,i);
currentClusterID = clusterIDs(i);
XbfMag = abs(Xbf(currentRngIdx, currentDopIdx));
```

```
deviationSquared = (rnggrid(currentRngIdx) - rngest(currentClusterID,1))^2;
sumproductXbfRngGridDeviationSquared(currentClusterID,1) =
sumproductXbfRngGridDeviationSquared(currentClusterID,1) + (XbfMag *
deviationSquared);
end
for j = 1:maxClusters
rngvar(j,1) = sumproductXbfRngGridDeviationSquared(j,1) / sumXbf(j,1);
end
end
```

```
function [velest velvar] = customRangeEstimator(Xbf, detidx, dopgrid,
numDetections, clusterIDs, maxClusters)
% Xbf: Beamformed signal array
% detidx: Range and doppler bin numbers of the detections
% dopgrid: Doppler axis of the range-doppler map
% numDetections: Total number of detections observed
% clusterIDs: The cluster number of each detection
% maxClusters: Total number of clusters formed
%% Initializations
sumproductXbfDopgrid = zeros(maxClusters,1);
sumproductXbfDopGridDeviationSquared= zeros(maxClusters,1);
velvar = zeros(maxClusters,1);
sumXbf = zeros(maxClusters,1);
velest = zeros(maxClusters,1);
%% Estimation of range
for i = 1:numDetections
currentRngIdx = detidx(1,i);
currentDopIdx = detidx(2,i);
currentClusterID = clusterIDs(i);
XbfMag = abs(Xbf(currentRngIdx, currentDopIdx));
sumproductXbfDopgrid(currentClusterID,1) =
sumproductXbfDopgrid(currentClusterID,1) + (XbfMag *
dopgrid(currentDopIdx));
sumXbf(currentClusterID,1) = sumXbf(currentClusterID,1) + XbfMag;
end
for j = 1:maxClusters
velest(j,1) = - (sumproductXbfDopgrid(j,1) / sumXbf(j,1));
end
%% Calculation of variance
for i = 1:numDetections
currentRngIdx = detidx(1,i);
currentDopIdx = detidx(2,i);
currentClusterID = clusterIDs(i);
```

```
XbfMag = abs(Xbf(currentRngIdx, currentDopIdx));
deviationSquared = (dopgrid(currentDopIdx) + velest(currentClusterID,1))^2;
sumproductXbfDopGridDeviationSquared(currentClusterID,1) =
sumproductXbfDopGridDeviationSquared(currentClusterID,1) + (XbfMag *
deviationSquared);
end
for j = 1:maxClusters
velvar(j,1) = sumproductXbfDopGridDeviationSquared(j,1) / sumXbf(j,1);
end
end
```

## A.2. Floating and Fixed Point Comparisons

```
% Set random number generator for repeatable results
rng(2017);

% Compute hardware parameters from specified long-range requirements
fc = 77e9; % Center frequency (Hz)
c = physconst('LightSpeed'); % Speed of light in air (m/s)
lambda = freq2wavelen(fc,c); % Wavelength (m)

% Set the chirp duration to be 5 times the max range requirement
rangeMax = 225; % Maximum range (m)
tm = 5*range2time(rangeMax,c); % Chirp duration (s)

% Determine the waveform bandwidth from the required range resolution
rangeRes = 0.5; % Desired range resolution (m)
bw = rangeres2bw(rangeRes,c); % Corresponding bandwidth (Hz)

% Set the sampling rate to satisfy both the range and velocity requirements
% for the radar
sweepSlope = bw/tm; % FMCW sweep slope (Hz/s)
fbeatMax = range2beat(rangeMax,sweepSlope,c); % Maximum beat frequency
(Hz)

vMax = 230*1000/3600; % Maximum Velocity of cars (m/s)
fdopMax = speed2dop(2*vMax,lambda); % Maximum Doppler shift (Hz)

fifMax = fbeatMax+fdopMax; % Maximum received IF (Hz)
fs = max(2*fifMax,bw); % Sampling rate (Hz)

% Configure the FMCW waveform using the waveform parameters derived from
% the long-range requirements
waveform = phased.FMCWWaveform('SweepTime',tm,'SweepBandwidth',bw,...
'SampleRate',fs,'SweepDirection','Up');
if strcmp(waveform.SweepDirection,'Down')
sweepSlope = -sweepSlope;
end
Nsweep = 192;
sig = waveform();
% Model the antenna element
```

```
antElmnt = phased.IsotropicAntennaElement('BackBaffled',true);

% Construct the receive array
Ne = 6;
rxArray = phased.ULA('Element',antElmnt,'NumElements',Ne,...
'ElementSpacing',lambda/2);

% Half-power beamwidth of the receive array
hpbw = beamwidth(rxArray,fc,'PropagationSpeed',c)
antAperture = 6.06e-4; % Antenna aperture (m^2)
antGain = aperture2gain(antAperture,lambda); % Antenna gain (dB)

txPkPower = db2pow(5)*1e-3; % Tx peak power (W)
txGain = antGain; % Tx antenna gain (dB)

rxGain = antGain; % Rx antenna gain (dB)
rxNF = 4.5; % Receiver noise figure (dB)

% Waveform transmitter
transmitter = phased.Transmitter('PeakPower',txPkPower,'Gain',txGain);

% Radiator for single transmit element
radiator = phased.Radiator('Sensor',antElmnt,'OperatingFrequency',fc);

% Collector for receive array
collector = phased.Collector('Sensor',rxArray,'OperatingFrequency',fc);

% Receiver preamplifier
receiver = phased.ReceiverPreamp('Gain',rxGain,'NoiseFigure',rxNF,...
'SampleRate',fs);

% Define radar
radar = radarTransceiver('Waveform',waveform,'Transmitter',transmitter,...
'TransmitAntenna',radiator,'ReceiveAntenna',collector,'Receiver',receiver);
% Direction-of-arrival estimator for linear phased array signals
doaest = phased.RootMUSICEstimator(...
'SensorArray',rxArray,...
'PropagationSpeed',c,'OperatingFrequency',fc,...
'NumSignalsSource','Property','NumSignals',1);
```

```
% Scan beams in front of ego vehicle for range-angle image display
angscan = -80:80;
beamscan = phased.PhaseShiftBeamformer('Direction',[angscan;0*angscan],...
'SensorArray',rxArray,'OperatingFrequency',fc);

% Form forward-facing beam to detect objects in front of the ego vehicle
beamformer = phased.PhaseShiftBeamformer('SensorArray',rxArray,...
'PropagationSpeed',c,'OperatingFrequency',fc,'Direction',[0;0]);
Nft = waveform.SweepTime*waveform.SampleRate; % Number of fast-time
samples
Nst = Nsweep; % Number of slow-time samples
Nr = 2^nextpow2(Nft); % Number of range samples
Nd = 2^nextpow2(Nst); % Number of Doppler samples
rngdopresp = phased.RangeDopplerResponse('RangeMethod','FFT',...
'DopplerOutput','Speed','SweepSlope',sweepSlope,...
'RangeFFTLengthSource','Property','RangeFFTLength',Nr,...
'RangeWindow','Hann',...
'DopplerFFTLengthSource','Property','DopplerFFTLength',Nd,...
'DopplerWindow','Hann',...
'PropagationSpeed',c,'OperatingFrequency',fc,'SampleRate',fs);
% Guard cell and training regions for range dimension
nGuardRng = 4;
nTrainRng = 4;
nCUTRng = 1+nGuardRng+nTrainRng;

% Guard cell and training regions for Doppler dimension
dopOver = round(Nd/Nsweep);
nGuardDop = 4*dopOver;
nTrainDop = 4*dopOver;
nCUTDop = 1+nGuardDop+nTrainDop;

cfar = phased.CFARDetector2D('GuardBandSize',[nGuardRng nGuardDop],...
'TrainingBandSize',[nTrainRng nTrainDop],...
'ThresholdFactor','Custom','CustomThresholdFactor',db2pow(13),...
'NoisePowerOutputPort',true,'OutputFormat','Detection index');

% Perform CFAR processing over all of the range and Doppler cells
freqs = ((0:Nr-1)'/Nr-0.5)*fs;
rnggrid = beat2range(freqs,sweepSlope);
iRngCUT = find(rnggrid>0);
```

```
iRngCUT = iRngCUT((iRngCUT>=nCUTRng)&(iRngCUT<=Nr-
nCUTRng+1));
iDopCUT = nCUTDop:(Nd-nCUTDop+1);
[iRng,iDop] = meshgrid(iRngCUT,iDopCUT);
idxCFAR = [iRng(:) iDop(:)]';

% Perform clustering algorithm to group detections
clusterer = clusterDBSCAN('Epsilon',2);
rmsRng = sqrt(12)*rangeRes;
rngestimator = phased.RangeEstimator('ClusterInputPort',true,...
'VarianceOutputPort',true,'NoisePowerSource','Input port',...
'RMSResolution',rmsRng);

dopestimator = phased.DopplerEstimator('ClusterInputPort',true,...
'VarianceOutputPort',true,'NoisePowerSource','Input port',...
'NumPulses',Nsweep);
tracker = radarTracker('FilterInitializationFcn',@initcvekf,...
'AssignmentThreshold',50);
% Create driving scenario
[scenario,egoCar,radarParams] = ...
helperAutoDrivingRadarSigProc('Setup Scenario',c,fc);
% Initialize display for driving scenario example
helperAutoDrivingRadarSigProc('Initialize Display',egoCar,radarParams,...
rxArray,fc,vMax,rangeMax);

tgtProfiles = actorProfiles(scenario);
tgtProfiles = tgtProfiles(2:end);
tgtHeight = [tgtProfiles.Height];

% Run the simulation loop
sweepTime = waveform.SweepTime;
while advance(scenario)
% Get the current scenario time
time = scenario.SimulationTime;
% Get current target poses in ego vehicle's reference frame
tgtPoses = targetPoses(egoCar);
tgtPos = reshape([tgtPoses.Position],3,[]);
% Position point targets at half of each target's height
tgtPos(3,:) = tgtPos(3,:)+0.5*tgtHeight;
tgtVel = reshape([tgtPoses.Velocity],3,[]);
```

```
% Assemble data cube at current scenario time
Xcube = zeros(Nft,Ne,Nsweep);
for m = 1:Nsweep
ntgt = size(tgtPos,2);
tgtStruct = struct('Position',mat2cell(tgtPos(:).',1,repmat(3,1,ntgt)),...
'Velocity',mat2cell(tgtVel(:).',1,repmat(3,1,ntgt)),...
'Signature',{rcsSignature,rcsSignature,rcsSignature});
rxsig = radar(tgtStruct,time+(m-1)*sweepTime);
% Dechirp the received signal
rxsig = dechirp(rxsig,sig);
% Save sweep to data cube
Xcube(:,:,m) = rxsig;
% Move targets forward in time for next sweep
tgtPos = tgtPos+tgtVel*sweepTime;
end
% Calculate the range-Doppler response
[Xrngdop,rnggrid,dopgrid] = rngdopresp(Xcube);
% % Beamform received data
% Xbf = permute(Xrngdop,[1 3 2]);
% Xbf = reshape(Xbf,Nr*Nd,Ne);
% Xbf = beamformer(Xbf);
% Xbf = reshape(Xbf,Nr,Nd);
%
lookDirection = 0;
%[Xbf] = customBeamforming(Xrngdop, Nr, Ne, Nd, fc, lookDirection);
[Xbf] = customBeamformingFixed(Xrngdop, Nr, Ne, Nd, fc, lookDirection);
% Detect targets
Xpow = abs(Xbf).^2;
[detidx,noisepwr] = cfar(Xpow,idxCFAR);
% Cluster detections
[~,clusterIDs] = clusterer(detidx.');
% Estimate azimuth, range, and radial speed measurements
[azest,azvar,snrdB] = ...
helperAutoDrivingRadarSigProc('Estimate Angle',doaest,...
conj(Xrngdop),Xbf,detidx,noisepwr,clusterIDs);
azvar = azvar+radarParams.RMSBias(1)^2;
[rngest,rngvar] = rngestimator(Xbf,rnggrid,detidx,noisepwr,clusterIDs);
rngvar = rngvar+radarParams.RMSBias(2)^2;
[rsest,rsvar] = dopestimator(Xbf,dopgrid,detidx,noisepwr,clusterIDs);
```

```
% Convert radial speed to range rate for use by the tracker
rrest = -rsest;
rrvar = rsvar;
rrvar = rrvar+radarParams.RMSBias(3)^2;
% Assemble object detections for use by tracker
numDets = numel(rngest);
dets = cell(numDets,1);
for iDet = 1:numDets
dets{iDet} = objectDetection(time,...
[azest(iDet) rngest(iDet) rrest(iDet)]',...
'MeasurementNoise',diag([azvar(iDet) rngvar(iDet) rrvar(iDet)]),...
'MeasurementParameters',{radarParams},...
'ObjectAttributes',{struct('SNR',snrdB(iDet))});
end
% Track detections
tracks = tracker(dets,time);
% Update displays
helperAutoDrivingRadarSigProc('Update Display',egoCar,dets,tracks,...
dopgrid,rnggrid,Xbf,beamscan,Xrngdop);
% Collect free space channel metrics
metricsFS = helperAutoDrivingRadarSigProc('Collect Metrics',...
radarParams,tgtPos,tgtVel,dets);
end
```

```
% MATLAB Beamformer & custom beamformer (fixed)
% Parameters
Nr = Nr; % Number of range bins
Ne = Ne; % Number of antenna elements
Nd = Nd; % Number of Doppler bins
fc = 77e9; % Carrier frequency
lookDirection = 30; % Look direction in degrees
% Input Data
Xrngdop = Xrngdop; % Ensure this is already loaded or generated
% MATLAB Beamforming
Xbf = permute(Xrngdop, [1 3 2]); % Rearrange dimensions
Xbf = reshape(Xbf, Nr * Nd, Ne); % Reshape for beamformer
Xbf = beamformer(Xbf); % MATLAB beamforming function
Xbf = reshape(Xbf, Nr, Nd); % Reshape back to original dimensions
% Custom Fixed-Point Beamforming
tic;
```

```
Xbf_fixed = customBeamforming_fixed_scaled1(Xrngdop, Nr, Ne, Nd, fc,
lookDirection);
time_fixed = toc;
% Mean Squared Error Calculation
mse = mean(abs(double(Xbf) - double(Xbf_fixed)).^2, 'all');
fprintf('Mean Squared Error (MSE) between MATLAB and Custom Fixed-Point
Beamformer: %.4e\n', mse);
% Range Bin Comparison Plot
range_bin = 1; % Select a specific range bin for comparison
MATLAB = abs(Xbf(range_bin, :));
Custom = abs(Xbf_fixed(range_bin, :));
figure;
plot(MATLAB, 'b-', 'LineWidth', 1.5);
hold on;
plot(Custom, 'r--', 'LineWidth', 1.5);
grid on;
title(sprintf('Comparison of Outputs for Range Bin %d', range_bin));
xlabel('Doppler Index');
ylabel('Magnitude');
legend('MATLAB Output', 'CUSTOM Fixed-Point Output', 'Location', 'best');
% Heatmap Comparison
MATLAB_output = abs(Xbf); % Magnitude of MATLAB output
Custom_output_fixed = abs(Xbf_fixed); % Magnitude of custom fixed-point output
Difference = abs(MATLAB_output - Custom_output_fixed); % Difference between
outputs
figure;
subplot(1,3,1);
imagesc(MATLAB_output);
colorbar;
title('MATLAB Beamforming Output');
xlabel('Doppler Index');
ylabel('Range Bin');
set(gca, 'YDir', 'normal'); % Ensure the heatmap is not flipped
subplot(1,3,2);
imagesc(Custom_output_fixed);
colorbar;
title('Custom Fixed-Point Output');
xlabel('Doppler Index');
ylabel('Range Bin');
set(gca, 'YDir', 'normal');
subplot(1,3,3);
imagesc(Difference);
colorbar;
title('Difference (MATLAB - Custom Fixed)');
```

```
xlabel('Doppler Index');
ylabel('Range Bin');
set(gca, 'YDir', 'normal');
sgtitle('Heatmap Comparison of Beamforming Outputs (Fixed-Point)');
```

```
%MATLAB Beamformer & custom beamformer (float)
% Parameters
Nr = Nr; % Number of range bins
Ne = Ne; % Number of antenna elements
Nd = Nd; % Number of Doppler bins
fc = 77e9; % Carrier frequency
lookDirection = 30; % Look direction in degrees
Xrngdop = Xrngdop;
% Beamform received data
Xbf = permute(Xrngdop,[1 3 2]);
Xbf = reshape(Xbf,Nr*Nd,Ne);
Xbf = beamformer(Xbf);
Xbf = reshape(Xbf,Nr,Nd);
tic;
Xbf_float = customBeamforming(Xrngdop, Nr, Ne, Nd, fc, lookDirection);
time_float = toc;
mse = mean(abs(Xbf - Xbf_float).^2, 'all');
fprintf('Mean Squared Error (MSE) between MATLAB and our Beamformer:
%.4e\n', mse);
% Heatmap comparison
MATLAB_output = abs(Xbf); % Magnitude of MATLAB output
Custom_output = abs(Xbf_float); % Magnitude of custom output
Difference = abs(MATLAB_output - Custom_output); % Difference between
outputs
% Range Bin for Comparison
range_bin = 1; % Specify the desired range bin for comparison
% Extract data for the selected range bin
MATLAB_range_bin = abs(Xbf(range_bin, :));
Custom_range_bin = abs(Xbf_float(range_bin, :));
figure;
plot(MATLAB_range_bin, 'b-', 'LineWidth', 1.5); % MATLAB output
hold on;
plot(Custom_range_bin, 'r--', 'LineWidth', 1.5); % Custom output
grid on;
title(sprintf('Comparison of Outputs for Range Bin %d', range_bin));
xlabel('Doppler Index');
ylabel('Magnitude');
legend('MATLAB Output', 'Custom Float-Point Output', 'Location', 'best');
```

```
figure;
subplot(1,3,1);
imagesc(MATLAB_output);
colorbar;
title('MATLAB Beamforming Output');
xlabel('Doppler Index');
ylabel('Range Bin');
set(gca, 'YDir', 'normal');
subplot(1,3,2);
imagesc(Custom_output);
colorbar;
title('Custom Beamforming Output');
xlabel('Doppler Index');
ylabel('Range Bin');
set(gca, 'YDir', 'normal');
subplot(1,3,3);
imagesc(Difference);
colorbar;
title('Difference (MATLAB - Custom)');
xlabel('Doppler Index');
ylabel('Range Bin');
set(gca, 'YDir', 'normal');
sgtitle('Heatmap Comparison of Beamforming Outputs');
```

```
% MATLAB CFAR & custom CFAR (float)
% Parameters
idxCFAR = idxCFAR;
nGuard = nGuardRng;
nTrain = nTrainRng;
Nd = Nd;
Nsweep = Nsweep;
Nr = Nr;
rnggrid = rnggrid;
customFactor = 13;
[detidx_builtin, noisepwr_builtin] = cfar(Xpow, idxCFAR);
[detidx_custom, noisepwr_custom, numDetections_custom] =
customCFARDetector( ...
Xpow, nGuard, nTrain, Nd, Nsweep, Nr, rnggrid, customFactor);
disp('--- Comparing CFAR Detection Results ---');
% 1. Number of Detections
numDetections_builtin = size(detidx_builtin, 2);
disp(['Number of Detections (Built-in): ', num2str(numDetections_builtin)]);
```

```
disp(['Number of Detections (Custom): ', num2str(numDetections_custom)]);
if numDetections_builtin == numDetections_custom
disp('Number of detections match.');
else
disp(['Number of detections differ by: ', ...
num2str(abs(numDetections_builtin - numDetections_custom))]);
end
% 2. Detection Indices Comparison
det_indices_match = ismember(detidx_builtin', detidx_custom', 'rows');
mismatched_indices = detidx_builtin(:, ~det_indices_match);
num_mismatched = size(mismatched_indices, 2);
disp(['Number of mismatched detection indices: ', num2str(num_mismatched)]);
if num_mismatched > 0
disp('Mismatched detection indices:');
disp(mismatched_indices);
end
% 3. Noise Power Comparison
minLength = min(length(noisepwr_builtin), length(noisepwr_custom));
noisepwr_builtin_trimmed = noisepwr_builtin(1:minLength);
noisepwr_custom_trimmed = noisepwr_custom(1:minLength);
% Compute error metrics
noise_diff = abs(noisepwr_builtin_trimmed - noisepwr_custom_trimmed);
mean_noise_error = mean(noise_diff);
max_noise_error = max(noise_diff);
disp(['Mean noise power error: ', num2str(mean_noise_error)]);
disp(['Max noise power error: ', num2str(max_noise_error)]);
% 4. Visualize Results
figure;
imagesc(Xpow);
hold on;
% Plot built-in detections
plot(detidx_builtin(2, :), detidx_builtin(1, :), 'r*', 'DisplayName', 'MATLAB
CFAR');
% Plot custom detections
plot(detidx_custom(2, :), detidx_custom(1, :), 'go', 'DisplayName', 'Custom CFAR');
legend;
title('CFAR Detection Comparison');
xlabel('Doppler Bins');
ylabel('Range Bins');
hold off;
% 5. Highlight Differences on the Plot
if num_mismatched > 0
figure;
imagesc(Xpow);
```

```
hold on;
plot(mismatched_indices(2, :), mismatched_indices(1, :), 'kx', 'DisplayName',
'Mismatched Detections');
legend;
title('Mismatched Detection Points');
xlabel('Doppler Bins');
ylabel('Range Bins');
hold off;
end
disp('--- Comparison Completed ---');
```

```
% MATLAB CFAR & custom CFAR (fixed)
% Parameters
idxCFAR = idxCFAR;
nGuard = nGuardRng;
nTrain = nTrainRng;
Nd = Nd;
Nsweep = Nsweep;
Nr = Nr;
rnggrid = rnggrid;
customFactor = 13;
% Built-in CFAR detection
[detidx_builtin, noisepwr_builtin] = cfar(Xpow, idxCFAR);
% Custom CFAR detection (Fixed-point)
[detidx_custom, noisepwr_custom, numDetections_custom] =
customCFARDetector_fixed_scaled( ...
Xpow, nGuard, nTrain, Nd, Nsweep, Nr, rnggrid, customFactor);
disp('--- Comparing CFAR Detection Results ---');
% 1. Number of Detections
numDetections_builtin = size(detidx_builtin, 2);
disp(['Number of Detections (Built-in): ', num2str(numDetections_builtin)]);
disp(['Number of Detections (Custom Fixed-point): ',
num2str(numDetections_custom)]);
if numDetections_builtin == numDetections_custom
disp('Number of detections match.');
else
disp(['Number of detections differ by: ', ...
num2str(abs(numDetections_builtin - numDetections_custom))]);
end
% 2. Detection Indices Comparison
det_indices_match = ismember(detidx_builtin', detidx_custom', 'rows');
mismatched_indices = detidx_builtin(:, ~det_indices_match);
num_mismatched = size(mismatched_indices, 2);
```

```
disp(['Number of mismatched detection indices: ', num2str(num_mismatched)]);
if num_mismatched > 0
disp('Mismatched detection indices:');
disp(mismatched_indices);
end
% 3. Noise Power Comparison
minLength = min(length(noisepwr_builtin), length(noisepwr_custom));
noisepwr_builtin_trimmed = noisepwr_builtin(1:minLength);
noisepwr_custom_trimmed = noisepwr_custom(1:minLength);
% Compute error metrics
noise_diff = abs(noisepwr_builtin_trimmed - noisepwr_custom_trimmed);
mean_noise_error = mean(noise_diff);
max_noise_error = max(noise_diff);
disp(['Mean noise power error: ', num2str(mean_noise_error)]);
disp(['Max noise power error: ', num2str(max_noise_error)]);
% 4. Visualize Results
figure;
imagesc(Xpow);
hold on;
% Plot built-in detections
plot(detidx_builtin(2, :), detidx_builtin(1, :), 'r*', 'DisplayName', 'MATLAB
CFAR');
% Plot custom fixed-point detections
plot(detidx_custom(2, :), detidx_custom(1, :), 'go', 'DisplayName', 'Custom CFAR
(Fixed-point)');
legend;
title('CFAR Detection Comparison');
xlabel('Doppler Bins');
ylabel('Range Bins');
hold off;
% 5. Highlight Differences on the Plot
if num_mismatched > 0
figure;
imagesc(Xpow);
hold on;
plot(mismatched_indices(2, :), mismatched_indices(1, :), 'kx', 'DisplayName',
'Mismatched Detections');
legend;
title('Mismatched Detection Points');
xlabel('Doppler Bins');
ylabel('Range Bins');
hold off;
end
disp('--- Comparison Completed ---');
```

```
% % custom CFAR (float & fixed)
% Test parameters
Nr = Nr; % Number of range bins
Nd = Nd; % Number of Doppler bins
Nsweep = Nsweep; % Number of sweeps
nGuard = nGuardRng; % Number of guard cells
nTrain = nTrainRng; % Number of training cells
customFactor = 13; % Custom threshold factor
rnggrid = rnggrid; % Example range grid
fc = 77e9; % Example radar frequency
% Generate power spectrum (assuming Xbf is complex)
Xpow = abs(Xbf).^2;
% Run floating-point CFAR detector
[detidx_float, noisepwr_float, numDetections_float] = customCFARDetector( ...
Xpow, nGuard, nTrain, Nd, Nsweep, Nr, rnggrid, customFactor);
% Run fixed-point CFAR detector
[detidx_fixed, noisepwr_fixed, numDetections_fixed] =
customCFARDetector_fixed_scaled( ...
Xpow, nGuard, nTrain, Nd, Nsweep, Nr, rnggrid, customFactor);
% Check if detection indices match
detidx_match = isequal(detidx_float, detidx_fixed);
% Convert noisepwr_fixed to double for comparison
noisepwr_fixed_double = double(noisepwr_fixed); % Convert to double to compare
% Ensure both noisepwr_float and noisepwr_fixed_double are column vectors
noisepwr_float = reshape(noisepwr_float, [], 1);
noisepwr_fixed_double = reshape(noisepwr_fixed_double, [], 1);
% If sizes are not the same, pad the smaller one with zeros
if numel(noisepwr_float) ~= numel(noisepwr_fixed_double)
% Pad the smaller array with zeros (or NaN if you prefer)
if numel(noisepwr_float) > numel(noisepwr_fixed_double)
noisepwr_fixed_double = [noisepwr_fixed_double; zeros(numel(noisepwr_float) -
numel(noisepwr_fixed_double), 1)];
else
noisepwr_float = [noisepwr_float; zeros(numel(noisepwr_fixed_double) -
numel(noisepwr_float), 1)];
end
end
% Compute noise power error
noisepwr_error = abs(noisepwr_float - noisepwr_fixed_double); % Compute
absolute error
noisepwr_error_max = max(noisepwr_error(:)); % Maximum error in noise power
```

```
% Compare number of detections
numDetections_error = abs(numDetections_float - numDetections_fixed);
% Display results
fprintf('Detection indices match: %d\n', detidx_match);
fprintf('Maximum noise power error: %.6f\n', noisepwr_error_max);
fprintf('Difference in number of detections: %d\n', numDetections_error);
% Optional: Visualize results
figure;
subplot(1, 2, 1);
scatter(detidx_float(1, :), detidx_float(2, :), 'b'); hold on;
scatter(detidx_fixed(1, :), detidx_fixed(2, :), 'r');
legend('Float', 'Fixed');
title('Detection Indices Comparison');
subplot(1, 2, 2);
plot(noisepwr_error);
title('Noise Power Error');
xlabel('Detection Index');
ylabel('Error');
```

## A.3. RTL Input Signal Generation Code

```
clc;
clear all;
close all;
%% Patameters
% Set random number generator for repeatable results
rng(2017);
% Compute hardware parameters from specified long-range requirements
fc = 77e9; % Center frequency (Hz)
c = physconst('LightSpeed'); % Speed of light in air (m/s)
lambda = freq2wavelen(fc,c); % Wavelength (m)
% Set the chirp duration to be 5 times the max range requirement
rangeMax = 225; % Maximum range (m)
tm = 5.5*range2time(rangeMax,c); % Chirp duration (s)
% Determine the waveform bandwidth from the required range resolution
rangeRes = 0.5; % Desired range resolution (m)
bw = rangeres2bw(rangeRes,c); % Corresponding bandwidth (Hz)
% Set the sampling rate to satisfy both the range and velocity requirements
% for the radar
sweepSlope = bw/tm; % FMCW sweep slope (Hz/s)
fbeatMax = range2beat(rangeMax,sweepSlope,c); % Maximum beat frequency (Hz)
vMax = 93*1000/3600; % Maximum Velocity of cars (m/s)
fdopMax = speed2dop(2*vMax,lambda); % Maximum Doppler shift (Hz)
```

```
fifMax = fbeatMax+fdopMax; % Maximum received IF (Hz)
fs = 256/tm; % Sampling rate (Hz)
% Configure the FMCW waveform using the waveform parameters derived from
% the long-range requirements
waveform = phased.FMCWWaveform('SweepTime',tm,'SweepBandwidth',bw,...
'SampleRate',fs,'SweepDirection','Up');
if strcmp(waveform.SweepDirection,'Down')
sweepSlope = -sweepSlope;
end
Nsweep = 128;
Nft = ceil(waveform.SweepTime*waveform.SampleRate); % Number of fast-time
samples
sig = waveform();
Nr = Nft;
Nd = Nsweep;
% Nr = 2^nextpow2(Nft);
% Nd = 2^nextpow2(Nsweep);
%% RF Front-end modeling
% Model the antenna element
antElmnt = phased.IsotropicAntennaElement('BackBaffled',true);
% Construct the receive array
Ne = 1;
% rxArray = phased.ULA('Element',antElmnt,'NumElements',Ne,...
% 'ElementSpacing',lambda/2);
% Half-power beamwidth of the receive array
% hpbw = beamwidth(rxArray,fc,'PropagationSpeed',c);
%% Model the radar transmitter
txPkPower = db2pow(13)*1e-3; % previously 5 % Tx peak power (W)
txGain = 17; % previously 27 % Tx antenna gain (dB)
rxGain = 32; % previously 27 % Rx antenna gain (dB)
rxNF = 12; % previously 4.5 % Receiver noise figure (dB)
% Waveform transmitter
transmitter = phased.Transmitter('PeakPower',txPkPower,'Gain',txGain);
% Radiator for single transmit element
radiator = phased.Radiator('Sensor',antElmnt,'OperatingFrequency',fc);
% Collector for receive array
collector = phased.Collector('Sensor',antElmnt,'OperatingFrequency',fc);
% Receiver preamplifier
receiver = phased.ReceiverPreamp('Gain',rxGain,'NoiseFigure',rxNF,...
'SampleRate',fs);
% Define radar
radar = radarTransceiver('Waveform',waveform,'Transmitter',transmitter,...
'TransmitAntenna',radiator,'ReceiveAntenna',collector,'Receiver',receiver);
%% Additional parameters for processing and estimations
```

```
% Frequency vector corresponding to FFT bins
freqRng = (0:Nr-1)*(fs/Nr);
% Preparing for doppler shift
Nd = 2^nextpow2(Nsweep);
fds = 1/tm;
% Frequency vector corresponding to FFT bins
freq_dop = (-Nd/2:Nd/2-1)*(fds/Nd);
%% Making windows for FFTs
hammingRange = hamming(Nft);
hammingDoppler = hamming(Nd);
%% Simulate the Driving ScenarioNr
% Create driving scenario
[scenario,egoCar,radarParams] = ...
helperAutoDrivingRadarSigProc('Setup Scenario',c,fc);
% Running Simulation
% Initialize display for driving scenario example
helperAutoDrivingRadarSigProc('Initialize Display',egoCar,radarParams,...
antElmnt,fc,vMax,rangeMax);
tgtProfiles = actorProfiles(scenario);
tgtProfiles = tgtProfiles(2:end);
tgtHeight = [tgtProfiles.Height];
% Run the simulation loop
sweepTime = waveform.SweepTime;
while advance(scenario)
% Get the current scenario time
time = scenario.SimulationTime;
% Get current target poses in ego vehicle's reference frame
tgtPoses = targetPoses(egoCar);
tgtPos = reshape([tgtPoses.Position],3,[]);
% Position point targets at half of each target's height
tgtPos(3,:) = tgtPos(3,:)+0.5*tgtHeight;
tgtVel = reshape([tgtPoses.Velocity],3,[]);
% Acquire receive signals and process them till range FFT at current scenario time
for dopplerIndex = 1:Nsweep
ntgt = size(tgtPos,2);
tgtStruct = struct('Position',mat2cell(tgtPos(:).',1,repmat(3,1,ntgt)),...
'Velocity',mat2cell(tgtVel(:).',1,repmat(3,1,ntgt)),...
'Signature',{rcsSignature,rcsSignature});
rxsig = radar(tgtStruct,time+(dopplerIndex-1)*sweepTime); % Signal before mixer
block
rx_plot(:,dopplerIndex, :) = rxsig;
dechirp_plot(:,dopplerIndex, :) = dechirp(rxsig, sig);
% Channeling the received signal
for antennaIndex = 1:Ne
```

```
dechirpSignal3D(:,dopplerIndex,antennaIndex) =
dechirp(rxsig(:,antennaIndex),sig);
end
% Move targets forward in time for next sweep
tgtPos = tgtPos+tgtVel*sweepTime;
disp(dopplerIndex)
end
% STEP 1: Scale up the values
scaleUpFactor = 2^12; % Example: scale up by 4096 to bring values closer to ±1
scaledSignal = dechirpSignal3D * scaleUpFactor;
% STEP 2: Separate real and imaginary parts
realPart = abs(real(scaledSignal));
imagPart = abs(imag(scaledSignal));
% STEP 3: Convert to Q1.15 format
q15Scale = 2^15; % 32768
real_q15 = round(realPart * q15Scale);
imag_q15 = round(imagPart * q15Scale);
% Clip to [-32768, 32767]
real_q15 = max(min(real_q15, 32767), -32768);
imag_q15 = max(min(imag_q15, 32767), -32768);
% STEP 4: Save to specially formatted text files
saveFormattedMatrix(real_q15, 'realPart_Q15.txt');
saveFormattedMatrix(imag_q15, 'imagPart_Q15.txt');
disp('Done! Saved real and imaginary parts into Q1.15 format with special
formatting.');
end
% ===== Helper Function to save formatted =====
function saveFormattedMatrix(matrix, filename)
fid = fopen(filename, 'w');
[rows, cols] = size(matrix);
for r = 1:rows
fprintf(fid, " '{ ");
for c = 1:cols
if c ~= cols
fprintf(fid, '%6d, ', matrix(r, c)); % Comma after every element except last
else
fprintf(fid, '%6d ', matrix(r, c)); % No comma after last element
end
end
if r ~= rows
fprintf(fid, '},\n'); % Add comma between rows
else
fprintf(fid, '}\n'); % No comma after the last row
end
```

```
end
fclose(fid);
end
```

```
% This is a helper function and may be changed or removed without notice.
% Copyright 2017 The MathWorks, Inc.
function varargout = helperAutoDrivingRadarSigProc(method,varargin)
persistent figHandle BEP rangeImageHandles metrics
varargout = {};
switch method
case 'Setup Scenario'
[scenario,egoCar,radarParams,tgts] = setupScenario(varargin{:});
varargout = {scenario,egoCar,radarParams,tgts};
case 'Initialize Display'
[figHandle, BEP, rangeImageHandles] = initDisplay(varargin{:});
case 'Update Display'
updateDisplay(varargin{:},BEP, rangeImageHandles);
case 'Run Simulation'
clear metrics
metrics = runSimulation(varargin{:});
varargout = {metrics};
case 'Two Ray Simulation'
clear metrics
metrics = runSimulation2ray(varargin{:});
varargout = {metrics};
case 'Plot Channels'
plotChannels(varargin{:});
case 'Collect Metrics'
metrics = collectMetrics(metrics,varargin{:});
if nargout>0
varargout = {metrics};
end
case 'Publish Snapshot'
publishSnapshot(figHandle,varargin{:});
case 'Cluster Detections'
clusterIDs = clusterDetections(varargin{:});
varargout = {clusterIDs};
case 'Estimate Angle'
[angest, angvar, snrdB] = doaestimator(varargin{:});
varargout = {angest, angvar, snrdB};
case 'Array Beamwidth'
bw = arraybeamwidth(varargin{:});
varargout = {bw};
```

```
case 'Create Target Struct'
tgtstruct = createTargetStruct(varargin{:});
varargout = {tgtstruct};
end
end
function [scenario,egoCar,radarParams,tgts]= setupScenario(c,fc)
% Create a driving scenario
scenario = drivingScenario;
scenario.SampleTime = 0.05; % s
scenario.StopTime = 0.1;
% Create road
roadCenters = [0 0; 50 0; 100 0; 250 20; 500 40];
% Six lanes, each 3.6 meters
ls = lanespec(6);
laneWidth = ls.Width(1);
road(scenario, roadCenters+[0 laneWidth/2], 'lanes',ls);
initialdist = 100+[0; 50; 70];
laneOffset = [0; 1; 1]*laneWidth;
initialspd = [30; 50; 70]*1000/3600;
egoCar = addCars(scenario,roadCenters,initialdist,laneOffset,initialspd);
radarPos = [3.4 0 0.2]';
radarYaw = 0;
radarPitch = 0;
radarRoll = 0;
radarAxes = rotz(radarYaw)*roty(radarPitch)*rotx(radarRoll);
radarParams = struct( ...
'Frame', drivingCoordinateFrameType.Spherical, ...
'OriginPosition',radarPos, ...
'OriginVelocity',zeros(3,1), ...
'Orientation', radarAxes, ...
'HasElevation', false, ...
'HasVelocity', true, ...
'RMSBias',[0 0.25 0.05]);
tgts = createPointTargets(scenario,c,fc);
end
function [hFigure, BEP, hndls] = initDisplay(egoCar, radarParams, rx_array, fc,
vmax, frontLim)
% Make a figure
hFigure = findobj('Tag',mfilename);
if isempty(hFigure)
hFigure = figure('Name', 'Automated Driving', 'Tag', mfilename, 'Visible','off');
set(hFigure,'Position', [0, 0, 1032, 600]); % This is the maximum figure size that
will print without being clipped
movegui(hFigure , [0 -1]) ;
```

```
end
clf(hFigure);
hFigure.Visible = 'off';
% Add a car plot that follows the ego vehicle from behind
hCarViewPanel = uipanel(hFigure, 'Position', [0 2/3 1/2 1/3], 'Title', 'Chase Camera
View');
hCarPlot = axes(hCarViewPanel);
chasePlot(egoCar, 'Parent', hCarPlot);
% Add a panel for a bird's-eye plot
hBEVPanel = uipanel(hFigure, 'Position', [1/2 0 1/2 1], 'Title', 'Bird''s-Eye Plot');
% Add a panel for range doppler plot
hDRPanel = uipanel(hFigure, 'Position', [0 1/3 1/2 1/3], 'Title', 'Range-Doppler ');
hdrange = axes(hDRPanel);
% Add a panel for range-angle of arrival plot
hRAOAPanel = uipanel(hFigure, 'Position', [0 0 1/2 1/3], 'Title', 'Range-Angle of
Arrival');
hraoa= axes(hRAOAPanel);
% Create bird's-eye plot for the ego car and sensor coverage
% frontLim = 100;
backLim = -20;
hBEVPlot = axes(hBEVPanel);
BEP = birdsEyePlot('Parent', hBEVPlot, 'Xlimits', [-10 frontLim], 'Ylimits', [-35
35]);
% Combine all radar detections into one entry and store it for later update
detectionPlotter(BEP,'DisplayName','Radar
Detections','MarkerEdgeColor','red','MarkerFaceColor','red');
trackPlotter(BEP,'DisplayName','Tracks','HistoryDepth',10);
% Add road borders to plot
laneBoundaryPlotter(BEP, 'DisplayName','road', 'Color', 0.75*[1 1 1]);
xlim(BEP.Parent, [backLim frontLim]);
ylim(BEP.Parent, [-20 20]);
laneMarkingPlotter(BEP, 'DisplayName','lane markings');
% Add an outline plotter for ground truth
outlinePlotter(BEP, 'Tag', 'Ground truth');
% Add receive array radiation pattern
theta = (-90:90)';
ant_patt = patternAzimuth(rx_array, fc,0 , 'Azimuth', theta');
ant_patt = ant_patt-max(ant_patt); % normalize to peak gain
ant_patt_pow = db2pow(ant_patt) ;
scale = frontLim*0.9; % scale beam's peak to extent of bird's-eye plot
hold(hBEVPlot,'on');
plot(hBEVPlot,radarParams.OriginPosition(1)+scale*ant_patt_pow.*cosd(theta),...
radarParams.OriginPosition(2)+scale*ant_patt_pow.*sind(theta), 'r')
hold(hBEVPlot,'off');
```

```
% Initialize plot parameters for Range-Doppler response
hi_rng_dopp = imagesc(hdrange,[-50 50],[backLim frontLim], zeros(2,2));
hdrange.YDir = 'normal';
ylim(hdrange,[backLim frontLim])
xlim(hdrange,[-vmax vmax]*3600/1000); % m/s -> km/hr
caxis(hdrange,[10 60])
ylabel(colorbar('peer',hdrange),'SNR (dB)');
xlabel(hdrange,'Radial Speed (km/hr)')
ylabel(hdrange,'Range (m)')
title(hdrange,'Range-Doppler Image')
% Initialize plot parameters for Doppler-Angle of Arrival response
hi_rng_aoa = imagesc(hraoa,[backLim frontLim],[-60 60], zeros(2,2));
hraoa.YDir = 'normal';
ylim(hraoa,[backLim frontLim])
xlim(hraoa,[-80 80])
caxis(hraoa,[10 60])
ylabel(colorbar('peer',hraoa),'SNR (dB)');
xlabel(hraoa,'Angle of Arrival (deg)')
ylabel(hraoa,'Range (m)')
title(hraoa,'Range-Angle Image' )
hndls = [hi_rng_dopp, hi_rng_aoa];
end
function updateDisplay(egoCar, dets, tracks, dopgrid, rnggrid, Xbf, beamscan,
Xrngdop, BEP, rngImagHndls)
% Check if display is still open
if ~ishghandle(rngImagHndls(1))
return % no, nothing to update
end
if ~isPublishing()
hfig = get(get(get(rngImagHndls(1),'Parent'),'Parent'),'Parent');
hfig.Visible = 'on';
end
% Update road boundaries and their display
rb = roadBoundaries(egoCar);
plotLaneBoundary(findPlotter(BEP,'DisplayName','road'),rb)
% Update lane markings and their display
[lmv, lmf] = laneMarkingVertices(egoCar);
plotLaneMarking(findPlotter(BEP,'DisplayName','lane markings'),lmv, lmf)
% Update ground truth data
[position, yaw, length, width, originOffset, color] = targetOutlines(egoCar);
plotOutline(findPlotter(BEP,'Tag','Ground truth'), position, yaw, length, width,
'OriginOffset', originOffset, 'Color', color);
% Get detections in ego coordinate frame for bird's-eye plot
numDets = numel(dets);
```

```
pos = NaN(numDets,2);
lbls = cell(1,numDets);
for iDet = 1:numDets
meas = dets{iDet}.Measurement;
params = dets{iDet}.MeasurementParameters{1};
[x,y,z] = sph2cart(meas(1)*pi/180,0,meas(2));
Xradar = [x;y;z];
Xego = params.Orientation*Xradar+params.OriginPosition;
pos(iDet,:) = Xego(1:2);
lbls{iDet} = sprintf(' %1.1f dB\n', dets{iDet}.ObjectAttributes{1}.SNR);
end
if numDets>0
plotDetection(findPlotter(BEP,'DisplayName','Radar Detections'),pos,lbls);
end
positionSelector = [1 0 0 0 0 0; 0 0 1 0 0 0]; % Position selector
velocitySelector = [0 1 0 0 0 0; 0 0 0 1 0 0]; % Velocity selector
[tracksPos, tracksCov] = getTrackPositions(tracks, positionSelector);
tracksVel = getTrackVelocities(tracks, velocitySelector);
trackIDs = {tracks.TrackID};
labels = cellfun(@(x)sprintf('\n ID%i',x), trackIDs, 'UniformOutput', false);
plotTrack(findPlotter(BEP,'DisplayName','Tracks'), tracksPos, tracksVel, tracksCov,
labels);
Xpow = abs(Xbf).^2;
noisefloor = db2pow(-85);
Xsnr = pow2db(Xpow/noisefloor);
hRngDopPlot = rngImagHndls(1);
hRngDopPlot.XData = dopgrid*3600/1000; % m/s -> km/hr
hRngDopPlot.YData = rnggrid;
hRngDopPlot.CData = Xsnr;
[Nr,Ne,Nd] = size(Xrngdop);
Xrngdop = permute(Xrngdop,[1 3 2]); % Nr x Nd x Ne
Xrngdop = reshape(Xrngdop,[],Ne);
win = taylorwin(Ne,5,-60);
win = win/norm(win);
Xrngang = beamscan(Xrngdop.*win');
Xrngang = reshape(Xrngang,Nr,Nd,[]); % Nr x Nd x Ns
Xrngang = permute(Xrngang,[1 3 2]); % Nr x Ns x Nd
Xpow = abs(Xrngang).^2;
Xpow = max(Xpow,[],3);
Xsnr = pow2db(Xpow/noisefloor);
hRngAngPlot = rngImagHndls(2);
hRngAngPlot.XData = beamscan.Direction(1,:);
hRngAngPlot.YData = rnggrid;
hRngAngPlot.CData = Xsnr;
```

```
end
function metrics = runSimulation(c,fc,rangeMax,vMax,waveform,Nsweep,...
transmitter,radiator,collector,receiver,...
rngdopresp,beamformer,cfar,idxCFAR,...
rngestimator,dopestimator,doaest,beamscan,tracker,...
txchannel,rxchannel)
rng(2017);
reset(transmitter);
reset(radiator);
reset(collector);
reset(receiver);
reset(rngestimator);
reset(dopestimator);
reset(doaest);
reset(beamscan);
reset(tracker);
fs = waveform.SampleRate;
if nargin==20
if isprop(txchannel,'TwoWayPropagation') && txchannel.TwoWayPropagation
txchannel.TwoWayPropagation = false;
end
rxchannel = clone(txchannel);
elseif nargin<20
txchannel = phased.FreeSpace('PropagationSpeed',c,...
'OperatingFrequency',fc,'SampleRate',fs,'TwoWayPropagation',false);
rxchannel = clone(txchannel);
end
reset(txchannel);
reset(rxchannel);
[scenario,egoCar,radarParams,pointTgts] = helperAutoDrivingRadarSigProc('Setup
Scenario',c,fc);
Nft = waveform.SweepTime*waveform.SampleRate;
Ne = collector.Sensor.NumElements;
Nr = rngdopresp.RangeFFTLength;
Nd = rngdopresp.DopplerFFTLength;
rxArray = collector.Sensor;
% Initialize display for driving scenario example
helperAutoDrivingRadarSigProc('Initialize Display', egoCar, radarParams, rxArray,
fc, vMax, rangeMax);
tgtProfiles = actorProfiles(scenario);
tgtProfiles = tgtProfiles(2:end);
tgtHeight = [tgtProfiles.Height];
sweepTime = waveform.SweepTime;
while advance(scenario)
```

```
% Get the current scenario time
time = scenario.SimulationTime;
% Get current target poses in ego car's reference frame
tgtPoses = targetPoses(egoCar);
paths = phased.internal.twoRayPaths(radarParams.OriginPosition,...
reshape([tgtPoses.Position],3,[]),radarParams.OriginVelocity,...
reshape([tgtPoses.Velocity],3,[]),[1 1 1],c/fc);
% tgtPos = reshape([tgtPoses.Position],3,[]);
% tgtPos(3,:) = tgtPos(3,:)+0.5*tgtHeight; % Point targets are positioned at half of
each target's height
% tgtVel = reshape([tgtPoses.Velocity],3,[]);
% Simulate the radar transmit and receive operation
Xcube = zeros(Nft, Ne, Nsweep);
for m = 1:Nsweep
% % Calculate the target angles as seen by the sensor
% tgtAngs = NaN(2,numel(tgtPoses));
% for iTgt = 1:numel(tgtPoses)
% tgtAxes =
rotz(tgtPoses(iTgt).Yaw)*roty(tgtPoses(iTgt).Pitch)*rotx(tgtPoses(iTgt).Roll);
% [~,tgtAngs(:,iTgt)] =
rangeangle(radarParams.OriginPosition,tgtPos(:,iTgt),tgtAxes);
% end
%
% % Transmit FMCW waveform
% sig = waveform();
% [~, txang] = rangeangle(tgtPos, radarParams.OriginPosition,
radarParams.Orientation);
% txsig = transmitter(sig);
% txsig = radiator(txsig,txang);
%
% % Propagate the signal and reflect off the targets
% txsig =
txchannel(txsig,radarParams.OriginPosition,tgtPos,radarParams.OriginVelocity,tgtV
el);
% txsig = pointTgts(txsig,tgtAngs);
% txsig =
rxchannel(txsig,radarParams.OriginPosition,tgtPos,radarParams.OriginVelocity,tgtV
el);
%
% % Collect received target echos
% rxsig = collector(txsig, txang);
% rxsig = receiver(rxsig);
rxsig = radar(paths,time);
% Dechirp the received signal
```

```
rxsig = dechirp(rxsig, sig);
% Save sweep to data cube
Xcube(:,:,m) = rxsig;
% Move targets forward for next sweep
tgtPos = tgtPos+tgtVel*sweepTime;
end
% Calculate the range-doppler response
[Xrngdop,rnggrid,dopgrid] = rngdopresp(Xcube);
% Beamform received data
Xbf = permute(Xrngdop,[1 3 2]);
Xbf = reshape(Xbf,Nr*Nd,Ne);
Xbf = beamformer(Xbf);
Xbf = reshape(Xbf,Nr,Nd);
% Detect targets
Xpow = abs(Xbf).^2;
[detidx,noisepwr] = cfar(Xpow,idxCFAR);
% Cluster detections
clusterIDs = helperAutoDrivingRadarSigProc('Cluster Detections', detidx);
% Azimuth, range, and range rate measurements
[azest,azvar,snrdB] = helperAutoDrivingRadarSigProc('Estimate Angle', doaest,
conj(Xrngdop), Xbf, detidx, noisepwr, clusterIDs);
azvar = azvar+radarParams.RMSBias(1)^2;
[rngest,rngvar] = rngestimator(Xbf,rnggrid,detidx,noisepwr,clusterIDs);
rngvar = rngvar+radarParams.RMSBias(2)^2;
[rsest,rsvar] = dopestimator(Xbf,dopgrid,detidx,noisepwr,clusterIDs);
% Radial speed is estimated, but the tracker expects range rate.
% Convert radial speed to range rate.
rrest = -rsest;
rrvar = rsvar;
rrvar = rrvar+radarParams.RMSBias(3)^2;
% Assemble object detections for use by tracker
numDets = numel(rngest);
dets = cell(numDets,1);
for iDet = 1:numDets
dets{iDet} = objectDetection(time, [azest(iDet) rngest(iDet) rrest(iDet)]', ...
'MeasurementNoise', diag([azvar(iDet) rngvar(iDet) rrvar(iDet)]), ...
'MeasurementParameters', {radarParams}, ...
'ObjectAttributes', {struct('SNR',snrdB(iDet))});
end
% Track detections
tracks = tracker(dets,time);
% Update bird's-eye plot
helperAutoDrivingRadarSigProc('Update Display', egoCar, dets, tracks, dopgrid,
rnggrid,...
```

```
Xbf, beamscan, Xrngdop);
% Publish snapshot
% helperAutoDrivingRadarSigProc('Publish Snapshot', time>=1.1);
% Collect channel metrics
metrics = helperAutoDrivingRadarSigProc('Collect Metrics', radarParams, tgtPos,
tgtVel, dets);
end
end
function metrics = runSimulation2ray(c,fc,rangeMax,vMax,Nsweep,...
rngdopresp,beamformer,cfar,idxCFAR,clusterer,...
rngestimator,dopestimator,doaest,beamscan,tracker,radar,sig)
rng(2017);
reset(rngestimator);
reset(dopestimator);
reset(doaest);
reset(beamscan);
reset(clusterer);
reset(tracker);
release(radar);
[scenario,egoCar,radarParams] = helperAutoDrivingRadarSigProc('Setup
Scenario',c,fc);
Nft = radar.Waveform.SweepTime*radar.Waveform.SampleRate;
Ne = radar.ReceiveAntenna.Sensor.NumElements;
Nr = rngdopresp.RangeFFTLength;
Nd = rngdopresp.DopplerFFTLength;
rxArray = radar.ReceiveAntenna.Sensor;
% Initialize display for driving scenario example
helperAutoDrivingRadarSigProc('Initialize Display', egoCar, radarParams, rxArray,
fc, vMax, rangeMax);
tgtProfiles = actorProfiles(scenario);
tgtProfiles = tgtProfiles(2:end);
tgtHeight = [tgtProfiles.Height];
sweepTime = radar.Waveform.SweepTime;
while advance(scenario)
% Get the current scenario time
time = scenario.SimulationTime;
% Get current target poses in ego car's reference frame
tgtPoses = targetPoses(egoCar);
tgtPos = reshape([tgtPoses.Position],3,[]);
tgtPos(3,:) = tgtPos(3,:)+0.5*tgtHeight; % Point targets are positioned at half of
each target's height
tgtVel = reshape([tgtPoses.Velocity],3,[]);
% Simulate the radar transmit and receive operation
Xcube = zeros(Nft, Ne, Nsweep);
```

```
for m = 1:Nsweep
paths = phased.internal.twoRayPaths(radarParams.OriginPosition,...
tgtPos,radarParams.OriginVelocity,tgtVel,[10 10 10],c/fc);
rxsig = radar(paths,time+(m-1)*sweepTime);
% Dechirp the received signal
rxsig = dechirp(rxsig, sig);
% Save sweep to data cube
Xcube(:,:,m) = rxsig;
% Move targets forward for next sweep
tgtPos = tgtPos+tgtVel*sweepTime;
end
% Calculate the range-doppler response
[Xrngdop,rnggrid,dopgrid] = rngdopresp(Xcube);
% Beamform received data
Xbf = permute(Xrngdop,[1 3 2]);
Xbf = reshape(Xbf,Nr*Nd,Ne);
Xbf = beamformer(Xbf);
Xbf = reshape(Xbf,Nr,Nd);
% Detect targets
Xpow = abs(Xbf).^2;
[detidx,noisepwr] = cfar(Xpow,idxCFAR);
% Cluster detections
[~,clusterIDs] = clusterer(detidx.');
% Azimuth, range, and range rate measurements
[azest,azvar,snrdB] = helperAutoDrivingRadarSigProc('Estimate Angle', doaest,
conj(Xrngdop), Xbf, detidx, noisepwr, clusterIDs);
azvar = azvar+radarParams.RMSBias(1)^2;
[rngest,rngvar] = rngestimator(Xbf,rnggrid,detidx,noisepwr,clusterIDs);
rngvar = rngvar+radarParams.RMSBias(2)^2;
[rsest,rsvar] = dopestimator(Xbf,dopgrid,detidx,noisepwr,clusterIDs);
% Radial speed is estimated, but the tracker expects range rate.
% Convert radial speed to range rate.
rrest = -rsest;
rrvar = rsvar;
rrvar = rrvar+radarParams.RMSBias(3)^2;
% Assemble object detections for use by tracker
numDets = numel(rngest);
dets = cell(numDets,1);
for iDet = 1:numDets
dets{iDet} = objectDetection(time, [azest(iDet) rngest(iDet) rrest(iDet)]', ...
'MeasurementNoise', diag([azvar(iDet) rngvar(iDet) rrvar(iDet)]), ...
'MeasurementParameters', {radarParams}, ...
'ObjectAttributes', {struct('SNR',snrdB(iDet))});
end
```

```
% Track detections
tracks = tracker(dets,time);
% Update bird's-eye plot
helperAutoDrivingRadarSigProc('Update Display', egoCar, dets, tracks, dopgrid,
rnggrid,...
Xbf, beamscan, Xrngdop);
% Publish snapshot
helperAutoDrivingRadarSigProc('Publish Snapshot', time>=1.1&&time<1.15);
% Collect channel metrics
metrics = helperAutoDrivingRadarSigProc('Collect Metrics', radarParams, tgtPos,
tgtVel, dets);
end
end
function plotChannels(metricsFS, metrics2Ray)
hfig = findobj('Tag','Compare Propagation Channels');
if isempty(hfig)
hfig = figure('Name','Channel Propagation','Tag','Channel Propagation');
end
figure(hfig);
clf(hfig); hold on;
title('Propagation Channels');
xlabel('Measured Radar Range (m)');
ylabel('CFAR SNR Estimate (dB)');
grid on;
% Only plot metrics for long range car
[~,y] = pol2cart(metricsFS.AzimuthEst*pi/180,metricsFS.RangeEst);
iFndy = y<0;
% Only plot metrics for long range car
[~,z] = pol2cart(metrics2Ray.AzimuthEst*pi/180,metrics2Ray.RangeEst);
iFndz = z<0;
plot(metricsFS.RangeEst(iFndy),metricsFS.SNREst(iFndy),'bo',...
metrics2Ray.RangeEst(iFndz),metrics2Ray.SNREst(iFndz),'r^');
legend('Free space','Two-ray');
end
function metrics = collectMetrics(metrics, radarParams, tgtPos, tgtVel, dets)
% Collects metrics from the simulated driving scenario. Returns the current
% collected metrics.
if isempty(metrics)
metrics = struct( ...
'Position',[],...
'Velocity',[],...
'Azimuth',[],...
'Range',[],...
'RadialSpeed',[],...
```

```
'AzimuthEst',[],...
'AzimuthVar',[],...
'RangeEst',[],...
'RangeVar',[],...
'RadialSpeedEst',[],...
'RadialSpeedVar',[],...
'SNREst',[]);
end
% Compute ground truth metrics
gtspd = radialspeed(tgtPos,tgtVel);
[gtrng, gtang] = rangeangle(tgtPos, radarParams.OriginPosition,
radarParams.Orientation);
gtaz = gtang(1,:)';
% Measurements
numDets = numel(dets);
azest = NaN(numDets,1);
azvar = NaN(numDets,1);
rngest = NaN(numDets,1);
rngvar = NaN(numDets,1);
rsest = NaN(numDets,1);
rsvar = NaN(numDets,1);
snrdB = NaN(numDets,1);
for m = 1:numDets
azest(m) = dets{m}.Measurement(1);
azvar(m) = dets{m}.MeasurementNoise(1,1);
rngest(m) = dets{m}.Measurement(2);
rngvar(m) = dets{m}.MeasurementNoise(2,2);
rsest(m) = dets{m}.Measurement(3);
rsvar(m) = dets{m}.MeasurementNoise(3,3);
snrdB(m) = dets{m}.ObjectAttributes{1}.SNR;
end
metrics.Position = [metrics.Position; tgtPos];
metrics.Velocity = [metrics.Position; tgtVel];
metrics.Azimuth = [metrics.Azimuth; gtaz(:)];
metrics.Range = [metrics.Range; gtrng(:)];
metrics.RadialSpeed = [metrics.RadialSpeed; gtspd(:)];
metrics.AzimuthEst = [metrics.AzimuthEst; azest(:)];
metrics.AzimuthVar = [metrics.AzimuthVar; azvar(:)];
metrics.RangeEst = [metrics.RangeEst; rngest(:)];
metrics.RangeVar = [metrics.RangeVar; rngvar(:)];
metrics.RadialSpeedEst = [metrics.RadialSpeedEst; rsest(:)];
metrics.RadialSpeedVar = [metrics.RadialSpeedVar; rsvar(:)];
metrics.SNREst = [metrics.SNREst; snrdB(:)];
end
```

```
function publishSnapshot(fig, takeSnapshot)
% When publishing, function takes one snapshot of the figure fig when
% takeSnapshot is true and then closes the figure so that another snapshot
% will not be generated from it. Does nothing when not publishing.
if ishghandle(fig)
hasSnapshot = takeSnapshot && isPublishing();
if hasSnapshot
figure(fig);
fig.Visible = 'on';
snapnow;
close(fig);
end
end
end
% Helper functions
function egoCar = addCars(scenario,roadcenters,initialdist,lane,initialspd)
% Adds all of the cars defined for the driving scenario and returns the
% vehicle object corresponding to the ego vehicle.
waypoints = interpWaypointsFromRoadCenters(roadcenters);
egoCar = addCar(scenario,waypoints,initialdist(1),lane(1),initialspd(1));
numCars = numel(initialspd);
for m = 2:numCars
addCar(scenario,waypoints,initialdist(m),lane(m),initialspd(m));
end
end
function car = addCar(scenario,waypoints,initialdist,lane,initialspd)
% Adds a car and its trajectory to the driving scenario. Returns the added
% vehicle object.
pathlength = [0;cumsum(sqrt(sum(diff(waypoints,1,1).^2,2)))];
car = vehicle(scenario);
ind = pathlength>=initialdist;
waypoints = waypoints(ind,:);
vecs = diff(waypoints,1,1);
nm = sqrt(sum(vecs.^2,2));
vecs = vecs./nm;
vecs = vecs([1 1:end],:);
vecs = [-vecs(:,2) vecs(:,1)];
waypoints = waypoints+vecs*lane;
trajectory(car, waypoints, initialspd);
end
function tgts = createPointTargets(scenario,c,fc)
% Returns point targets modeled using the |phased.BackscatterRadarTarget|
% object. The point target models are loaded from the point target models
% stored in the driving scenario's actor profiles.
```

```
profiles = actorProfiles(scenario);
profiles = profiles(2:end); % First profile is the ego car, not a target
numTgts = numel(profiles);
azAngles = profiles(2).RCSAzimuthAngles;
elAngles = profiles(2).RCSElevationAngles;
rcsPatterns = NaN(numel(elAngles),numel(azAngles),numTgts);
for m = 1:numTgts
rcsPatterns(:,:,m) = db2mag(profiles(m).RCSPattern);
end
tgts = phased.BackscatterRadarTarget( ...
'RCSPattern', rcsPatterns, ...
'AzimuthAngles', azAngles, ...
'ElevationAngles', elAngles, ...
'PropagationSpeed', c,...
'OperatingFrequency', fc);
end
function tgtstruct = createTargetStruct(tgtpos,tgtvel)
ntgt = size(tgtpos,2);
tgtstruct = struct('Position',mat2cell(tgtpos(:).',1,repmat(3,1,ntgt)),...
'Velocity',mat2cell(tgtvel(:).',1,repmat(3,1,ntgt)),...
'Signature',{rcsSignature,rcsSignature,rcsSignature});
end
function waypoints = interpWaypointsFromRoadCenters(roadCenters)
% Returns waypoints corresponding to the interpolated road centers, where
% the interpolation between the road centers is performed by
% drivingScenario.
num = size(roadCenters,2);
s = drivingScenario;
road(s, roadCenters, 1e-6);
rb = roadBoundaries(s);
rb = rb{1};
waypoints = [];
while ~isempty(rb)
pt = rb(1,:);
d = sqrt(sum((rb-pt).^2,2));
ind = d<1e-4;
waypoints = [waypoints;mean(rb(ind,:),1)]; %#ok<AGROW>
rb(ind,:) = [];
end
waypoints = waypoints(:,1:num);
end
function clusterIDs = clusterDetections(detidx)
% clusterIDs = clusterDetections(dets) clusters detections from a
% range-Doppler image. Detections which occur at adjacent range and Doppler
```

```
% cells within the image are associated to a single detection cluster. Each
% detection cluster is assigned a unique cluster ID which is used to
% identify the detections assigned to that cluster.
%
% detidx is a 2-by-L matrix of detection indices. Each column of detidx
% identifies the range and Doppler cell in the range-Doppler image where a
% detection was found as [rngidx; dopidx].
%
% clusterIDs is a 1-by-L vector of cluster IDs assigned to each detection
% index in detidx.
% Number of detections found in the range-Doppler image
numDet = size(detidx,2);
clusterIDs = NaN(1,numDet);
iGd = ~isnan(detidx(1,:));
if any(iGd(:))
% Cluster adjacent points
iRng = detidx(1,iGd);
iDop = detidx(2,iGd);
iGroups = cell(1,0);
allCells = [iRng;iDop];
while ~all(isnan(allCells(1,:)))
% Select an unassigned range-Doppler cell and remove it from the set
iFnd = find(~isnan(allCells(1,:)),1);
thisCell = allCells(:,iFnd);
allCells(:,iFnd) = NaN;
% Find all cells adjacent to the selected cell and remove them from set
[iAdjCells,allCells] = findAdjacentCells(allCells,thisCell);
iGroups{end+1} = [iFnd iAdjCells]; %#ok<AGROW>
end
% Assign unique cluster IDs to each group of adjacent range-Doppler
% cells
ids = NaN(sum(iGd),1);
for m = 1:numel(iGroups)
ids(iGroups{m}) = m;
end
clusterIDs(iGd) = ids';
end
end
function [iAdjCells,allCells] = findAdjacentCells(allCells,thisCell)
% [iAdjCells,allCells] = findAdjacentCells(allCells,thisCell) finds the
% cells in the 2-by-L matrix of allCells that are adjacent to the 2-by-1
% vector for the current cell, thisCell. Both allCells and thisCell contain
% indices of the rows and columns of a matrix.
%
```

```
% iAdjCells is a 2-by-M matrix of cells in allCells that are adjacent to
% thisCell. M is less-than or equal-to L.
%
% allCells is returned as a 2-by-L matrix, where the adjacent cells
% returned in iAdjCells have been set to NaN.
% Find all cells adjacent to the current cell and remove it from the set
delta = allCells-repmat(thisCell,[1 size(allCells,2)]);
iAdjCells = find(all(abs(delta)<=2,1));
thisCell = allCells(:,iAdjCells);
allCells(:,iAdjCells) = NaN;
% Find call cells next to each of the adjacent cells that were just found
for m = 1:numel(iAdjCells)
[iAdjNext,allCells] = findAdjacentCells(allCells,thisCell(:,m));
iAdjCells = [iAdjCells iAdjNext]; %#ok<AGROW>
end
end
function [angest, angvar, snrdB] =
doaestimator(doa,Xrngdop,Xbf,detidx,noisepwr,clusterIDs)
% [angest, angvar, snrdB] =
doaestimator(doa,Xrngdop,Xbf,detidx,noisepwr,clusterIDs)
% returns the estimated angles for the detections generated from a linear
% phased array.
%
% doa is the phased.RootMUSICEstimator object used to estimate the
% direction-of-arrival of signals detected on a linear phased array.
%
% Xrngdop is a Nr-by-Ne-by-Nd date cube containing the range-Doppler
% processed data received by the Ne elements of the linear phased array.
%
% Xbf is the Nr-by-Nd beamformed range-Doppler image used to identify
% detections in the Xrngdop data cube.
%
% detidx is a 2-by-L matrix of detection indices generated from the
% beamformed range-Doppler image, Xbf. Each column of detidx identifies the
% range and Doppler cell in the range-Doppler image where a detection was
% found as [rngidx; dopidx].
%
% noisepwr is a 1-by-L vector of the estimated noise powers at each
% detection location reported in detidx.
%
% clusterIDs is a 1-by-L vector of cluster IDs assigned to each detection
% index in detidx.
%
% angest is an N-by-1 vector of angle estimates generated from the
```

```
% detection indices and clusterIDs. If clusterIDs is not provided, each
% detection is assigned its own angle estimate. Angle estimates are
% reported in degrees.
%
% angvar is an N-by-1 vector of variances corresponding to each of the
% angle estimates reported in angest. The angle variances are reported in
% degrees-squared (deg^2).
%
% snrdB is an N-by-1 vector of the signal-to-noise ratios (SNR) estimated
% of each detection in detidx. SNR is reported in decibels (dB).
persistent bw
if isempty(bw)
bw = arraybeamwidth(doa);
end
% If not provided, assign each detection to its own cluster
numDet = size(detidx,2);
if nargin<4
clusterIDs = (1:numDet);
clusters = clusterIDs;
numEst = numDet;
else
clusters = unique(clusterIDs(~isnan(clusterIDs)));
numEst = length(clusters);
end
angest = NaN(numEst,1);
angvar = NaN(numEst,1);
snrdB = NaN(numEst,1);
Nr = size(Xrngdop,1);
hasDoppler = ndims(Xrngdop)==3;
if hasDoppler
Xrngdop = permute(Xrngdop,[1 3 2]); % Nr x Nd x Ne
Nd = size(Xrngdop,2);
Nelmnt = size(Xrngdop,3);
else
Nd = 1;
Nelmnt = size(Xrngdop,2);
end
Xrngdop = reshape(Xrngdop,[],Nelmnt);
for m = 1:numEst
% Select detections in this cluster
thisCluster = clusterIDs==clusters(m);
iRng = detidx(1,thisCluster);
iDop = detidx(2,thisCluster);
idx = sub2ind([Nr Nd],iRng,iDop);
```

```
y = Xrngdop(idx,:);
angest(m) = doa(y);
[pow,iMax] = max(abs(Xbf(idx)).^2);
noise = noisepwr(thisCluster);
snr = pow/noise(iMax);
sigma = bw/(1.6*sqrt(2*snr));
angvar(m) = sigma.^2;
snrdB(m) = pow2db(snr);
end
end
function bw = arraybeamwidth(doa,c,fc)
% bw = arraybeamwidth(doa) returns the half-power (3dB) beamwidth estimated
% from the linear phased array used by doa.
%
% doa is the phased.RootMUSICEstimator object used to estimate the
% direction-of-arrival of signals detected on a linear phased array.
%
% bw is the estimated half power beamwidth in degrees.
if isprop(doa,'SensorArray')
array = doa.SensorArray;
fc = doa.OperatingFrequency;
c = doa.PropagationSpeed;
else
array = doa;
end
ang = linspace(-90,90);
pat =
patternAzimuth(array,fc,0,'Type','powerdb','PropagationSpeed',c,'Azimuth',ang);
pk = max(pat);
iPk1 = find(pat>=pk-3,1,'first');
iPk2 = find(pat>=pk-3,1,'last');
bw = abs(ang(iPk2)-ang(iPk1));
end
function tf = isPublishing
% Returns true if the example is currently publishing
tf = false;
try
s = numel(dbstack);
tf = s > 3;
catch
end
end
```

*Appendix B*

# SYSTEM VERILOG CODES

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 03/12/2025 02:18:59 PM
// Design Name:
// Module Name: FFT_Processor_Chain
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


(* keep_hierarchy = "yes" *)
module FFT_Processor_Chain #(
    parameter int Width = 16
)(
    input logic clk,
    input logic reset,
    input logic [Width-1:0] data_in_real,
    input logic [Width-1:0] data_in_imag,

    input logic read_en_detectmem,
    output logic [$clog2(256)-1:0] range_bin_out,
    output logic [$clog2(128)-1:0] doppler_bin_out
```

```
);

    // === Wires and Signals ===

    // Pre-Range FFT Buffers
    logic is_empty_prerangefftRAM_real, is_full_prerangefftRAM_real;
    logic is_empty_prerangefftRAM_imag, is_full_prerangefftRAM_imag;
    logic write_en_prerangefftRAM_real, read_en_prerangefftRAM_real;
    logic write_en_prerangefftRAM_imag, read_en_prerangefftRAM_imag;
    logic [Width-1:0] buffer_out_real, buffer_out_imag;

    // DFT-256
    logic signed [Width-1:0] dft256_out_real, dft256_out_imag;
    logic load_en_dft256, ctrl_flag_dft256;
    logic dft256_load_done, dft256_compute_done;
    logic dft256_out_valid;

    // Ping-Pong Buffer
    logic pingpong_full;
    logic active_buffer_sel;
    logic load_en_pingpong;
    logic [Width-1:0] pingpong_out_real, pingpong_out_imag;
    logic read_en_pingpong;

    // DFT-128
    logic signed [Width-1:0] dft128_out_real, dft128_out_imag;
    logic load_en_dft128, ctrl_flag_dft128;
    logic dft128_load_done, dft128_compute_done;
    logic dft128_out_valid;

    // Square Law
    logic signed [Width-1:0] squarelaw_out;
    logic enable_squarelaw;

    // CFAR
    logic enable_cfar;
    logic detection_valid;
    logic [7:0] detection_range_bin;
    logic [6:0] detection_doppler_bin;
    logic cfar_done;
```

```
// Detection Memory
logic detection_available, memory_full;
logic [7:0] range_bin_out_t;
logic [6:0] doppler_bin_out_t;

// === MODULES ===

// --- Control Unit
control_unit CU (
    .clk(clk),
    .reset(reset),
    .is_empty_prerangefftRAM_real(is_empty_prerangefftRAM_real),
    .is_full_prerangefftRAM_real(is_full_prerangefftRAM_real),
    .is_empty_prerangefftRAM_imag(is_empty_prerangefftRAM_imag),
    .is_full_prerangefftRAM_imag(is_full_prerangefftRAM_imag),
    .dft256_load_done(dft256_load_done),
    .dft256_compute_done(dft256_compute_done),
    .pingpong_full(pingpong_full),
    .dft128_load_done(dft128_load_done),
    .dft128_compute_done(dft128_compute_done),
    .cfar_done(cfar_done),
    .write_en_prerangefftRAM_real(write_en_prerangefftRAM_real),
    .read_en_prerangefftRAM_real(read_en_prerangefftRAM_real),
    .write_en_prerangefftRAM_imag(write_en_prerangefftRAM_imag),
    .read_en_prerangefftRAM_imag(read_en_prerangefftRAM_imag),
    .load_en_dft256(load_en_dft256),
    .ctrl_flag_dft256(ctrl_flag_dft256),
    .load_en_dft128(load_en_dft128),
    .ctrl_flag_dft128(ctrl_flag_dft128),
    .load_en_pingpong(load_en_pingpong),
    .enable_squarelaw(enable_squarelaw),
    .enable_cfar(enable_cfar),
    .read_en_pingpong(read_en_pingpong)
);

// --- Pre-Range FFT Buffers
buffer_16x256 preRangeFFTBuffer1 (
    .clk(clk),
    .reset(reset),
    .read_en(read_en_prerangefftRAM_real),
```

```
        .write_en(write_en_prerangefftRAM_real),
        .data_in(data_in_real),
        .data_out(buffer_out_real),
        .is_empty(is_empty_prerangefftRAM_real),
        .is_full(is_full_prerangefftRAM_real)
    );

    buffer_16x256 preRangeFFTBuffer2 (
        .clk(clk),
        .reset(reset),
        .read_en(read_en_prerangefftRAM_imag),
        .write_en(write_en_prerangefftRAM_imag),
        .data_in(data_in_imag),
        .data_out(buffer_out_imag),
        .is_empty(is_empty_prerangefftRAM_imag),
        .is_full(is_full_prerangefftRAM_imag)
    );

    // --- DFT-256
    dft_256pt_streaming dft256 (
        .clk(clk),
        .reset(reset),
        .load_en(load_en_dft256),
        .ctrl_flag(ctrl_flag_dft256),
        .in_real(buffer_out_real),
        .in_imag(buffer_out_imag),
        .out_real(dft256_out_real),
        .out_imag(dft256_out_imag),
        .out_valid(dft256_out_valid),
        .ram_full(),
        .ram_empty(),
        .dft_load_done(dft256_load_done),
        .dft_compute_done(dft256_compute_done)
    );

    // --- Ping-Pong Buffer
    ping_pong_buffer #(
        .WIDTH(Width),
        .RANGE(256),
        .DOPPLER(128)
    ) pingpong (
```

```
        .clk(clk),
        .reset(reset),
        .write_en(dft256_out_valid & load_en_pingpong),
        .data_in_real(dft256_out_real),
        .data_in_imag(dft256_out_imag),
        .read_en(read_en_pingpong),
        .data_out_real(pingpong_out_real),
        .data_out_imag(pingpong_out_imag),
        .buffer_full(pingpong_full),
        .active_buffer_sel(active_buffer_sel)
    );

    // --- DFT-128
    dft_128pt_streaming dft128 (
        .clk(clk),
        .reset(reset),
        .load_en(load_en_dft128),
        .ctrl_flag(ctrl_flag_dft128),
        .in_real(pingpong_out_real),
        .in_imag(pingpong_out_imag),
        .out_real(dft128_out_real),
        .out_imag(dft128_out_imag),
        .out_valid(dft128_out_valid),
        .ram_full(),
        .ram_empty(),
        .dft_load_done(dft128_load_done),
        .dft_compute_done(dft128_compute_done)
    );

    // --- Square Law
    square_law sqw (
        .reset(reset),
        .in_real(dft128_out_real),
        .in_imag(dft128_out_imag),
        .sqw_real_out(squarelaw_out)
    );

    // --- CFAR
    cfar CFAR_inst (
        .clk(clk),
        .reset(reset),
```

```
        .enable(enable_cfar),
        .data_in(squarelaw_out),
        .detection_valid(detection_valid),
        .range_bin(detection_range_bin),
        .doppler_bin(detection_doppler_bin)
    );

    // --- Detection Memory
    detection_memory detect_mem (
        .clk(clk),
        .reset(reset),
        .detection_valid(detection_valid),
        .range_bin_in(detection_range_bin),
        .doppler_bin_in(detection_doppler_bin),
        .read_en(read_en_detectmem),
        .range_bin_out(range_bin_out_t),
        .doppler_bin_out(doppler_bin_out_t),
        .detection_available(detection_available),
        .memory_full(memory_full)
    );

    assign range_bin_out = range_bin_out_t;
    assign doppler_bin_out = doppler_bin_out_t;

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 03/12/2025 02:44:54 PM
// Design Name:
// Module Name: buffer_16x256
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
```

```
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module buffer_16x256 #(
    parameter int WIDTH = 16,
    parameter int DEPTH = 256,
    parameter int ADDRESSRANGE = $clog2(DEPTH)
)(
    input logic clk,
    input logic reset,
    input logic read_en,
    input logic write_en,
    input logic [WIDTH-1:0] data_in,
    output logic is_empty,
    output logic is_full,
    output logic [WIDTH-1:0] data_out
);

    logic [WIDTH-1:0] buffers [0:DEPTH-1];
    logic [ADDRESSRANGE:0] address_counter;

    // Proper synchronous reset handling
    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
            foreach (buffers[i]) buffers[i] <= 0;
            address_counter <= 0;
            is_empty <= 1;
            is_full <= 0;
        end
        else begin
            if (write_en) begin
                buffers[address_counter] <= data_in;
                address_counter <= address_counter + 1;
```

```
                if (address_counter == DEPTH-1) begin
                    is_empty <= 0;
                    is_full <= 1;
                    address_counter <= 0;
                end
            end

            if (read_en) begin
                data_out <= buffers[address_counter];
                address_counter <= address_counter + 1;

                if (address_counter == DEPTH-1) begin
                    is_empty <= 1;
                    is_full <= 0;
                    address_counter <= 0;
                end
            end
        end
    end

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 04/23/2025 08:08:08 PM
// Design Name:
// Module Name: dft_256pt_streaming
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
```

```
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
////////////



module dft_256pt_streaming #(
    parameter int Width = 16
)(
    input  logic clk,
    input  logic reset,

    // Input sample interface
    input  logic load_en,
    input  logic signed [Width-1:0] in_real,
    input  logic signed [Width-1:0] in_imag,

    // Control signal to trigger DFT
    input  logic ctrl_flag,

    // Output interface
    output logic signed [Width-1:0] out_real,
    output logic signed [Width-1:0] out_imag,
    output logic out_valid,

    // Status flags
    output logic ram_full,
    output logic ram_empty,

    // Integration flags
    output logic dft_load_done,
    output logic dft_compute_done
);

    // === Internal Buffers ===
    logic signed [Width-1:0] buffer_real [0:255];
    logic signed [Width-1:0] buffer_imag [0:255];
    logic signed [Width-1:0] result_real [0:255];
    logic signed [Width-1:0] result_imag [0:255];
```

```
        // ROM: DFT coefficients in Q15
    logic signed [Width-2:0] W_real [0:255][0:255] = '{
        '{  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,
```

```
17530, -18204, -18868, -19519, -20159, -20787, -21403, -22005, -22594,
-23170, -23731, -24279, -24811, -25329, -25832, -26319, -26790, -
27245, -27683, -28105, -28510, -28898, -29268, -29621, -29956, -30273,
-30571, -30852, -31113, -31356, -31580, -31785, -31971, -32137, -
32285, -32412, -32521, -32609, -32678, -32728, -32757, -32767, -32757,
-32728, -32678, -32609, -32521, -32412, -32285, -32137, -31971, -
31785, -31580, -31356, -31113, -30852, -30571, -30273, -29956, -29621,
-29268, -28898, -28510, -28105, -27683, -27245, -26790, -26319, -
25832, -25329, -24811, -24279, -23731, -23170, -22594, -22005, -21403,
-20787, -20159, -19519, -18868, -18204, -17530, -16846, -16151, -
15446, -14732, -14010, -13279, -12539, -11793, -11039, -10278,  -9512,
-8739,  -7962,  -7179,  -6393,  -5602,  -4808,  -4011,  -3212,  -2410,
-1608,   -804,      0,    804,   1608,   2410,   3212,   4011,   4808,
5602,   6393,   7179,   7962,   8739,   9512,  10278,  11039,  11793,
12539,  13279,  14010,  14732,  15446,  16151,  16846,  17530,  18204,
18868,  19519,  20159,  20787,  21403,  22005,  22594,  23170,  23731,
24279,  24811,  25329,  25832,  26319,  26790,  27245,  27683,  28105,
28510,  28898,  29268,  29621,  29956,  30273,  30571,  30852,  31113,
31356,  31580,  31785,  31971,  32137,  32285,  32412,  32521,  32609,
32678,  32728,  32757 }
    };

    logic signed [Width-2:0] W_imag [0:255][0:255] = '{
        '{      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,
```

```
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0 },
        ...
        '{      0,    804,   1608,   2410,   3212,   4011,   4808,
5602,   6393,   7179,   7962,   8739,   9512,  10278,  11039,  11793,
12539,  13279,  14010,  14732,  15446,  16151,  16846,  17530,  18204,
18868,  19519,  20159,  20787,  21403,  22005,  22594,  23170,  23731,
24279,  24811,  25329,  25832,  26319,  26790,  27245,  27683,  28105,
28510,  28898,  29268,  29621,  29956,  30273,  30571,  30852,  31113,
31356,  31580,  31785,  31971,  32137,  32285,  32412,  32521,  32609,
32678,  32728,  32757,  32767,  32757,  32728,  32678,  32609,  32521,
32412,  32285,  32137,  31971,  31785,  31580,  31356,  31113,  30852,
30571,  30273,  29956,  29621,  29268,  28898,  28510,  28105,  27683,
27245,  26790,  26319,  25832,  25329,  24811,  24279,  23731,  23170,
22594,  22005,  21403,  20787,  20159,  19519,  18868,  18204,  17530,
16846,  16151,  15446,  14732,  14010,  13279,  12539,  11793,  11039,
10278,   9512,   8739,   7962,   7179,   6393,   5602,   4808,   4011,
3212,   2410,   1608,    804,      0,   -804,  -1608,  -2410,  -3212,
-4011,  -4808,  -5602,  -6393,  -7179,  -7962,  -8739,  -9512, -10278,
-11039, -11793, -12539, -13279, -14010, -14732, -15446, -16151, -
16846, -17530, -18204, -18868, -19519, -20159, -20787, -21403, -22005,
-22594, -23170, -23731, -24279, -24811, -25329, -25832, -26319, -
26790, -27245, -27683, -28105, -28510, -28898, -29268, -29621, -29956,
-30273, -30571, -30852, -31113, -31356, -31580, -31785, -31971, -
32137, -32285, -32412, -32521, -32609, -32678, -32728, -32757, -32767,
-32757, -32728, -32678, -32609, -32521, -32412, -32285, -32137, -
31971, -31785, -31580, -31356, -31113, -30852, -30571, -30273, -29956,
-29621, -29268, -28898, -28510, -28105, -27683, -27245, -26790, -
26319, -25832, -25329, -24811, -24279, -23731, -23170, -22594, -22005,
-21403, -20787, -20159, -19519, -18868, -18204, -17530, -16846, -
16151, -15446, -14732, -14010, -13279, -12539, -11793, -11039, -10278,
-9512,  -8739,  -7962,  -7179,  -6393,  -5602,  -4808,  -4011,  -3212,
-2410,  -1608,   -804 }
    };
```

```
// FSM states
typedef enum logic [1:0] {IDLE, LOAD, COMPUTE, STREAM} state_t;
state_t state;

logic [7:0] write_ptr, read_ptr, k, n;

logic signed [2*Width+8:0] acc_real, acc_imag;
logic signed [2*Width+8:0] mult_re, mult_im;

// === Status Outputs ===
assign ram_full         = (write_ptr == 8'd255 && load_en);
assign ram_empty        = (write_ptr == 0 && ~load_en);
assign out_valid        = (state == STREAM);
assign dft_load_done    = (state == LOAD && write_ptr == 8'd255);
assign dft_compute_done = (state == STREAM && read_ptr == 8'd255);

// === Sample Loading ===
always_ff @(posedge clk or posedge reset) begin
    if (reset)
        write_ptr <= 0;
    else if (load_en && state == LOAD && write_ptr < 9'd256) begin
        buffer_real[write_ptr] <= in_real;
        buffer_imag[write_ptr] <= in_imag;
        write_ptr <= write_ptr + 1;
    end
end

// === FSM Logic ===
always_ff @(posedge clk or posedge reset) begin
if (reset) begin
    state     <= IDLE;
    write_ptr <= 0;
    k         <= 0;
    n         <= 0;
    read_ptr  <= 0;
    acc_real  <= 0;
    acc_imag  <= 0;
    out_real  <= 0;
    out_imag  <= 0;
end else begin
```

```
        case (state)

            IDLE: begin
                if (ctrl_flag) begin
                    state <= COMPUTE;
                    k <= 0;
                    n <= 0;
                    acc_real <= 0;
                    acc_imag <= 0;
                end else if (load_en) begin
                    state <= LOAD;
                    write_ptr <= 0; // reset write pointer for new
load
                end
            end

            LOAD: begin
                if (load_en) begin
                    buffer_real[write_ptr] <= in_real;
                    buffer_imag[write_ptr] <= in_imag;
                    write_ptr <= write_ptr + 1;
                    if (write_ptr == 8'd255)
                        state <= IDLE; // move to IDLE after loading
                end
            end

            COMPUTE: begin
                mult_re = buffer_real[n] * W_real[k][n] -
buffer_imag[n] * W_imag[k][n];
                mult_im = buffer_real[n] * W_imag[k][n] +
buffer_imag[n] * W_real[k][n];

                acc_real <= acc_real + mult_re;
                acc_imag <= acc_imag + mult_im;

                if (n == 8'd255) begin
                    result_real[k] <= acc_real[30:15];
                    result_imag[k] <= acc_imag[30:15];

                    if (k == 8'd255) begin
                        state <= STREAM;
```

```
                        read_ptr <= 0;
                    end else begin
                        k <= k + 1;
                        n <= 0;
                        acc_real <= 0;
                        acc_imag <= 0;
                    end
                end else begin
                    n <= n + 1;
                end
            end

            STREAM: begin
                out_real <= result_real[read_ptr];
                out_imag <= result_imag[read_ptr];
                if (read_ptr == 8'd255)
                    state <= IDLE;
                else
                    read_ptr <= read_ptr + 1;
            end

            default: state <= IDLE;
        endcase
    end
end


endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 04/25/2025 12:05:12 AM
// Design Name:
// Module Name: ping_pong_buffer
// Project Name:
// Target Devices:
```

```
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module ping_pong_buffer #(
    parameter int WIDTH = 16,
    parameter int RANGE = 256,
    parameter int DOPPLER = 128,
    parameter int DEPTH = RANGE * DOPPLER,
    parameter int ADDR_WIDTH = $clog2(DEPTH)
)(
    input  logic clk,
    input  logic reset,

    // Write interface
    input  logic write_en,
    input  logic [WIDTH-1:0] data_in_real,
    input  logic [WIDTH-1:0] data_in_imag,

    // Read interface
    input  logic read_en,
    output logic [WIDTH-1:0] data_out_real,
    output logic [WIDTH-1:0] data_out_imag,

    output logic buffer_full,
    output logic active_buffer_sel
);

    // Internal signals
    logic [ADDR_WIDTH-1:0] write_count;
    logic [ADDR_WIDTH-1:0] read_count;
    logic buffer_filled;
```

```
    logic current_buffer; // 0 = ping active write, pong available
read

    logic [WIDTH-1:0] ping_real_rd;
    logic [WIDTH-1:0] ping_imag_rd;
    logic [WIDTH-1:0] pong_real_rd;
    logic [WIDTH-1:0] pong_imag_rd;

    // --- Write pointer ---
    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
            write_count <= 0;
            buffer_filled <= 0;
            current_buffer <= 0;
        end else if (write_en) begin
            if (write_count == (DEPTH-1)) begin
                buffer_filled <= 1;
                current_buffer <= ~current_buffer;
                write_count <= 0;
            end else begin
                write_count <= write_count + 1;
            end
        end
    end

    assign buffer_full = buffer_filled;
    assign active_buffer_sel = current_buffer;

    // --- Read pointer ---
    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
            read_count <= 0;
        end else if (read_en) begin
            if (read_count == (DEPTH-1))
                read_count <= 0;
            else
                read_count <= read_count + 1;
        end
    end

    // --- RAMs ---
```

```
    simple_dual_port_ram #(
        .WIDTH(WIDTH),
        .DEPTH(DEPTH)
    ) ping_real_ram (
        .clk(clk),
        .wr_addr(write_count),
        .wr_data(data_in_real),
        .wr_en(write_en & (current_buffer == 0)),
        .rd_addr(read_count),
        .rd_data(ping_real_rd)
    );

    simple_dual_port_ram #(
        .WIDTH(WIDTH),
        .DEPTH(DEPTH)
    ) ping_imag_ram (
        .clk(clk),
        .wr_addr(write_count),
        .wr_data(data_in_imag),
        .wr_en(write_en & (current_buffer == 0)),
        .rd_addr(read_count),
        .rd_data(ping_imag_rd)
    );

    simple_dual_port_ram #(
        .WIDTH(WIDTH),
        .DEPTH(DEPTH)
    ) pong_real_ram (
        .clk(clk),
        .wr_addr(write_count),
        .wr_data(data_in_real),
        .wr_en(write_en & (current_buffer == 1)),
        .rd_addr(read_count),
        .rd_data(pong_real_rd)
    );

    simple_dual_port_ram #(
        .WIDTH(WIDTH),
        .DEPTH(DEPTH)
    ) pong_imag_ram (
        .clk(clk),
```

```
        .wr_addr(write_count),
        .wr_data(data_in_imag),
        .wr_en(write_en & (current_buffer == 1)),
        .rd_addr(read_count),
        .rd_data(pong_imag_rd)
    );

    // --- Output selection ---
    always_comb begin
        if (current_buffer == 0) begin
            data_out_real = pong_real_rd;
            data_out_imag = pong_imag_rd;
        end else begin
            data_out_real = ping_real_rd;
            data_out_imag = ping_imag_rd;
        end
    end

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 04/25/2025 12:23:21 AM
// Design Name:
// Module Name: dft_128pt_streaming
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
```

```
////////////////////////////////////////////////////////////////////////
////////////


module dft_128pt_streaming #(
    parameter int Width = 16
)(
    input  logic clk,
    input  logic reset,

    // Input sample interface
    input  logic load_en,
    input  logic signed [Width-1:0] in_real,
    input  logic signed [Width-1:0] in_imag,

    // Control signal to trigger DFT
    input  logic ctrl_flag,

    // Output interface
    output logic signed [Width-1:0] out_real,
    output logic signed [Width-1:0] out_imag,
    output logic out_valid,

    // Status flags
    output logic ram_full,
    output logic ram_empty,

    // Integration flags
    output logic dft_load_done,
    output logic dft_compute_done
);

    // === Internal Buffers ===
    logic signed [Width-1:0] buffer_real [0:127]; // 128 points for
real
    logic signed [Width-1:0] buffer_imag [0:127]; // 128 points for
imaginary
    logic signed [Width-1:0] result_real [0:127]; // 128 points for
real results
```

```
    logic signed [Width-1:0] result_imag [0:127]; // 128 points for
imaginary results

    // ROM: DFT coefficients in Q15 for 128-point DFT
    logic signed [Width-1:0] W_real [0:127][0:127] = '{
        '{  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,  32767,
32767,  32767,  32767,  32767 },
        ...
        '{  32767,  32728,  32609,  32412,  32137,  31785,  31356,
30852,  30273,  29621,  28898,  28105,  27245,  26319,  25329,  24279,
23170,  22005,  20787,  19519,  18204,  16846,  15446,  14010,  12539,
11039,   9512,   7962,   6393,   4808,   3212,   1608,      0,  -1608,
-3212,  -4808,  -6393,  -7962,  -9512, -11039, -12539, -14010, -15446,
-16846, -18204, -19519, -20787, -22005, -23170, -24279, -25329, -
26319, -27245, -28105, -28898, -29621, -30273, -30852, -31356, -31785,
-32137, -32412, -32609, -32728, -32767, -32728, -32609, -32412, -
32137, -31785, -31356, -30852, -30273, -29621, -28898, -28105, -27245,
-26319, -25329, -24279, -23170, -22005, -20787, -19519, -18204, -
16846, -15446, -14010, -12539, -11039,  -9512,  -7962,  -6393,  -4808,
-3212,  -1608,      0,   1608,   3212,   4808,   6393,   7962,   9512,
11039,  12539,  14010,  15446,  16846,  18204,  19519,  20787,  22005,
23170,  24279,  25329,  26319,  27245,  28105,  28898,  29621,  30273,
30852,  31356,  31785,  32137,  32412,  32609,  32728 }
    };
    logic signed [Width-1:0] W_imag [0:127][0:127] = '{
        '{      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
```

```
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0,      0,      0,      0,      0,      0,
0,      0,      0,      0 },
        ...
        '{      0,   1608,   3212,   4808,   6393,   7962,   9512,
11039,  12539,  14010,  15446,  16846,  18204,  19519,  20787,  22005,
23170,  24279,  25329,  26319,  27245,  28105,  28898,  29621,  30273,
30852,  31356,  31785,  32137,  32412,  32609,  32728,  32767,  32728,
32609,  32412,  32137,  31785,  31356,  30852,  30273,  29621,  28898,
28105,  27245,  26319,  25329,  24279,  23170,  22005,  20787,  19519,
18204,  16846,  15446,  14010,  12539,  11039,   9512,   7962,   6393,
4808,   3212,   1608,      0,  -1608,  -3212,  -4808,  -6393,  -7962,
-9512, -11039, -12539, -14010, -15446, -16846, -18204, -19519, -20787,
-22005, -23170, -24279, -25329, -26319, -27245, -28105, -28898, -
29621, -30273, -30852, -31356, -31785, -32137, -32412, -32609, -32728,
-32767, -32728, -32609, -32412, -32137, -31785, -31356, -30852, -
30273, -29621, -28898, -28105, -27245, -26319, -25329, -24279, -23170,
-22005, -20787, -19519, -18204, -16846, -15446, -14010, -12539, -
11039,  -9512,  -7962,  -6393,  -4808,  -3212,  -1608 }
    };

    // FSM states
    typedef enum logic [1:0] {IDLE, LOAD, COMPUTE, STREAM} state_t;
    state_t state;

    logic [7:0] write_ptr, read_ptr, k, n;

    logic signed [2*Width+8:0] acc_real, acc_imag;
    logic signed [2*Width+8:0] mult_re, mult_im;

    // === Status Outputs ===
    assign ram_full         = (write_ptr == 8'd127 && load_en); //
Update with 128-point
```

```
    assign ram_empty        = (write_ptr == 0 && ~load_en);
    assign out_valid        = (state == STREAM);
    assign dft_load_done    = (state == LOAD && write_ptr == 8'd127);
// Update with 128-point
    assign dft_compute_done = (state == STREAM && read_ptr == 8'd127);
// Update with 128-point

    // === Sample Loading ===
    always_ff @(posedge clk or posedge reset) begin
    if (reset) begin
        state     <= IDLE;
        write_ptr <= 0;
        k         <= 0;
        n         <= 0;
        read_ptr  <= 0;
        acc_real  <= 0;
        acc_imag  <= 0;
        out_real  <= 0;
        out_imag  <= 0;
    end else begin
        case (state)

            IDLE: begin
                if (ctrl_flag) begin
                    state <= COMPUTE;
                    k <= 0;
                    n <= 0;
                    acc_real <= 0;
                    acc_imag <= 0;
                end else if (load_en) begin
                    state <= LOAD;
                    write_ptr <= 0; // reset write pointer for new
load
                end
            end

            LOAD: begin
                if (load_en) begin
                    buffer_real[write_ptr] <= in_real;
                    buffer_imag[write_ptr] <= in_imag;
                    write_ptr <= write_ptr + 1;
```

```
                    if (write_ptr == 8'd127)
                        state <= IDLE; // move to IDLE after loading
                end
            end

            COMPUTE: begin
                mult_re = buffer_real[n] * W_real[k][n] -
buffer_imag[n] * W_imag[k][n];
                mult_im = buffer_real[n] * W_imag[k][n] +
buffer_imag[n] * W_real[k][n];

                acc_real <= acc_real + mult_re;
                acc_imag <= acc_imag + mult_im;

                if (n == 8'd127) begin
                    result_real[k] <= acc_real[30:15];
                    result_imag[k] <= acc_imag[30:15];

                    if (k == 8'd127) begin
                        state <= STREAM;
                        read_ptr <= 0;
                    end else begin
                        k <= k + 1;
                        n <= 0;
                        acc_real <= 0;
                        acc_imag <= 0;
                    end
                end else begin
                    n <= n + 1;
                end
            end

            STREAM: begin
                out_real <= result_real[read_ptr];
                out_imag <= result_imag[read_ptr];
                if (read_ptr == 8'd127)
                    state <= IDLE;
                else
                    read_ptr <= read_ptr + 1;
            end
```

```
            default: state <= IDLE;
        endcase
    end
end



endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 04/25/2025 05:08:36 PM
// Design Name:
// Module Name: square_law
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
////////////



module square_law #(
    parameter int Width = 16
)(
    input logic clk,
    input logic reset,
    input logic signed [Width-1:0] in_real,
    input logic signed [Width-1:0] in_imag,
    output logic signed [Width-1:0] sqw_real_out
```

```
);

    logic signed [2*Width-1:0] real_squared;
    logic signed [2*Width-1:0] imag_squared;
    logic signed [2*Width-1:0] magnitude_squared;

    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
            sqw_real_out <= 0;
        end else begin
            real_squared <= in_real * in_real;
            imag_squared <= in_imag * in_imag;
            magnitude_squared <= real_squared + imag_squared;
            sqw_real_out <= magnitude_squared[2*Width-1:Width]; //
Scale down
        end
    end

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////
////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 04/26/2025 10:14:36 PM
// Design Name:
// Module Name: cfar
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
```

```
////////////////////////////////////////////////////////////////////////////
////////////



module cfar #(
    parameter int WIDTH = 16,
    parameter int RANGE = 256,
    parameter int DOPPLER = 128,
    parameter int T = 4,
    parameter int G = 2,
    parameter logic [15:0] THRESH_SCALE = 16'd19661
)(
    input logic clk,
    input logic reset,
    input logic enable,

    input logic signed [WIDTH-1:0] data_in,   // From ping-pong buffer

    output logic detection_valid,
    output logic [$clog2(RANGE)-1:0] range_bin,
    output logic [$clog2(DOPPLER)-1:0] doppler_bin,
    output logic cfar_done  // <<== Added
);

    // === Internal RAM ===
    logic [$clog2(RANGE*DOPPLER)-1:0] wr_addr;
    logic [$clog2(RANGE*DOPPLER)-1:0] rd_addr;
    logic wr_en;
    logic signed [WIDTH-1:0] rd_data;

    simple_dual_port_ram #(
        .WIDTH(WIDTH),
        .DEPTH(RANGE*DOPPLER)
    ) buffer_mem (
        .clk(clk),
        .wr_addr(wr_addr),
        .wr_data(data_in),
        .wr_en(wr_en),
        .rd_addr(rd_addr),
        .rd_data(rd_data)
    );
```

```
    typedef enum logic [1:0] {IDLE, LOAD, PROCESS} state_t;
    state_t state, next_state;

    logic [$clog2(RANGE)-1:0] proc_row;
    logic [$clog2(DOPPLER)-1:0] proc_col;
    logic signed [2*WIDTH+15:0] train_sum;
    logic signed [WIDTH-1:0] cut_value;
    logic [8:0] read_count;
    logic [15:0] threshold;
    logic [7:0] total_training_cells;

    int row_offset, col_offset;
    logic [$clog2(RANGE*DOPPLER)-1:0] calc_addr;

    function automatic int get_flat_addr(input int row, input int
col);
        return (row * DOPPLER) + col;
    endfunction

    always_ff @(posedge clk or posedge reset) begin
        if (reset)
            state <= IDLE;
        else
            state <= next_state;
    end

    always_comb begin
        next_state = state;
        case (state)
            IDLE:    if (enable) next_state = LOAD;
            LOAD:    if (wr_addr == RANGE*DOPPLER) next_state =
PROCESS;
            PROCESS: if (proc_row == RANGE-1 && proc_col == DOPPLER-1
&& read_count == total_training_cells) next_state = IDLE;
            default: next_state = IDLE;
        endcase
    end

    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
```

```
            wr_addr <= 0;
        end else if (state == LOAD) begin
            wr_en <= 1;
            wr_addr <= wr_addr + 1;
        end else begin
            wr_en <= 0;
        end
    end

    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
            proc_row <= 0;
            proc_col <= 0;
            read_count <= 0;
            train_sum <= 0;
            detection_valid <= 0;
        end else if (state == PROCESS) begin
            detection_valid <= 0;

            if (proc_row >= (T+G) && proc_row < (RANGE-(T+G)) &&
                proc_col >= (T+G) && proc_col < (DOPPLER-(T+G))) begin

                if (read_count == 0) begin
                    train_sum <= 0;
                end

                row_offset = ((read_count / (2*(T+G)+1)) - (T+G));
                col_offset = ((read_count % (2*(T+G)+1)) - (T+G));

                if (!(row_offset >= -G && row_offset <= G &&
col_offset >= -G && col_offset <= G)) begin
                    calc_addr = get_flat_addr(proc_row + row_offset,
proc_col + col_offset);
                    rd_addr <= calc_addr;
                    train_sum <= train_sum + rd_data;
                end

                if (read_count == total_training_cells-1) begin
                    rd_addr <= get_flat_addr(proc_row, proc_col);
                    cut_value <= rd_data;
```

```
                    threshold <= (train_sum * THRESH_SCALE) /
(total_training_cells - (2*G+1)*(2*G+1)) >>> 15;

                    if (cut_value > threshold) begin
                        detection_valid <= 1;
                        range_bin <= proc_row;
                        doppler_bin <= proc_col;
                    end

                    if (proc_col == DOPPLER-1) begin
                        proc_col <= 0;
                        proc_row <= proc_row + 1;
                    end else begin
                        proc_col <= proc_col + 1;
                    end
                    read_count <= 0;
                end else begin
                    read_count <= read_count + 1;
                end
            end else begin
                if (proc_col == DOPPLER-1) begin
                    proc_col <= 0;
                    proc_row <= proc_row + 1;
                end else begin
                    proc_col <= proc_col + 1;
                end
                read_count <= 0;
            end
        end
    end

    assign total_training_cells = (2*(T+G)+1)*(2*(T+G)+1);
    assign cfar_done = (proc_row == RANGE-1) && (proc_col == DOPPLER-
1);

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////
////////////
```

```
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 04/28/2025 05:33:04 AM
// Design Name:
// Module Name: simple_dual_port_ram
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
////////////


module simple_dual_port_ram #(
    parameter int WIDTH = 16,
    parameter int DEPTH = 32768, // Example: 32k depth
    parameter int ADDR_WIDTH = $clog2(DEPTH)
)(
    input logic clk,

    // Write port
    input logic [ADDR_WIDTH-1:0] wr_addr,
    input logic [WIDTH-1:0] wr_data,
    input logic wr_en,

    // Read port
    input logic [ADDR_WIDTH-1:0] rd_addr,
    output logic [WIDTH-1:0] rd_data
);

    (* ram_style = "block" *) logic [WIDTH-1:0] mem [0:DEPTH-1];

    always_ff @(posedge clk) begin
```

```
        if (wr_en) begin
            mem[wr_addr] <= wr_data;
        end
        rd_data <= mem[rd_addr];
    end

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 04/27/2025 12:50:35 AM
// Design Name:
// Module Name: detection_memory
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module detection_memory #(
    parameter int RANGE_BITS = 8,
    parameter int DOPPLER_BITS = 7,
    parameter int MAX_DETECTIONS = 512
)(
    input logic clk,
    input logic reset,
```

```
    input logic detection_valid,
    input logic [RANGE_BITS-1:0] range_bin_in,
    input logic [DOPPLER_BITS-1:0] doppler_bin_in,

    input logic read_en,
    output logic [RANGE_BITS-1:0] range_bin_out,
    output logic [DOPPLER_BITS-1:0] doppler_bin_out,
    output logic detection_available,
    output logic memory_full
);

    typedef struct packed {
        logic [RANGE_BITS-1:0] range_bin;
        logic [DOPPLER_BITS-1:0] doppler_bin;
    } detection_t;

    detection_t mem [0:MAX_DETECTIONS-1];

    logic [$clog2(MAX_DETECTIONS)-1:0] write_ptr;
    logic [$clog2(MAX_DETECTIONS)-1:0] read_ptr;
    logic [$clog2(MAX_DETECTIONS+1)-1:0] detection_count;

    // === Unified Write/Read Logic ===
    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
            write_ptr <= 0;
            read_ptr <= 0;
            detection_count <= 0;
        end else begin
            if (detection_valid && !memory_full) begin
                mem[write_ptr].range_bin <= range_bin_in;
                mem[write_ptr].doppler_bin <= doppler_bin_in;
                write_ptr <= write_ptr + 1;
                detection_count <= detection_count + 1;
            end
            if (read_en && detection_available) begin
                read_ptr <= read_ptr + 1;
                detection_count <= detection_count - 1;
            end
        end
    end
```

```
    // === Outputs ===
    assign range_bin_out = mem[read_ptr].range_bin;
    assign doppler_bin_out = mem[read_ptr].doppler_bin;
    assign detection_available = (detection_count != 0);
    assign memory_full = (detection_count == MAX_DETECTIONS);

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 03/14/2025 09:06:14 AM
// Design Name:
// Module Name: control_unit
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module control_unit(
    input logic clk, reset,

    // Pre-Range FFT buffer status
    input logic is_empty_prerangefftRAM_real,
is_full_prerangefftRAM_real,
```

```
    input logic is_empty_prerangefftRAM_imag,
is_full_prerangefftRAM_imag,

    // DFT 256 status
    input logic dft256_load_done, dft256_compute_done,

    // Ping-Pong buffer status
    input logic pingpong_full,

    // DFT 128 status
    input logic dft128_load_done, dft128_compute_done,

    // CFAR status
    input logic cfar_done,

    // Control outputs
    output logic write_en_prerangefftRAM_real,
read_en_prerangefftRAM_real,
    output logic write_en_prerangefftRAM_imag,
read_en_prerangefftRAM_imag,

    output logic load_en_dft256, ctrl_flag_dft256,
    output logic load_en_dft128, ctrl_flag_dft128,

    output logic load_en_pingpong,
    output logic enable_squarelaw,
    output logic enable_cfar,
    output logic read_en_pingpong
);

    typedef enum logic [3:0] {
        IDLE,
        BUFFERING,
        LOAD_TO_DFT256,
        COMPUTE_DFT256,
        LOAD_TO_PINGPONG,
        LOAD_FROM_PINGPONG,
        LOAD_TO_DFT128,
        COMPUTE_DFT128,
        SQUARE_LAW,
```

```
        CFAR_PHASE
    } state_t;

    state_t curr_state, next_state;

    always_ff @(posedge clk or posedge reset) begin
        if (reset)
            curr_state <= IDLE;
        else
            curr_state <= next_state;
    end

    always_comb begin
        next_state = curr_state;
        case (curr_state)
            IDLE: next_state = BUFFERING;

            BUFFERING:
                if (is_full_prerangefftRAM_real &&
is_full_prerangefftRAM_imag)
                    next_state = LOAD_TO_DFT256;

            LOAD_TO_DFT256:
                if (dft256_load_done)
                    next_state = COMPUTE_DFT256;

            COMPUTE_DFT256:
                if (dft256_compute_done)
                    next_state = LOAD_TO_PINGPONG;

            LOAD_TO_PINGPONG:
                if (pingpong_full)
                    next_state = LOAD_FROM_PINGPONG;

            LOAD_FROM_PINGPONG:
                next_state = LOAD_TO_DFT128;

            LOAD_TO_DFT128:
                if (dft128_load_done)
                    next_state = COMPUTE_DFT128;
```

```
            COMPUTE_DFT128:
                if (dft128_compute_done)
                    next_state = SQUARE_LAW;

            SQUARE_LAW:
                next_state = CFAR_PHASE;

            CFAR_PHASE:
                if (cfar_done)
                    next_state = LOAD_FROM_PINGPONG;

            default: next_state = IDLE;
        endcase
    end

    // Outputs
    always_comb begin
        // Default values
        write_en_prerangefftRAM_real = 0;
        read_en_prerangefftRAM_real = 0;
        write_en_prerangefftRAM_imag = 0;
        read_en_prerangefftRAM_imag = 0;

        load_en_dft256 = 0;
        ctrl_flag_dft256 = 0;
        load_en_dft128 = 0;
        ctrl_flag_dft128 = 0;
        load_en_pingpong = 0;
        enable_squarelaw = 0;
        enable_cfar = 0;
        read_en_pingpong = 0;

        case (curr_state)
            BUFFERING: begin
                write_en_prerangefftRAM_real = 1;
                write_en_prerangefftRAM_imag = 1;
            end

            LOAD_TO_DFT256: begin
```

```
                read_en_prerangefftRAM_real = 1;
                read_en_prerangefftRAM_imag = 1;
                load_en_dft256 = 1;
            end

            COMPUTE_DFT256: ctrl_flag_dft256 = 1;

            LOAD_TO_PINGPONG: load_en_pingpong = 1;

            LOAD_FROM_PINGPONG: read_en_pingpong = 1;

            LOAD_TO_DFT128: load_en_dft128 = 1;

            COMPUTE_DFT128: ctrl_flag_dft128 = 1;

            SQUARE_LAW: enable_squarelaw = 1;

            CFAR_PHASE: enable_cfar = 1;
        endcase
    end

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer: Muhammad Ahmad Nazir
//
// Create Date: 03/14/2025 07:49:30 AM
// Design Name:
// Module Name: FFT_Processor_Chain_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
```

```
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
////////////

`timescale 1ns/1ps

module FFT_Processor_Chain_tb;

    parameter int Width = 16;

    logic clk;
    logic reset;
    logic read_en_detectmem;
    logic signed [Width-1:0] data_in_real;
    logic signed [Width-1:0] data_in_imag;
    logic [$clog2(256)-1:0] range_bin_out;
    logic [$clog2(128)-1:0] doppler_bin_out;

    // Instantiate DUT
    FFT_Processor_Chain #(
        .Width(Width)
    ) dut (
        .clk(clk),
        .reset(reset),
        .data_in_real(data_in_real),
        .data_in_imag(data_in_imag),
        .read_en_detectmem(read_en_detectmem),
        .range_bin_out(range_bin_out),
        .doppler_bin_out(doppler_bin_out)
    );

    // Memories
    logic signed [Width-1:0] W_real [0:255][0:127] = '{
         '{     288,    220,    381,    333,     70,     56,    177,
26,     26,    155,    174,    273,    185,     28,    196,    147,
278,    196,    379,    177,    429,     47,    253,    379,    195,
189,    249,     52,     61,    303,    107,    108,    146,    237,
261,    146,    353,    507,    236,    295,    170,     79,     46,
49,      3,    133,    206,     79,     29,    148,    110,     91,
```

```
87,     21,    241,    175,    490,    331,    426,    338,    147,
465,     26,    318,    215,    160,    305,    129,    145,    164,
180,     22,     33,    272,     91,     50,    281,    155,     85,
111,    434,    156,    137,    362,    294,     16,    158,    287,
175,     96,     67,      1,     77,     69,     29,    182,    191,
326,    246,    123,    368,    310,    227,    206,     53,    175,
159,      9,     45,    201,     42,     93,     78,    279,    101,
74,    223,    480,    105,    157,    214,    370,    339,    344,
3,    306,    402,    222 },
        ...
        '{      35,    246,     48,      8,    228,    285,    179,
44,    348,    301,    437,    402,    376,    399,    280,    279,
106,    179,     97,    167,     79,    161,     69,     29,     25,
238,    215,    219,     25,    297,    233,    408,    265,    308,
358,    343,     45,    261,    245,    103,     93,    103,    160,
78,    135,    183,     15,     96,    145,    225,    378,    326,
295,    303,    196,    346,    207,    293,    340,    124,    160,
116,     14,    103,    118,    149,     64,    207,    316,    282,
325,    181,    341,    188,    384,    362,    388,    407,    245,
312,    296,    249,    281,     97,    136,     84,     97,    226,
285,    120,    379,    150,    100,     58,    226,    117,    205,
366,    399,    365,    127,    523,    523,    373,      6,    191,
245,     66,     95,     24,    268,    169,     96,    199,     71,
232,    357,    137,    493,    111,    186,    108,    137,    184,
200,      2,    239,     83 }
      };

    logic signed [Width-1:0] W_imag [0:255][0:127] = '{
        '{     171,    383,    286,    252,     39,    367,    172,
167,    180,    123,    319,    226,    324,    132,    346,    110,
56,    247,    186,    234,    311,    159,    201,     24,      4,
120,    387,    363,    340,    394,    212,    230,    189,    190,
156,    271,    115,     43,    201,    106,    211,     45,    149,
63,    240,    200,    264,    296,    205,    193,    423,    119,
56,    181,    117,    395,    289,    144,      0,     18,    128,
32,    107,    305,    163,    223,    210,    137,    433,    427,
235,    474,    284,    154,      8,     70,     20,    182,    280,
228,     59,     70,     63,    206,    303,    171,    280,    113,
443,    310,    206,    351,    489,     99,    243,    100,    109,
92,    106,     88,    112,     16,     91,    180,    183,    316,
299,    173,    420,    313,    229,    421,    106,    274,    223,
```

```
258,    220,     84,     98,    216,     28,    174,    153,     32,
240,     16,    158,    116 },
         ...
         '{     534,    580,    226,    119,    468,    255,    123,
93,     12,    160,    129,     66,    102,     28,    159,    146,
1,    256,    274,    156,    459,    114,    390,    389,    598,
251,    168,     41,     67,     66,    159,     55,     38,    115,
177,     64,     11,     11,     91,    270,    160,      2,    409,
440,    303,    290,    177,    526,     64,     20,    197,     54,
128,     32,    134,      2,    274,     83,    119,    114,    380,
209,     42,    211,    338,    294,    242,    219,    110,    135,
138,    123,    146,    108,     57,     53,     46,     68,     88,
163,    145,    179,    124,    273,    252,    351,    219,    415,
248,    218,    132,     64,    292,    165,    174,     94,     58,
113,    183,    398,     63,    213,    199,    220,    258,     14,
390,    289,     41,    276,    184,    194,    100,     78,    200,
148,    141,     62,     94,    304,      7,    216,    285,    220,
318,    210,    540,    300 }
    };


    real range_grid [0:255] = '{-250.000000000000, -248.046875000000,
-246.093750000000, -244.140625000000, -242.187500000000, -
240.234375000000, -238.281250000000, -236.328125000000, -
234.375000000000, -232.421875000000, -230.468750000000, -
228.515625000000, -226.562500000000, -224.609375000000, -
222.656250000000, -220.703125000000, -218.750000000000, -
216.796875000000, -214.843750000000, -212.890625000000, -
210.937500000000, -208.984375000000, -207.031250000000, -
205.078125000000, -203.125000000000, -201.171875000000, -
199.218750000000, -197.265625000000, -195.312500000000, -
193.359375000000, -191.406250000000, -189.453125000000, -
187.500000000000, -185.546875000000, -183.593750000000, -
181.640625000000, -179.687500000000, -177.734375000000, -
175.781250000000, -173.828125000000, -171.875000000000, -
169.921875000000, -167.968750000000, -166.015625000000, -
164.062500000000, -162.109375000000, -160.156250000000, -
158.203125000000, -156.250000000000, -154.296875000000, -
152.343750000000, -150.390625000000, -148.437500000000, -
146.484375000000, -144.531250000000, -142.578125000000, -
140.625000000000, -138.671875000000, -136.718750000000, -
```

```
134.765625000000, -132.812500000000, -130.859375000000, -
128.906250000000, -126.953125000000, -125.000000000000, -
123.046875000000, -121.093750000000, -119.140625000000, -
117.187500000000, -115.234375000000, -113.281250000000, -
111.328125000000, -109.375000000000, -107.421875000000, -
105.468750000000, -103.515625000000, -101.562500000000, -
99.6093750000000, -97.6562500000000, -95.7031250000000, -
93.7500000000000, -91.7968750000000, -89.8437500000000, -
87.8906250000000, -85.9375000000000, -83.9843750000000, -
82.0312500000000, -80.0781250000000, -78.1250000000000, -
76.1718750000000, -74.2187500000000, -72.2656250000000, -
70.3125000000000, -68.3593750000000, -66.4062500000000, -
64.4531250000000, -62.5000000000000, -60.5468750000000, -
58.5937500000000, -56.6406250000000, -54.6875000000000, -
52.7343750000000, -50.7812500000000, -48.8281250000000, -
46.8750000000000, -44.9218750000000, -42.9687500000000, -
41.0156250000000, -39.0625000000000, -37.1093750000000, -
35.1562500000000, -33.2031250000000, -31.2500000000000, -
29.2968750000000, -27.3437500000000, -25.3906250000000, -
23.4375000000000, -21.4843750000000, -19.5312500000000, -
17.5781250000000, -15.6250000000000, -13.6718750000000, -
11.7187500000000, -9.76562500000000, -7.81250000000000, -
5.85937500000000, -3.90625000000000, -1.95312500000000, 0,
1.95312500000000, 3.90625000000000, 5.85937500000000,
7.81250000000000, 9.76562500000000, 11.7187500000000,
13.6718750000000, 15.6250000000000, 17.5781250000000,
19.5312500000000, 21.4843750000000, 23.4375000000000,
25.3906250000000, 27.3437500000000, 29.2968750000000,
31.2500000000000, 33.2031250000000, 35.1562500000000,
37.1093750000000, 39.0625000000000, 41.0156250000000,
42.9687500000000, 44.9218750000000, 46.8750000000000,
48.8281250000000, 50.7812500000000, 52.7343750000000,
54.6875000000000, 56.6406250000000, 58.5937500000000,
60.5468750000000, 62.5000000000000, 64.4531250000000,
66.4062500000000, 68.3593750000000, 70.3125000000000,
72.2656250000000, 74.2187500000000, 76.1718750000000,
78.1250000000000, 80.0781250000000, 82.0312500000000,
83.9843750000000, 85.9375000000000, 87.8906250000000,
89.8437500000000, 91.7968750000000, 93.7500000000000,
95.7031250000000, 97.6562500000000, 99.6093750000000,
101.562500000000, 103.515625000000, 105.468750000000,
107.421875000000, 109.375000000000, 111.328125000000,
```

```
113.281250000000, 115.234375000000, 117.187500000000,
119.140625000000, 121.093750000000, 123.046875000000,
125.000000000000, 126.953125000000, 128.906250000000,
130.859375000000, 132.812500000000, 134.765625000000,
136.718750000000, 138.671875000000, 140.625000000000,
142.578125000000, 144.531250000000, 146.484375000000,
148.437500000000, 150.390625000000, 152.343750000000,
154.296875000000, 156.250000000000, 158.203125000000,
160.156250000000, 162.109375000000, 164.062500000000,
166.015625000000, 167.968750000000, 169.921875000000,
171.875000000000, 173.828125000000, 175.781250000000,
177.734375000000, 179.687500000000, 181.640625000000,
183.593750000000, 185.546875000000, 187.500000000000,
189.453125000000, 191.406250000000, 193.359375000000,
195.312500000000, 197.265625000000, 199.218750000000,
201.171875000000, 203.125000000000, 205.078125000000,
207.031250000000, 208.984375000000, 210.937500000000,
212.890625000000, 214.843750000000, 216.796875000000,
218.750000000000, 220.703125000000, 222.656250000000,
224.609375000000, 226.562500000000, 228.515625000000,
230.468750000000, 232.421875000000, 234.375000000000,
236.328125000000, 238.281250000000, 240.234375000000,
242.187500000000, 244.140625000000, 246.093750000000,
248.046875000000};
    real doppler_grid [0:127] = '{291.803629460006, 287.244197749693,
282.684766039381, 278.125334329068, 273.565902618755,
269.006470908443, 264.447039198130, 259.887607487818,
255.328175777505, 250.768744067192, 246.209312356880,
241.649880646567, 237.090448936255, 232.531017225942,
227.971585515629, 223.412153805317, 218.852722095004,
214.293290384692, 209.733858674379, 205.174426964067,
200.614995253754, 196.055563543441, 191.496131833129,
186.936700122816, 182.377268412504, 177.817836702191,
173.258404991878, 168.698973281566, 164.139541571253,
159.580109860941, 155.020678150628, 150.461246440315,
145.901814730003, 141.342383019690, 136.782951309378,
132.223519599065, 127.664087888753, 123.104656178440,
118.545224468127, 113.985792757815, 109.426361047502,
104.866929337190, 100.307497626877, 95.7480659165644,
91.1886342062518, 86.6292024959392, 82.0697707856266,
77.5103390753140, 72.9509073650015, 68.3914756546888,
63.8320439443763, 59.2726122340636, 54.7131805237511,
```

```
50.1537488134385, 45.5943171031259, 41.0348853928133,
36.4754536825007, 31.9160219721881, 27.3565902618756,
22.7971585515629, 18.2377268412504, 13.6782951309377,
9.11886342062520, 4.55943171031254, 0, -4.55943171031259, -
9.11886342062518, -13.6782951309378, -18.2377268412504, -
22.7971585515629, -27.3565902618755, -31.9160219721881, -
36.4754536825007, -41.0348853928133, -45.5943171031259, -
50.1537488134385, -54.7131805237511, -59.2726122340637, -
63.8320439443762, -68.3914756546889, -72.9509073650014, -
77.5103390753140, -82.0697707856266, -86.6292024959392, -
91.1886342062518, -95.7480659165644, -100.307497626877, -
104.866929337190, -109.426361047502, -113.985792757815, -
118.545224468127, -123.104656178440, -127.664087888752, -
132.223519599065, -136.782951309378, -141.342383019690, -
145.901814730003, -150.461246440315, -155.020678150628, -
159.580109860941, -164.139541571253, -168.698973281566, -
173.258404991878, -177.817836702191, -182.377268412504, -
186.936700122816, -191.496131833129, -196.055563543441, -
200.614995253754, -205.174426964067, -209.733858674379, -
214.293290384692, -218.852722095004, -223.412153805317, -
227.971585515629, -232.531017225942, -237.090448936255, -
241.649880646567, -246.209312356880, -250.768744067192, -
255.328175777505, -259.887607487818, -264.447039198130, -
269.006470908443, -273.565902618755, -278.125334329068, -
282.684766039381, -287.244197749693};

    // === Internal Variables ===
    logic [3:0] detection_count;
    int limit = 4;
    int row, col;
    int flat_index;
    real est_range;
    real est_velocity;

    // === Clock Generation ===
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // 100 MHz clock
    end
```

```
// === Stimulus Process ===
initial begin
    // Initialization
    reset = 1;
    read_en_detectmem = 0;
    data_in_real = 0;
    data_in_imag = 0;
    #100;
    reset = 0;

    // Feed input samples
    for (row = 0; row < 256; row++) begin
        for (col = 0; col < 128; col++) begin
            data_in_real <= W_real[row][col];
            data_in_imag <= W_imag[row][col];
            @(posedge clk); // 1 clock per sample
        end
    end

    // Wait for processing inside DUT
    #327800;

    // Soft Test: Read back detections
    repeat (10) begin
        @(posedge clk);
        read_en_detectmem <= 1;
        @(posedge clk);
        read_en_detectmem <= 0;
    end
end



// === Monitor Output ===
always_ff @(posedge clk) begin
    if (reset) begin
        detection_count <= 0;
    end else if (read_en_detectmem) begin
        if (detection_count < limit) begin
            est_range = range_grid[range_bin_out];
            est_velocity = doppler_grid[doppler_bin_out];
```

```
                $display("Detection #%0d -> Range = %0.2f meters,
Velocity = %0.2f m/s", detection_count, est_range, est_velocity);
                detection_count <= detection_count + 1;
            end
            $finish;
        end
    end

endmodule
```